

Row Reduction Techniques for n -Party Garbling

Kelong Cong^{1*}, Emmanuela Orsini², Erik Pohle³, and Oliver Zajonc³

¹ Zama, Paris, France

kelong.cong@zama.ai

² Bocconi University, Milan, Italy

emmanuela.orsini@unibocconi.it

³ COSIC, KU Leuven, Leuven, Belgium

{erik.pohle,oliver.zajonc}@esat.kuleuven.be

Abstract. Recent advancements in maliciously secure garbling have significantly improved the efficiency of constant-round multi-party computation. Research in the field has primarily focused on reducing communication complexity through row reduction techniques and improvements to the preprocessing phase with the use of simpler correlations.

In this work, we present two contributions to reduce the communication complexity of state of the art multi-party garbling with an arbitrary number of corruptions. First, we show how to achieve full row reduction for n -party garbled circuits in HSS17-style protocols (Hazay et al., Asiacrypt'17 & JC'20) and authenticated garbling (Yang et al., CCS'20), reducing the size of the garbled circuit by 25% from $4n\kappa$ to $3n\kappa$ and from $(4n - 6)\kappa$ to $3(n - 1)\kappa$ bits per AND gate, respectively. Achieving row reduction in multi-party garbling has been an open problem which was partly addressed by the work of Yang et al. for authenticated garbling. In our work, we show a full row reduction for both garbling approaches, thus addressing this open problem completely.

Second, drawing inspiration from the work of Dittmer et al. (Crypto 2022), we propose a new preprocessing protocol to obtain the required materials for the garbling phase using large field triples that can be generated with sublinear communication. The new preprocessing significantly reduces the communication overhead of garbled circuits. Our optimizations result in up to a $6\times$ reduction in communication compared to HSS17 and a $2.2\times$ reduction over the state of the art authenticated garbling of Yang et al. for 3 parties in a circuit with 10 million AND gates.

1 Introduction

In secure multi-party computation (MPC), a set of distrusting parties jointly computes a public function while keeping their inputs secret, only learning the final output. MPC protocol families can be classified into either constant-round or non constant-round protocols. In constant-round protocols, typically implemented using garbled-circuits [Yao86], the number of communication rounds is fixed and independent of the function's complexity. In contrast, non-constant-round protocols require a number of interactions that generally scales with the function's non-linear depth. For real-world applications, where parties are spread across different data centres and continents, network latency quickly becomes a significant obstacle, making constant-round protocols particularly appealing.

In this work, we focus on garbled circuit based protocols with active security and all-but-one corruptions. In this setting, both two-party and multi-party protocols tend to suffer from high communication costs. At a high level, all existing protocols in this setting follow a similar structure consisting of three phases: The *function-independent phase*, where the parties generate all the random correlations required for the garbled circuit; the *function-dependent phase*, or *garbling phase*, where the parties jointly create the garbled circuit; and the *online phase*, which is both input-

* This work was partially done while the author was affiliated with COSIC, KU Leuven.

and function-dependent, where after providing their inputs, parties evaluate the garbled circuit without further interaction. Since the online phase is very efficient in terms of communication, research has mainly focused on reducing the size of the garbled circuit, through row reduction techniques, and optimizing the communication of the preprocessing (function-independent) phase to improve overall performance.

While the FreeXOR [KS08] method effectively reduces the cost of XOR gates to zero in both settings, row reduction for AND gates remained a key challenge (which we will now simply refer to as row reduction). Techniques for this were first explored for the two party case [NPS99,KS08,ZRE15] and have now reached a 62% reduction compared to the original circuit size [RR21]. For actively secure authenticated garbling [WRK17a], row reduction was first introduced in the two-party setting [KRRW18] and later partially extended to the multi-party setting [YWZ20]. While [YWZ20] successfully reduces the number of ciphertexts by a constant factor (2κ bit per party), the authentication part of the garbled circuit still requires a *full* table of additional elements over a large field for each pair of parties (specifically, four additional elements in $\mathbb{F}_{2^\kappa}$). This naturally leads to the question of whether it is possible to achieve row reduction without sending any additional element.

In parallel, a significant body of work has focused on reducing the communication cost of the preprocessing phase. Recent work, starting with Dittmer et al. [DILO22], has revisited the challenge of achieving efficient constant-round maliciously secure two-party computation by using *simple correlations*, i.e., sources of correlated randomness that can be securely generated with sub-linear communication complexity and good practical efficiency. In particular, the works [BCG⁺20, BBC⁺24, LXYY25] construct such simple correlations with sublinear communication cost. This motivates the additional question of whether these techniques can be efficiently extended to the multi-party setting.

Our Contribution. In this work, we provide a positive answer to the above questions.

First, we reduce the number of ciphertexts in multi-party garbling from four to three, reducing communication costs per AND gate by $n \cdot \kappa$ bits per party in HSS17-style protocols [HSS20]. We extend this approach to authenticated garbling [WRK17b, YWZ20], achieving full row reduction in this framework. While the best-known partial row reduction [YWZ20] saves a constant factor of 2κ bits per AND gate, still requiring $2\kappa + 4(n - 2)\kappa$ bits, our full row reduction saves a linear factor of $(n - 3)\kappa$ bits per party. Our multi-party authenticated garbling protocol sends $3(n - 1)\kappa$ bits per AND gate. This optimization yields communication benefits for all $n \geq 3$, outperforming prior work. We achieve the reduction by also reducing authentication overhead part of the circuit, an aspect not addressed in previous work. Table 1 shows the communication cost reduction compared to previous works. We achieve an asymptotic reduction of 25% compared to the best previous work [YWZ20]. Further, we prove the security of both multi-party garbling protocols using circular correlation robust PRFs for naturally derived keys, improving upon prior work on authenticated garbling protocols [WRK17a, WRK17b, KRRW18, YWZ20], which were only analyzed in the random oracle model.

Our second contribution is a new preprocessing protocol with improves communication when a large number of AND gates (in one or more circuits) need to be garbled, by generalizing some of the ideas described in [DILO22] for the two-party case to the multi-party setting. The main bottleneck when creating garbled gates is the generation of the sharing of the product $\lambda_u \cdot \lambda_v \cdot \Delta$, where λ_u, λ_v are bits and Δ is a bit-string of length κ . Our new approach avoids expensive bucketing techniques and realizes the bit-bit multiplication $\lambda_u \cdot \lambda_v$ in the same step as obtaining shares of $\lambda_u \cdot \lambda_v \cdot \Delta$ using a single Beaver’s multiplication in a *large* field. While choosing a large field triple (in $\mathbb{F}_{2^\rho}$,

Table 1: Improvement of the garbled circuit size by our row reduction for authenticated garbling compared to related works.

Protocol	Number of parties n									
	3	4	5	10	15	20	25	50	75	100
HSS17 [HSS20]	50%	44%	40%	33%	30%	29%	28%	27%	26%	26%
WRK17 [WRK17b]	43%	43%	43%	43%	43%	43%	43%	43%	43%	43%
YWZ20 [YWZ20]	0%	10%	14%	21%	22%	23%	23%	24%	24%	25%

Table 2: Cost comparison of our preprocessing for the number of parties n and circuit size m , excluding the communication of the garbled circuit itself. We count communication in terms of semi-honest n -party garbled circuits.

Protocol	Comm. Cost (garbled circuits)									
	$m = 2^{20}$	$n = 3$		2^{23}	2^{22}	$n = 4$		2^{23}	2^{24}	2^{25}
Semi-honest BMR [BMR90]	1	1	1	1	1	1	1	1	1	1
HSS17 [HSS20]	$18.38 \times$	$18.38 \times$	$18.38 \times$	$18.38 \times$	$27.57 \times$	$27.57 \times$	$27.57 \times$	$36.77 \times$	$36.77 \times$	$36.77 \times$
WRK17 [WRK17b]	$8.07 \times$	$8.07 \times$	$8.07 \times$	$8.07 \times$	$12.11 \times$	$12.11 \times$	$12.11 \times$	$16.14 \times$	$16.14 \times$	$16.14 \times$
YWZ20 [YWZ20]	$6.51 \times$	$6.51 \times$	$6.51 \times$	$6.51 \times$	$9.76 \times$	$9.76 \times$	$9.76 \times$	$13.02 \times$	$13.02 \times$	$13.02 \times$
This work	$8.01 \times$	$5.63 \times$	$3.98 \times$	$2.84 \times$	$11.56 \times$	$8.17 \times$	$5.79 \times$	$16.17 \times$	$11.42 \times$	$8.1 \times$

to be precise, where ρ is the statistical security parameter) may not sound particularly appealing since opening values now costs ρ instead of 1 bit, [DILO22] noted that multiplication triples in larger fields can be preprocessed with *sublinear* communication. In contrast, the sublinear generation of multi-party authenticated AND triples remains an open problem. We take advantage of this observation to significantly reduce the communication cost of the preprocessing using sublinear communication primitives, in our case based on LPN [BCG⁺20], for the generation of large field triples. In addition to generating triples, we use the n -party vector oblivious evaluation (VOLE) protocol [RS22] to create authenticated bits also with sublinear communication. As a result, while the function-dependent phase is slightly more expensive due to the large field multiplication compared to the state of the art preprocessing, we reduce the total preprocessing cost for circuits with ≈ 5 million AND gates by a factor 4.6 compared to HSS17 [HSS20] and a factor 1.6 compared to the state of the art multi-party authenticated garbling protocol YWZ20 [YWZ20] in the three-party case. For larger circuits, our preprocessing becomes more advantageous, e.g., for 10 million AND gates, we reduce the preprocessing cost by factor 6.4 and 2.2 for HSS17 and YWZ20, respectively.

We summarize the preprocessing results in Table 2. The table presents the communication reduction as a ratio to a semi-honest, n -party garbled GC based on BMR-style garbling [BMR90]. Specifically, we estimate communication costs using computational security $\kappa = 128$, statistical security $\rho = 40$, and a bucket size $B = 3$ for a circuit with m AND gates, assuming a broadcast channel for the number of parties $n = 3, 4, 5$. LPN-related parameters are discussed in Section 5.2. Viewed in this way, achieving active security with our protocol has an overhead equivalent to 2.84

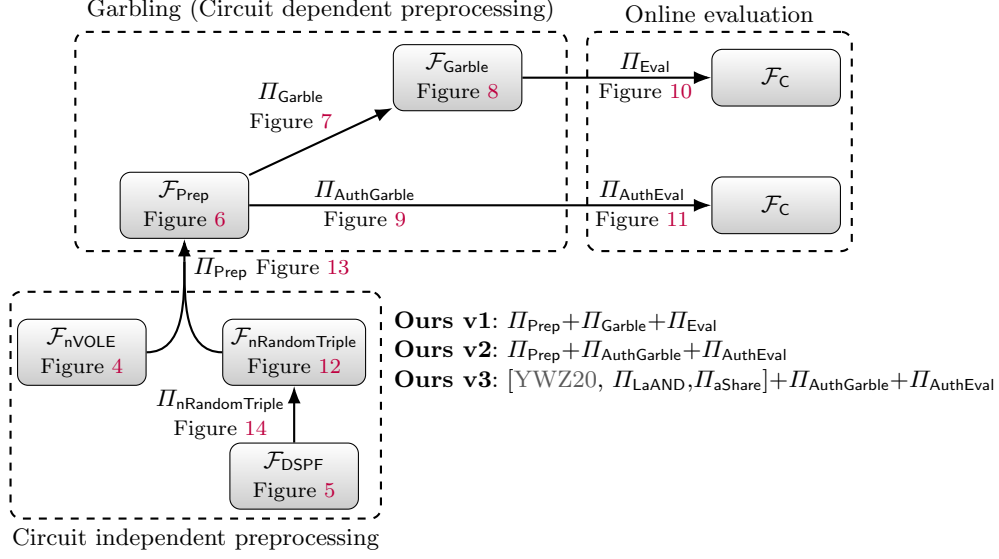


Fig. 1: Overview of our constant-round protocol(s): Relation between our main functionalities and protocols. $\mathcal{F}_{\mathcal{C}}$ denotes the target functionality computing a circuit $\mathcal{C}$.

semi-honest garbled circuits while authenticated garbling costs at least 7 GC and HSS17 18 GC for about 10 million AND gates.

Notably, our two contributions, row reduction and improved preprocessing, are independent. While our sublinear preprocessing techniques introduce a significant setup cost and higher computational overhead, they become advantageous for large circuits, leading to better communication efficiency. The main bottleneck in preprocessing lies in three-party distributed point functions (DPF). However, the high modularity of our protocol ensures that any future improvements in simple correlation techniques will directly benefit our approach.

We denote by **Ours v1** and **Ours v2** the protocols that combine the improved preprocessing with the new row reduction technique in HSS17-style and authenticated garbling, respectively. Further, **Ours v3** refers to our row reduction for authenticated garbling combined with the preprocessing method from [YWZ20]. At the moment, we expect **Ours v3** to outperform all studied protocols in terms of concrete computation cost, as well as in communication efficiency for smaller circuits and larger numbers of parties.

1.1 Technical Overview

Here we describe our results in greater technical detail. Our protocols follow the same three-phase framework we have described before and visualized in Figure 1, which also shows the relationships between the functionalities and protocols that are used in this paper.

Since our constructions assume a circuit-based representation, we fix some conventions and notation we adopt across the paper. We consider binary circuits $\mathcal{C}$ with input wires $\mathcal{I}$, output wires $\mathcal{O}$ and output wires of AND gates $W_{\wedge}$. We denote by $\mathcal{I}_i$ the set of circuit-input wires corresponding to party P_i .

Background on Actively Secure Multi-Party Garbling. Most of the work in multi-party garbled circuits is based on the BMR protocol by Beaver, Micali and Rogaway [BMR90], which has

been recently improved by a large body of works in the case of passive and active security. We will introduce the approach described in [BLO16, HSS20] first, and later we summarize authenticated garbling [WRK17b]. While in Yao's two-party protocol only one party, the garbler, creates the garbled circuit, in BMR all parties generate it in a distributed way. This means that, instead of having a single key associated to each wire of the circuit, in multi-party garbling we have n keys for each wire, one for each party.

At the beginning of the preprocessing step, each party P_i chooses a global correlation $\Delta^i \in \mathbb{F}_2^\kappa$ to support FreeXOR, and, for each wire w that is not the output wire of a XOR gate, samples a random key $k_{w,0}^i$, associated to the value 0, and sets $k_{w,1}^i = k_{w,0}^i \oplus \Delta^i$ for the value 1. Moreover, each P_i samples a random wire mask $\lambda_w^i \in \mathbb{F}_2$, for all the input wires $w \in \mathcal{I}_i$ and output wires of AND gates $w \in W_\wedge$. The actual wire mask for such wires is given by $\lambda_w = \bigoplus_{i \in [n]} \lambda_w^i$. Let g denote an AND gate with input wires u, v and output wire w . Given wire masks $\lambda_u, \lambda_v, \lambda_w$ and wire keys $\{k_{u,a}^i, k_{v,b}^i, k_{w,0}^i\}_{(a,b) \in \{0,1\}^2, i \in [n]}$, parties generate a garbled gate corresponding to the AND gate truth table. It consists of four rows, indexed by the values $(a, b) \in \{0, 1\}^2$ on the input wires. Every row contains n ciphertexts, each of which is encrypted under $2n$ keys as follows,

$$\tilde{g}_{a,b}^j = \left(\bigoplus_{i=1}^n F_{k_{u,a}^i, k_{v,b}^i}(g \| j) \right) \oplus k_{w,0}^j \oplus ((\lambda_u \oplus a) \cdot (\lambda_v \oplus b) \oplus \lambda_w) \cdot \Delta^j, \quad (1)$$

where $\tilde{g}_{a,b}^j$ for $j \in [n]$ represents the j -th ciphertext on the (a, b) -row and F is a double-key PRF. Note that, as FreeXOR asks for every pair of keys $(k_{w,0}^j, k_{w,1}^j)$ to be correlated according to Δ^j , we further need F to be a circular 2-correlation robust PRF [HSS17].

In the online phase, these encrypted truth tables, along with the input and output wire masks, are revealed to all parties to allow local evaluation of the circuit. Upon collecting all the keys and masked inputs, parties can start evaluating the circuit. Given complete sets of input keys $(k_{u,\hat{u}}^1, \dots, k_{u,\hat{u}}^n)$ and $(k_{v,\hat{v}}^1, \dots, k_{v,\hat{v}}^n)$, it is possible to decrypt a single row of garbled AND gates obtaining $(k_{w,\hat{w}}^1, \dots, k_{w,\hat{w}}^n)$. Note that during evaluation each party decrypts the entire row, requiring n^2 PRF evaluations. Once these output keys are obtained, every party P_i can check that the i -th key corresponds to one of its keys $k_{w,0}^i, k_{w,1}^i$ generated in the garbling phase. This check allows: 1) To determine the masked output value, i.e., if $k_{w,\hat{w}}^i = k_{w,0}^i$, P_i sets $\hat{w} = 0$, and $\hat{w} = 1$ otherwise; 2) To ensure active security for the online evaluation. Hazay et al. [HSS20] show that, in order to obtain an actively secure BMR-style protocol, it is enough to generate an *unauthenticated* additive sharing of the garbled circuit, provided that the values $((\lambda_u \oplus a) \cdot (\lambda_v \oplus b) \oplus \lambda_w) \cdot \Delta^j$ in Equation (1) are correctly generated.

Authenticated Garbling. In the authenticated garbling framework, there is a single evaluator P_1 while all other parties $P_i, i \neq 1$, are garblers. Each P_i creates their garbled circuit that is sent to P_1 to evaluate. Unlike in HSS17, where the keys of party P_j are also used to encrypt P_i 's circuit, P_j encrypts authentication information about P_i 's construction in P_j 's circuit.

Concretely, each garbler $P_i, i \neq 1$, obtains authenticated shares $\llbracket r_{ab} \rrbracket$ for $a, b \in \{0, 1\}$ where $r_{ab} = (\lambda_u \oplus a)(\lambda_v \oplus b) \oplus \lambda_w$, and computes its garbled circuit as

$$\tilde{g}_{ab}^i := H(k_{u,a}^i, k_{v,b}^i) \oplus (r_{ab}^i \| k_{w,0}^i \oplus \langle r_{ab} \Delta^i \rangle_i \| \{ \langle r_{ab} \Delta^j \rangle_i \}_{j \neq i}) ,$$

where r_{ab}^i is the bit share and $\langle r_{ab} \Delta^i \rangle_i$ and $\langle r_{ab} \Delta^j \rangle_i$ are authentication values (n -party additive shares) obtained from $\llbracket r_{ab} \rrbracket$, and H denotes a random oracle with output length $n\kappa + 1$ bit. We denote concatenation as $\|$, extended to concatenating all elements from a set.

During the evaluation phase, P_1 knows active keys $\{k_{u,\hat{u}}^i, k_{v,\hat{v}}^i\}_{i \neq 1}$ for each gate and the external values $\hat{u}, \hat{v}$. For an AND gate, for each party P_i , $i \neq 1$, P_1 obtains the new active key $k_{w,\hat{w}}^i$ and share of the external value $r_{\hat{u}\hat{v}}^i$ by decrypting the $(\hat{u}, \hat{v})$ -th row of P_i 's garbled circuit, i.e.,

$$(r_{\hat{u}\hat{v}}^i \| k_{w,0}^i \oplus \langle r_{\hat{u}\hat{v}} \Delta^i \rangle_i \| \{\langle r_{\hat{u}\hat{v}} \Delta^i \rangle_j\}_{j \neq i}) := H(k_{u,\hat{u}}^i, k_{v,\hat{v}}^i) \oplus \tilde{g}_{\hat{u}\hat{v}}^i,$$

and then reconstructing $k_{w,\hat{w}}^i := k_{w,0}^i \oplus \langle r_{\hat{u}\hat{v}} \Delta^i \rangle_i + \bigoplus_{j \neq i} \langle r_{\hat{u}\hat{v}} \Delta^i \rangle_j$. Note that $\langle r_{\hat{u}\hat{v}} \Delta^i \rangle_j$ is obtained from P_j 's circuit $\tilde{g}_{\hat{u}\hat{v}}^j$ and P_1 knows $\langle r_{\hat{u}\hat{v}} \Delta^i \rangle_1$.

Finally, P_1 computes the new external value $\hat{w} = \bigoplus_{i=1}^n r_{\hat{u}\hat{v}}^i$. It can now use $(\hat{w}, \{k_{w,\hat{w}}^i\}_{i \neq 1})$ to proceed with the next gate. P_1 can further check the correctness of $\hat{w}$ using the authentication information $\langle r_{\hat{u}\hat{v}} \Delta^1 \rangle_i$ obtained from each $\tilde{g}_{\hat{u}\hat{v}}^i$ and its own $\langle r_{\hat{u}\hat{v}} \Delta^1 \rangle_1$.

HSS17 vs. Authenticated Garbling. Both the number of communicated bits per party and the local computation cost is lower in the authenticated garbling approach, making it the preferred approach to constant-round, actively secure computation. However, the symmetry in HSS17 is leveraged in mixed-mode protocols for reactive computation [AOR⁺19] or for efficient oblivious RAM [KY18]. This makes the HSS17-based approach a suitable building block. Since a row reduction in either case leads to improved communication cost, we study both row reductions.

On the Difficulty of Row Reduction for Multi-Party Garbling. As explained in [KRRW18], row reduction in multi-party garbling is particularly challenging. In the two-party setting, the garbler can choose output wire labels of AND gates as a free variable (as was clearly expressed in the linear garbling scheme framework [ZRE15]). This degree of freedom gives rise to various techniques to compress the circuit. On the contrary, in the multi-party case, the garblers jointly garble (one or more) circuits. If the output wire label were a function of previous ciphertexts or wire labels, the label would not be known in full to each garbler. Thus to continue garbling the next layer of AND gates, the parties would need to reconstruct and communicate layer by layer which would eliminate the constant-round advantage. To maintain constant-round communication, multi-party garbling protocols must select all output wire labels simultaneously and independently. This constraint makes row reduction challenging to achieve.

Garbled Row Reduction to Three Ciphertexts. We will first show in a simplified presentation on how we achieve a row reduction to three ciphertexts for AND gates. XOR gates don't require communication, as in previous work. We use $u, v \in \{0, 1\}$ to denote the semantic value on the input wires to an AND gate and call $\lambda_u, \lambda_v \in \{0, 1\}$ the secret masks that hide the semantic value. Then w and λ_w denote the semantic value and secret mask of the output wire, respectively. The value $\hat{u} = (u \oplus \lambda_u)$ is the observed value for the input wire during the online phase (similar for $\hat{v}, \hat{w}$). We compute an AND gate as follows

$$uv \oplus \lambda_w = (u \oplus \lambda_u)v \oplus (v \oplus \lambda_v)\lambda_u \oplus \lambda_u\lambda_v \oplus \lambda_w,$$

which is split into three computations: $\text{prod}_1 = (u \oplus \lambda_u)v$, $\text{prod}_2 = (v \oplus \lambda_v)\lambda_u$ and $\text{masks} = \lambda_u\lambda_v \oplus \lambda_w$. In the two-party, passively secure case, we can use garbling gadgets similar to [HK21] to compute prod_1 and prod_2 using one ciphertext each and masks can be included in the output zero label for free.

In the multiparty case, one might hope to apply the same row reduction technique to both the HSS17-style and authenticated garbling protocol; however, while the underlying high-level algebraic idea is similar in both cases, we could not find a unified framework. Hence we present the corresponding row-reduction technique separately.

Row Reduction for HSS17 Garbling. To generalize the gadgets to the HSS17-style multi-party protocol with active security, we compute $\mathbf{prod}_1$ in a similar, straight-forward extension as in the two party case and obtain one ciphertext

$$C_1 = F_{\mathbf{k}_{u,0}} \oplus F_{\mathbf{k}_{u,1}} \oplus k_{v,0} \oplus \lambda_v \Delta,$$

that can be evaluated as

$$k_{\mathbf{prod}_1} = F_{\mathbf{k}_{u,\hat{u}}} \oplus \hat{u}(C_1 \oplus k_{v,\hat{v}}) = F_{\mathbf{k}_{u,0}} \oplus (u \oplus \lambda_u)v\Delta,$$

where $F_{\mathbf{k}}$ is a function keyed by every party's key (which will be detailed later) used to encrypt the rows. We assume F is a circular correlation robust PRF under naturally derived keys, a similar assumption as in [ZRE15] and give definitions and a concrete instantiation based in fixed-key AES in Section 2.5. The evaluation is correct as $F_{\mathbf{k}_{u,\hat{u}}} \oplus \hat{u}(F_{\mathbf{k}_{u,0}} \oplus F_{\mathbf{k}_{u,1}}) = F_{\mathbf{k}_{u,0}}$. We now observe that $\mathbf{prod}_2 \oplus \mathbf{masks}$ only has one input wire and thus can be realized as a unary gate with two garbled rows/ciphertexts⁴. This way, the output wire label $k_{w,0}$ and $F_{\mathbf{k}_{u,0}}$ can be added to every row in the unary gate, so that the XOR of the row-reduced $\mathbf{prod}_1$ gate with the unary gate produces the desired evaluation key, i.e.,

$$C_2^a = F_{\mathbf{k}_{v,a}} \oplus (\lambda_w \oplus \lambda_u \lambda_v \oplus a \lambda_u) \Delta \oplus k_{w,0} \oplus F_{\mathbf{k}_{u,0}} \quad \text{for } a \in \{0, 1\},$$

which leads to the final evaluation as

$$k_{w,\hat{w}} = F_{\mathbf{k}_{u,\hat{u}}} \oplus \hat{u}(C_1 \oplus k_{v,\hat{v}}) \oplus F_{\mathbf{k}_{v,\hat{v}}} \oplus C_2^{\hat{v}}.$$

Row Reduction for Authenticated Garbling. Turning to the row reduction for authenticated garbling, each garbler now computes $\llbracket r_1 \rrbracket = \llbracket \lambda_u \rrbracket$ and $\llbracket r_{2,a} \rrbracket = \llbracket \lambda_w \oplus \lambda_u \lambda_v \oplus a \cdot \lambda_u \rrbracket$, for $a \in \{0, 1\}$, and sets

$$\begin{aligned} C_1^i &:= H(k_{u,0}^i) \oplus H(k_{u,1}^i) \oplus (r_1^i \| k_{v,0}^i \oplus \langle r_1 \Delta^i \rangle_i \| \{\langle r_1 \Delta^j \rangle_i\}_{j \neq i}) , \\ C_2^{i,a} &:= H(k_{v,a}^i) \oplus H(k_{u,0}^i) \oplus (r_{2,a}^i \| k_{w,0}^i \oplus \langle r_{2,a} \Delta^i \rangle_i \| \{\langle r_{2,a} \Delta^j \rangle_i\}_{j \neq i}) \quad \text{for } a \in \{0, 1\}. \end{aligned}$$

The evaluator P_1 can, similarly to the HSS17 row reduction, obtain $k_{w,\hat{w}}^i$ by computing

$$(\hat{u}r_1^i \oplus r_{2,\hat{v}}^i \| k^i \| \{\langle (\hat{u}r_1 \oplus r_{2,\hat{v}}) \Delta^j \rangle_i\}_{j \neq i}) := H(k_{u,\hat{u}}^i) \oplus H(k_{v,\hat{v}}^i) \oplus \hat{u}C_1^i \oplus C_2^{i,\hat{v}} \text{ for } i \neq 1,$$

and obtains $k_{w,\hat{w}}^i := k^i \oplus \hat{u}k_{v,\hat{v}}^i \oplus \bigoplus_{j \neq i} \langle (\hat{u}r_1 \oplus r_{2,\hat{v}}) \Delta^j \rangle_j$. Similarly, the new external value is $\hat{w} = \bigoplus_{i=1}^n (\hat{u}r_1^i \oplus r_{2,\hat{v}}^i)$. Again, P_1 checks the correctness of $\hat{w}$ using $\langle (\hat{u}r_1 \oplus r_{2,\hat{v}}) \Delta^1 \rangle_i$ from each P_i 's circuit and its own share $\langle (\hat{u}r_1 \oplus r_{2,\hat{v}}) \Delta^1 \rangle_1$.

The simplified row reduction above yields $3n\kappa + 3$ bits of ciphertext per AND gate and uses a random oracle H . In Section 3.2, we give a row reduction that costs at most $3(n-1)\kappa + 1$ bits per AND gate using techniques of [YWZ20] to remove the bit r_1^i and $r_{2,a}^i$, and introducing a batched correctness check at the end that does not necessitate that P_i includes $\langle r_1 \Delta^1 \rangle_i$ and $\langle r_{2,a} \Delta^1 \rangle_i$ in the

⁴ An observant reader may note that $\mathbf{prod}_2$ can also be computed similar to $\mathbf{prod}_1$ at the cost of one ciphertext only. However, if the $\mathbf{masks}$ part is included as part of the output wire label, similar to the two-party case, the garbling will no longer be constant-round since the output wire label now depends on $\mathbf{masks}$. On the other hand, when encrypting $\mathbf{masks}$ at the cost of another ciphertext using the keys $\mathbf{k}_u$ and/or $\mathbf{k}_v$, the reduction to (tweakable) circular correlation robustness is hindered since three ciphertexts have to be constructed from only two oracle queries. We settle with a normal unary gate, thus this solution has the same cost (three ciphertexts).

circuit. We also replace H with an encryption function based on F (the circular correlation robust PRF).

Novel Preprocessing with Low Communication. Next, we give an overview of the novel function-dependent preprocessing protocol that is used to compute $\lambda_u \lambda_v \Delta$. We start with the common observation that an authenticated sharing $\llbracket x \rrbracket$ that uses pairwise information-theoretic MACs to authenticate the shares of each party, can be turned into an additive but no longer authenticated sharing of $x\Delta$ where Δ is the MAC key of a party. We write x^i or $\langle x \rangle$ to denote P_i 's additive share of x .

During the preprocessing, parties hold two keys: $\Delta^i \in \mathbb{F}_{2^\kappa}$ and $\alpha^i \in \mathbb{F}_{2^\rho}$. The former is used as an authentication key for κ -bit relations and is further used as the FreeXOR offset in P_i 's garbled circuit. The latter is used as authentication key for ρ -bit relations. From the *function-independent* preprocessing phase, i.e., when the circuit is not known except for an upper bound on the number of multiplications, we generate multiplication triples of the form: x^i, y^i, z^i and $\{\langle x\alpha^j \rangle, \langle y\alpha^j \rangle, \langle z\alpha^j \rangle\}_{j \in [n]}$ where $x, y \in \mathbb{F}_{2^\rho}$ and $x \cdot y = z$. These correlations can be generated with $\mathcal{O}(\sqrt{m})$ bits of communication for m triples. We modify the SPDZ-style triple PCG from [AS22] to output our required triples. The communication cost is dominated by the setup phase of 3-party DPFs.

In the *function-dependent* preprocessing phase, we now obtain shares of the input wire masks and $\llbracket \lambda_u \rrbracket, \llbracket \lambda_v \rrbracket$ that are input to an AND gate either with sublinear communication cost from n -party VOLE or locally compute them from the output of XOR gates. With Beaver's trick [Bea92] we obtain shares of the product $\lambda_u \cdot \lambda_v$ and $\{\langle \lambda_u \lambda_v \alpha^j \rangle\}_{j \in [n]}$ at the same time costing only 2ρ bits of communication. Two points to note: First, $\langle \lambda_u \cdot \lambda_v \rangle$ is a $\mathbb{F}_{2^\rho}$ -element, however the underlying secret is 0 or 1. We can therefore take the least significant bit of each share to locally convert it into a $\mathbb{F}_2$ sharing. This trick has also been used in [DILO22] in the context of two-party authenticated garbling. Second, we obtained $\langle \lambda_u \lambda_v \alpha^j \rangle$ which is a ρ -bit correlation but for the garbling we require the κ -bit relation $\langle \lambda_u \lambda_v \Delta \rangle$. Thus, in the final step, we extend the labels from ρ to κ bit using n -party VOLE. This can be cheaply done with sending $m + O(\rho)$ bits to extend m correlations. During the whole procedure, we employ cheap consistency checks to ensure correct behaviour. The checks are run among all parties together instead of pairwise to save local computation.

1.2 Related Work

The focus of our work is on reducing the bandwidth in the multi-party setting. As such, we discuss recent advancements on multi-party garbling that are secure against malicious adversaries in the full-threshold setting.

(Malicious) Two-Party Garbling. As many bandwidth reduction techniques originate from the two-party, passively secure setting, we briefly summarize these first. FreeXOR [KS08] reduces the circuit size of XOR gates to zero. In this setting, bandwidth decreased from 4κ [Yao86] to 3κ [NPS99, PSSW09], to 2κ [ZRE15] and to 1.5κ [RR21]. Achieving active security is costly and relies on cut-and-choose on circuit, e.g. [LP12], or gate level [NO09], on zero-knowledge proofs, e.g. [sS11], or on authenticated garbling which we discuss later.

BMR-style Garbling. The BMR approach [BMR90] has been used as the basis to create many actively secure multi-party garbled circuit protocols, e.g., [LPSY19, BLO16, LSS16]. In essence, every party is both a garbler and an evaluator and every party contributes a secret key or mask to the garbled truth table. Hazay, Scholl and Soria-Vazquez [HSS20] introduced a technique that

Table 3: Detailed cost comparison of related work. Let $|\mathcal{I}|$, $|\mathcal{C}|$, and $|\mathcal{O}|$ denote the number of inputs, AND gates, and outputs of the circuit, respectively. Further, $|\mathcal{I}_i|$ represents the number of inputs of party P_i . The parameters c and t are LPN parameters.

Circuit Independent	Communication (bits/party)	Rounds
HSS17 [HSS20]	$(3B^2 + 1)(n - 1) \mathcal{C} (\kappa + \rho) + (n - 1) \mathcal{I} (\kappa + \rho)$	13
WRK17 [WRK17b]	$(3B + \frac{\kappa}{\kappa + \rho}B + 1)(n - 1) \mathcal{C} (\kappa + \rho) + (n - 1) \mathcal{I} (\kappa + \rho)$	9
YWZ20 [YWZ20]	$(4B + 1)(n - 1) \mathcal{C} \kappa + (1 - 1/n) \mathcal{I} \kappa$	8
$\Pi_{\text{nRandomTriple}}$ (Figure 14)	$3n^2ct \log(\mathcal{C}) + 4n^2c^2t^2 \log(2 \mathcal{C}) + n^3c^2t^2 \sqrt{2 \mathcal{C} }$	5
Circuit Dependent		
HSS17 [HSS20]	$2(n - 1) \mathcal{C} + (n - 1) \mathcal{O} + (n - 1) \mathcal{I} $ $+ 4n \mathcal{C} \kappa + (n - 1)\kappa$	2
WRK17 [WRK17b]	$4(n - 1) \mathcal{C} (\kappa + \rho) + 2(n - 1) \mathcal{C} + (n - 1) \mathcal{I} $	2
YWZ20 [YWZ20]	$(4n - 6) \mathcal{C} \kappa + (2n - 1) \mathcal{C} $	2
Ours v1	$3n \mathcal{C} \kappa + (2\rho + 1) \mathcal{C} + \mathcal{O} $	5
Ours v2	$3(n - 1) \mathcal{C} \kappa + (2\rho + 2) \mathcal{C} + \mathcal{O} $	5
Ours v3	$3(n - 1) \mathcal{C} \kappa + 2 \mathcal{C} + \mathcal{O} $	2
Online Phase		
HSS17 [HSS20]	$(n - 1) \mathcal{I} \kappa + \mathcal{I}_i $	2
WRK17 [WRK17b]	$ \mathcal{I} \kappa + \mathcal{I}_i + \mathcal{O} $	3
YWZ20 [YWZ20]	$\max\{ \mathcal{I} \kappa, \mathcal{C} \} + \mathcal{I}_i + \mathcal{O} $	4
Ours v1	$ \mathcal{I} \kappa + \mathcal{I} + \mathcal{O} $	3
Ours v2, Ours v3	$\max\{ \mathcal{I} \kappa, \mathcal{C} \} + \mathcal{I}_i + \mathcal{O} $	4

uses unauthenticated shares in the garbling phase and a check in the online phase to remain actively secure. The communication cost for one AND gate in this scheme is $\mathcal{O}(4n\kappa)$ bit per party. Our protocol (**Ours v1**) follows a similar approach to get active security but with the new row reduction, each AND gate only costs $\mathcal{O}(3n\kappa)$ bit per party.

Authenticated Garbling. Authenticated garbling [WRK17a, WRK17b] achieves active security by secret-sharing the garbled circuit using pairwise MACs. In the two-party setting, bandwidth reduction has been proposed in the form of reducing the circuit size [KRRW18, CWYY23] and the preprocessing cost [DILO22]. Supporting n parties, the cost is $\mathcal{O}(4(n - 1)(\kappa + \rho))$ from [WRK17b] which was reduced to $\mathcal{O}((4n - 6)\kappa)$ in [YWZ20]. Our technique (**Ours v2** and **Ours v3**) reduces this cost to $\mathcal{O}(3(n - 1)\kappa)$.

Garbling with Key-Homomorphic PRFs. Another approach is using a special PRF such that the evaluation cost no longer depends on the number of parties. That is, after opening the garbled circuit (which still depends on n), the evaluation does not depend on n any more. Ben-Efraim et al. [BCO⁺21] and Beck et al. [BGH⁺23] have reduced the ciphertext size for a large number of parties due to the homomorphic properties of LPN-based encryption schemes. Due to the size of LPN ciphertexts and its high computation cost, [HSS20] is still more efficient when the number

of parties is low. For instance, [BCO⁺21] only becomes more efficient when the number of parties is larger than 100. In a concurrent work, MYao [BEBB⁺24] also introduced full row reduction in the multi-party setting. Their primary technique is to create the special PRF by evaluating a PRF within MPC which is traditionally inefficient. The authors overcame this problem by using an “MPC-friendly” PRF. The resulting garbled circuit for an AND gate is 768 bits compared to our best circuit size of $384(n - 1)$ bits. However, preprocessing of doubly-shared values in $\mathbb{Z}_2$ and $\mathbb{Z}_3$ is needed to support the PRF evaluation and the preprocessing protocol runs in $O(\log n)$ rounds. Using concrete numbers, MYao’s communication cost for preprocessing is about $23000n$ bits whereas our approach only requires $1664(n - 1)$ bits. We use the BMR approach to achieve multi party garbling, which allows us to use a standard PRF based on AES, as well as a more standard preprocessing phase in a constant number of rounds. Furthermore, MYao is only secure in the semi-honest setting whereas our work targets the malicious setting. In summary, despite the stronger security setting, our communication cost is about $11\times$ smaller for any number of parties n compared to MYao.

Preprocessing. The multi-party garbling protocols mentioned above all require a cut-and-choose style preprocessing based on TinyOT [BLN⁺21]. These protocols depend on the bucket size parameter $B = O(\rho/\log |C|)$ and typically have complexity $O(Bn^2)$ per AND gate. The preprocessing was improved in [YWZ20] but the cut-and-choose/bucketing approach still exists. In our work, we change the preprocessing to create larger field triples with active security which no longer requires bucketing. Very recently, Li et al. [LXY25] construct sublinear preprocessing for authenticated Boolean triples for the two party setting but their technique alone does not lead to *authenticated* Boolean triples for n parties. Current techniques to extend two-party to multi-party pseudo-random correlation generators (PCGs) for authenticated multiplications require three-party DPFs which, to the best of our knowledge, are missing for the trace-based PCG of [LXY25]. Our work uses $\mathbb{F}_{2^\rho}$ -triple PCGs for which three-party DPFs exist [BKKO20]. On the other hand, if PCG for authenticated n -party boolean triples becomes efficient in the future, it can be immediately combined with our work to improve the performance further.

We refer the reader to Table 3 for a detailed comparison of our work with the most relevant state of the art.

2 Preliminaries

In the following, we introduce our notation, the security model, n -party vector oblivious linear evaluation, circular correlation robustness, distributed point functions, the Module LPN assumption and almost universal hashing.

2.1 Notation

We use κ and ρ to denote the computational and statistical security parameter, respectively. We write $[n] = \{1, \dots, n\}$, and use $\odot$ and $\otimes$ to denote component-wise multiplication and outer product, respectively. We use bold lower-case letters $\mathbf{x}$ for column vectors and x_i to denote the i -th component of $\mathbf{x}$. We denote by $\mathcal{P} = \{P_i\}_{i \in [n]}$ the set of parties and use superscript to denote secret values held by a single party, i.e., r^i denotes a value known by party P_i , and subscript to denote individual shares, e.g., $\langle x \rangle_j$ is the share held by party P_j . Uniformly random sampling from a finite set is denoted as $x \leftarrow \$ S$. We write $:=$ to denote assignment and $=$ for equality.

Functionality $\mathcal{F}_{\text{Commit}}$
PARAMETERS: Finite field $\mathbb{F}_{2^\rho}$.
Commit: On input (Commit) by a party P_i , P_i sends $x \in \mathbb{F}_{2^\rho}$ and τ_x to the functionality. The functionality stores (x, i, τ_x) and sends (i, τ_x) to all parties.
Open: On input (Open, i, τ_x) by party P_i , the functionality outputs the stored (x, i, τ_x) to all parties. If P_i is corrupted, it can also send (NoOpen, i, τ_x), then the functionality outputs $\perp, i, \tau_x$ to all parties instead.

Figure 2: Commitment functionality $\mathcal{F}_{\text{Commit}}$, e.g., [DPSZ12].

Functionality $\mathcal{F}_{\text{Coin}}$
PARAMETERS: Finite field $\mathbb{F}_{2^\rho}$.
Random: On input (Random) from all parties, the functionality samples $r \leftarrow \mathbb{F}_{2^\rho}$ and sends r to all parties.

Figure 3: Functionality $\mathcal{F}_{\text{Coin}}$ to generate a public, random value.

We consider Boolean circuits $\mathcal{C}$ consisting of a number of XOR and AND gates. The set $\mathcal{I}_i$ denotes the set of input wires of party P_i and $\mathcal{I} = \bigcup_{i \in [n]} \mathcal{I}_i$. Analogously, we write $\mathcal{O}$ to indicate the set of all output wires and $\mathcal{O}_i$ the output wires of P_i . $W_\wedge$ and $W_\oplus$ denote the set of output wires of AND and XOR gates, respectively, and W denotes all wires, i.e., $W = \mathcal{I} \cup W_\oplus \cup W_\wedge \cup \mathcal{O}$. We write $|W_\wedge|$ to denote the number of AND gates in the circuit.

In the context of the finite fields $\mathbb{F}_2, \mathbb{F}_{2^\rho}$ and $\mathbb{F}_{2^\kappa}$, we use $+$ and $\oplus$ interchangeably. Further, the conversion from/to bit strings of length ρ and κ is implicit for $\mathbb{F}_{2^\rho}$ and $\mathbb{F}_{2^\kappa}$, respectively. We write $x \| y$ to denote concatenation of bit strings, and $x \| \{y_j\}_{j \in S}$ is understood as concatenation of x and all elements y_j from S in some fixed order. For $x \in \mathbb{F}_{2^\kappa}$, we write $\lfloor x \rfloor \in \mathbb{F}_{2^\rho}$ to denote the conversion by keeping the ρ least significant bits of x .

2.2 Security and Network Model

The protocols presented in this work are proved secure in the Universal Composability framework of Canetti [Can01]. We consider security against a static, malicious adversary who corrupts a subset $\mathcal{C} \subset \{P_1, \dots, P_n\}$, $|\mathcal{C}| \leq n - 1$, of parties at the beginning of the protocol. We assume all parties are connected via secure and authenticated point-to-point channels and via a broadcast channel. The default method of communication is through authenticated channels, unless otherwise specified. Note that in practice, these can all be implemented with standard techniques. For broadcast, a simple 2-round protocol suffices since we allow abort.

In our protocols we will use the following standard functionalities $\mathcal{F}_{\text{Commit}}$ in Figure 2 and $\mathcal{F}_{\text{Coin}}$ in Figure 3.

2.3 n -Party VOLE Correlations

In this work, we rely on a multi-party variant of the subfield *vector oblivious linear evaluation* (VOLE) correlations. We use the multi-party VOLE functionality $\mathcal{F}_{\text{nVOLE}}$ (Figure 4) described in [RS22], where every pair of parties (P_i, P_j) is given a VOLE instance $M[\mathbf{u}^i] = K[\mathbf{u}^i] + \mathbf{u}^i \cdot \Delta^j$, with $\mathbf{u}^i \in \mathbb{F}_2^m$, $\Delta^j \in \mathbb{F}_{2^\kappa}$, and in addition it is guaranteed that, for all $i \in [n]$, the same Δ^i is used

Functionality $\mathcal{F}_{\text{nVOLE}}$	
PARAMETERS: Finite fields $\mathbb{F}_2, \mathbb{F}_{2^\kappa}$, number of parties n, number of correlations m. An expansion function $\text{Expand} : S \rightarrow \mathbb{F}_2^m$ with seed space S.	
Initialise:	On receiving (Init) from P_i , for $i \in [n]$, sample $\Delta^i \leftarrow \mathbb{F}_{2^\kappa}$, send it to P_i , and ignore all subsequent Init commands from P_i .
Extend:	On input (Extend) from all parties P_i : 1. Sample $\text{seed}^i \leftarrow S$, for each $P_i, i \in [n]$. 2. Compute $\mathbf{u}^i := \text{Expand}(\text{seed}^i)$ for $i \in [n]$. 3. Sample $K_i^j[\mathbf{u}^i]$ for $i \in [n], j \neq i$ and compute $M_j^i[\mathbf{u}^i] = \mathbf{u}^i \cdot \Delta^j + K_i^j[\mathbf{u}^i]$. 4. Output $(\text{seed}^i, \{M_j^i[\mathbf{u}^i]\}_{j \neq i}, \{K_j^i[\mathbf{u}^j]\}_{j \neq i})$ to P_i, for $i \in [n]$.
Corrupt parties:	A corrupt P_i can choose Δ^i and seed^i . It can also choose $M_j^i[\mathbf{u}^i]$ (the value $K_i^j[\mathbf{u}^i]$ given to P_j is recomputed accordingly) and $K_i[\mathbf{u}^j]$.
Global key query:	If P_i is corrupted, receive guess, Δ' from $\mathcal{A}$ with $\Delta' \in \mathbb{F}_{2^\kappa}^n$. If $\Delta = \Delta'$ where $\Delta = (\Delta^1, \dots, \Delta^n)$, send success to P_i and ignore any subsequent global key query. Else, send (abort, Δ) to P_i , abort to the other parties P_j and abort .

Figure 4: The n -party VOLE functionality [RS22].

across multiple calls of the **Extend** command and the same $\mathbf{u}^i$ is used for every pair $(P_i, P_j), j \neq i$. Note that the functionality outputs seed^i to P_i that can be later used as an input to a programmable VOLE [RS22].

Instantiation and Setup Communication Cost. The instantiation is given in [RS22]. On a high level, every pair of parties setup and execute a programmable VOLE where the input seed from every party is fixed. Then a consistency check involving a random linear combination is performed to ensure the same seed is used by every party. In practice, every pair of parties needs to perform a distributed setup to generate the VOLE correlation. This setup phase can be considered as an input-independent preprocessing phase. The state of the art protocol for performing distributed setup in the malicious setting for VOLE can be found in [WYKW21]. The primary cost for setting up VOLE correlations is the distributed setup for programmable pseudorandom functions (PPRF), which involves $\log(m)$ parallel OTs to create the punctured key where m is the number of correlations, which in turn depends on the LPN instantiation. The dual version of LPN uses more computation and less communication and the primal version uses more communication and less computation. Without accounting for the exact LPN parameters, the asymptotic cost is $\tilde{O}(\kappa \cdot \log m)$ and the protocol runs in a constant number of rounds. It was shown in [WYKW21] that the concrete setup cost for 10 million correlations is 1.1 MB.

2.4 Secret-Shared Representation

We use $\langle x \rangle$ to denote that $x \in \mathbb{F}$ is additively secret shared between parties, i.e., $\langle x \rangle = \langle x \rangle_1 + \dots + \langle x \rangle_n$, where P_i holds $\langle x \rangle_i$. We use x^i to denote a value known by party P_i , sometimes this can be a share.

We use additive shares authenticated via pairwise MACs denoted as $\llbracket x \rrbracket$,

$$\llbracket x \rrbracket = (\langle x \rangle_i, \{M_j^i[x], K_j^i[x]\}_{j \neq i})_{i \in [n]}, \quad M_j^i[x] = K_i^j[x] + \langle x \rangle_i \cdot \Delta^j,$$

where each party P_i holds $n - 1$ MACs $M_j^i[x]$ on $\langle x \rangle_i$ under each party's global key Δ^j and $n - 1$ keys $K_j^i[x]$ that authenticate the other party's share $\langle x \rangle^j$.

Note that the parties can locally compute an additive share of $x\Delta^i$ (for any $i \in [n]$) as follows. P_i sets $\langle x\Delta^i \rangle_i := \langle x \rangle_i \Delta^i + \sum_{i \neq j} K_j^i[x]$ and all other P_j , $j \neq i$ set $\langle x\Delta^i \rangle_j := M_i^j[x]$. Note that the additive shares of $x\Delta^i$ are not authenticated in any way.

We can use $\mathcal{F}_{\text{nVOLE}}$ (Figure 4) to obtain $\llbracket \mathbf{u} \rrbracket$ for random, secret shared $\mathbf{u} \in \mathbb{F}_2^m$ that is unknown to all parties.

2.5 Circular Correlation Robustness Under Naturally Derived Keys

To instantiate our protocols, we adapt the definition of *circular correlation robustness under naturally derived keys* (ccrnd) [ZRE15] given for hash functions to single-key PRFs. The notion crnd is a weaker version of tweakable circular correlation robustness [GKWY20] and is sufficient for garbled circuits. Indeed, during garbling, the tweaks are not adversarially chosen, but instead they are generated by honest parties. We consider a function $F : \{0, 1\}^\kappa \times \{0, 1\}^\kappa \rightarrow \{0, 1\}^\kappa$. For $\Delta \in \{0, 1\}^\kappa$, we define the oracle $\mathcal{O}_\Delta(k, i, g, b) = F_{k \oplus \Delta}(i \| g) \oplus b \cdot \Delta$. The outcome of $\mathcal{O}_\Delta$ is compared with the output of Rand defined as follows: Rand(k, i, g, b) returns the same string answered before for every previously queried inputs and a random $r \leftarrow \{0, 1\}^\kappa$ otherwise.

A sequence of queries $\mathcal{Q}_1, \dots, \mathcal{Q}_q$, is said to be natural if each query $\mathcal{Q}_\iota$ with response k_ι is one of the following:

- $k_\iota \leftarrow \{0, 1\}^\kappa$, or
- $k_\iota = k_{\iota_1} \oplus k_{\iota_2}$ where $\iota_1 \leq \iota_2 < \iota$, or
- $k_\iota = F_{k_{\iota_1}}(\iota)$ where $\iota_1 < \iota$, or
- $k_\iota = \mathcal{O}(k_{\iota_1}, \iota_1, b)$ where $\iota_1 < \iota$.

Definition 1 (Circular Correlation Robustness for Natural Keys [ZRE15]). Let F be a single-key PRF and $\mathcal{Q} = (\mathcal{Q}_1, \dots, \mathcal{Q}_q)$ a sequence of natural queries. We define two experiments: $\text{ideal}_\mathcal{Q}$, where the distinguisher $\mathcal{D}$ is given oracle access to Rand $^\mathcal{Q}$, and $\text{real}_\mathcal{Q}$, where the oracle $\mathcal{O}$ is set to be $\mathcal{O}_\Delta^\mathcal{Q}$, for some uniformly sampled $\Delta \in \{0, 1\}^\kappa$. Both the experiments define a distribution over values $k_1, \dots, k_q$ and are defined by executing the operation in $\mathcal{Q}$ in order. We say that F is crnd if for any distinguisher $\mathcal{D}$ there exists a negligible function $\text{negl}(\kappa)$ such that $\text{Adv}_\mathcal{D}^{\text{ccrnd}}(1^\kappa) \leq \text{negl}(\kappa)$, where

$$\text{Adv}_\mathcal{D}^{\text{ccrnd}}(1^\kappa) = \left| \Pr_{\{k_i\} \leftarrow \text{ideal}_\mathcal{Q}} [\mathcal{D}^{\text{Rand}^\mathcal{Q}}(\{k_i\}) = 1] - \Pr_{\{k_i\} \leftarrow \text{real}_\mathcal{Q}} [\mathcal{D}^{\mathcal{O}_\Delta^\mathcal{Q}}(\{k_i\}) = 1] \right|.$$

We write $\mathcal{D}^\mathcal{O}$ to denote that $\mathcal{D}$ has access to oracle $\mathcal{O}$. Note that $\mathcal{D}$ always has access to F . In addition, the distinguisher is not allowed to query both $\mathcal{O}(k, i, g, 0)$ and $\mathcal{O}(k, i, g, 1)$ for any values k, i, g since this would allow to trivially recover Δ . Any distinguisher respecting this restriction is said to make *legal queries*.

Instantiation. We instantiate F in the random permutation model with the construction $F_x(t) = \pi(\sigma(S \oplus x \oplus t)) \oplus \sigma(S \oplus x \oplus t)$ from [GKWY20, Theorem 3 and 5] with $\sigma(x_L \| x_R) = x_R \oplus x_L \| x_L$ (from [CLL⁺14]) and π as fixed-key AES. Note that the uniformly random value S is chosen by each party for their own function F , fixed for the whole computation and communicated together with the garbled circuit.

2.6 2- and 3-party Distributed (Sum of) Point Functions

We make use of two- and three-party distributed sum of point functions. Let $f_{\omega, \beta}^t$ be the sum of point function where $f_{\omega, \beta}^t(\omega_\ell) = \beta_\ell$ for $\ell \in [t]$ and $f(x) = 0$ for all other inputs. We call $\omega \in D^t$ the special positions in the input domain D and $\beta \in \mathbb{F}_{2^\rho}^t$ the special values for each position.

Definition 2 (2-DSPF [GI14]). A two-party distributed sum of point functions is a pair of algorithms $(\text{Gen}, \text{Eval})$ with the fixed, public parameters input domain $D \in \mathbb{N}$ and number of special points $t \in \mathbb{N}$ where

- $\text{Gen}(1^\kappa, \omega, \beta)$ takes t special positions $\omega \in D^t$ and special values $\beta \in \mathbb{F}_{2^\rho}^t$ and outputs two keys K^1, K^2 .
- $\text{Eval}(K^i, x)$ takes a key K^i and position $x \in D$ and outputs $\langle f_{\omega, \beta}^t(x) \rangle_i \in \mathbb{F}_{2^\rho}$.

A 2-DSPF is correct if for all ω, β, t, x , $\text{Eval}(K^1, x) + \text{Eval}(K^2, x) = f_{\omega, \beta}^t(x)$ when K^1, K^2 are obtained from Gen .

Intuitively, security requires that a single key K^i does not reveal any information about the other key, ω or β beyond what is implied by the output of $\text{Eval}(K^i, \cdot)$. When Gen is executed in a distributed manner among two parties, ω and β are two-party, additive secret-shared inputs to the computation.

Definition 3 (3-DSPF [BKKO20]). A three-party distributed sum of point function is defined similarly to a 2-DSPF but Gen outputs three keys K^1, K^2, K^3 and correctness requires that

$$\text{Eval}(K^1, x) + \text{Eval}(K^2, x) + \text{Eval}(K^3, x) = f_{\omega, \beta}^t(x).$$

The security intuition for 3-DSPF is similar to 2-DSPF. Any two keys don't reveal information about the missing key, ω or β beyond the output of Eval on the two keys. Note that when Gen is distributed among three parties, P_i, P_j and P_k , our protocols input a *two-party* sharing of ω among P_i and P_j but a *three-party* sharing of β . Naturally, ω is leaked if P_i and P_j are both corrupted by the adversary.

We make use of 2- and 3-party DSPFs with an actively secure distributed Gen . The functionality $\mathcal{F}_{\text{DSPF}}$ depicted in Figure 5 has two commands that output the result of Eval on all $x \in D$ of a 2- or 3-party DSPF to each party. The key generation is implicit.

Instantiation. We instantiate a DSPF as a sum of t distributed point functions (DPF) with *sublinear* communication cost and *constant-round* complexity during the distributed Gen . The 2-party DPF can be instantiated using the construction of [BGIK22]. The 3-party DPF is instantiated following [BKKO20, AS22] with some minor modifications since we have binary fields. In this case, to achieve constant-round Gen , we have two options: we can either eliminate iterations entirely, as suggested in [AS22], which increases communication cost from $\sqrt[4]{m}$ to $\sqrt{m}$; or we can terminate the iteration phase after a fixed number of iterations.

2.7 Module-LPN

The Module-LPN assumption [BCG⁺20] informally states that for a ring $R := \mathbb{F}[X]/(F(X))$ for finite field $\mathbb{F}$ and polynomial $F(X)$ of degree m , the inner product $\sum_{i=1}^c a_i \cdot e_i$ is indistinguishable

Functionality $\mathcal{F}_{\text{DSPF}}$

PARAMETERS: Finite field $\mathbb{F}_{2^\rho}$, number of points t , output length m with $m = 2^{m'}$.

Input: On input (Input, id_x, D) from party P_i , the functionality receives $x \in D$ from P_i where D is the domain and stores (id_x, x, D) . It returns to all parties a handle $[x]$.

Mult: On input (Multiply, $[\beta], [\alpha]$) from all parties, the functionality retrieves $\beta \in \mathbb{F}_{2^\rho}^t$ and $\alpha \in \mathbb{F}_{2^\rho}$. The functionality computes $\beta' := \beta\alpha$, stores the result and returns a handle $[\beta']$ to all parties.

OuterSum: On input (OuterSum, $[\omega], [\eta]$) from two parties P_i, P_j , the functionality retrieves $\omega \in [m]^t$ and $[\eta] \in [m]^t$.

- If $P_i, P_j \in \mathcal{C}$, it sends ω, η to the adversary and stops.
- Otherwise, it computes the outer sum $\omega \boxplus \eta := (\omega_1 + \eta_1, \dots, \omega_1 + \eta_t, \omega_2 + \eta_1, \dots, \omega_t + \eta_1, \dots, \omega_t + \eta_t) \in [2m]^{t^2}$ where $+$ denotes integer addition. The functionality stores the result and returns a handle $[\omega \boxplus \eta]$ to all parties.

OuterProduct: On input (OuterProd, $[\beta], [\gamma]$) from two parties P_i, P_j , the functionality retrieves $\beta \in \mathbb{F}_{2^\rho}^t$ and $[\gamma] \in \mathbb{F}_{2^\rho}^t$.

- If $P_i, P_j \in \mathcal{C}$, it sends β, γ to the adversary and stops.
- Otherwise, it computes the outer product $\beta \otimes \gamma := (\beta_1 \cdot \gamma_1, \dots, \beta_1 \cdot \gamma_t, \dots, \beta_t \cdot \gamma_t) \in \mathbb{F}_{2^\rho}^{t^2}$. The functionality stores the result and returns a handle $[\beta \otimes \gamma]$ to all parties.

2-DSPF: On input (2-DSPF, $t, m, i, j, [\omega], [\beta]$) from two parties P_i, P_j , the functionality retrieves $\omega \in [m]^t$ and $\beta \in \mathbb{F}_{2^\rho}^t$.

- If $P_i, P_j \in \mathcal{C}$, it sends ω, β to the adversary.
- Otherwise, it sets $\mathbf{v}_\ell := (\underbrace{0, \dots, 0}_{\omega_\ell}, \beta_\ell, 0, \dots, 0) \in \mathbb{F}_{2^\rho}^m$, $\ell \in [t]$ and $\mathbf{v} := \sum_{\ell=1}^t \mathbf{v}_\ell$.
- If either $P_i \in \mathcal{C}$ or $P_j \in \mathcal{C}$, the functionality receives $\langle \mathbf{v} \rangle_i$ or $\langle \mathbf{v} \rangle_j$ from the adversary, respectively, and adapts $\langle \mathbf{v} \rangle_j$ or $\langle \mathbf{v} \rangle_i$ accordingly. It also receives t sets $I_\ell \subseteq [m]$, $\ell \in [t]$ from $\mathcal{A}$ and aborts if any $\omega_\ell \in I_\ell$.
- The functionality outputs $\langle \mathbf{v} \rangle_i$ and $\langle \mathbf{v} \rangle_j$ to P_i and P_j , respectively.

3-DSPF: On input (3-DSPF, $t, m, i, j, k, [\omega], [\beta]$) from three parties P_i, P_j and P_k , the functionality retrieves $\omega \in [m]^t$ and $\beta \in \mathbb{F}_{2^\rho}^t$.

- If $P_i, P_j \in \mathcal{C}$, it sends ω to the adversary. If all $P_i, P_j, P_k \in \mathcal{C}$, it sends β to $\mathcal{A}$ as well.
- Otherwise, the functionality sets $\mathbf{v}_\ell := (\underbrace{0, \dots, 0}_{\omega_\ell}, \beta_\ell, 0, \dots, 0) \in \mathbb{F}_{2^\rho}^m$, $\ell \in [t]$ and $\mathbf{v} := \sum_{\ell=1}^t \mathbf{v}_\ell$. It chooses $\langle \mathbf{v} \rangle_i, \langle \mathbf{v} \rangle_j, \langle \mathbf{v} \rangle_k \leftarrow \mathbb{F}_{2^\rho}^m$ under the condition that $\langle \mathbf{v} \rangle_i + \langle \mathbf{v} \rangle_j + \langle \mathbf{v} \rangle_k = \mathbf{v}$.
- For each corrupted party P_σ , the functionality receives $\langle \mathbf{v} \rangle_\sigma \in \mathbb{F}_{2^\rho}^m$ and adapts the shares of the honest parties accordingly. It also receives t sets $I_\ell \subseteq [m]$, $\ell \in [t]$ from $\mathcal{A}$ and aborts if any $\omega_\ell \in I_\ell$.
- It outputs $\langle \mathbf{v} \rangle_\sigma$ to P_σ for $\sigma \in \{i, j, k\}$.

Figure 5: Functionality for 2- and 3-party distributed sum of point functions [AS22].

from a random element in R where $a_1, \dots, a_c \leftarrow \$ R$ and $e_1, \dots, e_c$ are *sparse* polynomials in R with t non-zero coefficients. Note that $\mathbf{a} \in R^c$ is public and can be described with a short seed only. In our protocols, we pick $R := \mathbb{F}_{2^\rho}/(F(X))$ and use this assumption to locally generate a random-looking element in R using short descriptions for $\mathbf{e} \in R^c$ which are independent of the degree m . We use the variant with static leakage where the distinguisher $\mathcal{D}$ is allowed a polynomial number of adaptive guesses on the positions of the non-zero elements in $\mathbf{e}$.

Definition 4 (Module-LPN with static leakage [BCG⁺20]). Let $R := \mathbb{F}[X]/(F(X))$ with finite field $\mathbb{F}$ and polynomial $F(X)$ of degree m and further $\mathcal{HW}_t$ be the distribution of t -sparse polynomials, i.e., $\omega_1, \dots, \omega_t \leftarrow \$ [m]$ and $\beta_1, \dots, \beta_t \leftarrow \$ \mathbb{F}$ such that $e(X) := \sum_{i=1}^t \beta_i X^{\omega_i}$, and for $c \geq 2$ define the following game.

1. The challenger draws $e_1, \dots, e_c \leftarrow \$ \mathcal{HW}_t$.

2. The distinguisher $\mathcal{D}$ is allowed to guess multiple (i, k, P) adaptively where $i \in [c], k \in [t]$ and P is a predicate $P : [m] \rightarrow \{0, 1\}$. If $P(\omega_i[k]) = 0$, the challenger aborts, otherwise it sends success to $\mathcal{D}$ and continues the game.
3. Sample $a_1, \dots, a_{c-1} \leftarrow \$ R$. Set $u_0 := \sum_{i=1}^{c-1} a_i e_i + e_c$ and $u_1 \leftarrow \$ R$. Return $(a_1, \dots, a_{c-1}, u_b)$ where $b \leftarrow \$ \{0, 1\}$ to $\mathcal{D}$.

The R^c -LPN $_t$ problem with static leakage is hard if for every polynomial-time distinguisher $\mathcal{D}$, the probability of guessing b is negligible in the security parameter, i.e.,

$$|\Pr[\mathcal{D}^{G_0} = 1] - \Pr[\mathcal{D}^{G_1} = 1]| \leq \text{negl}(\kappa).$$

Converting into Multiplication Triples. As in previous work [AS22], we note that a random multiplication triple $x, y, z := x \cdot y$ with $x, y \in R$ can be turned into m independent, random multiplication triples $(x_1, y_1, z_1 := x_1 \cdot y_1), \dots, (x_m, y_m, z_m := x_m \cdot y_m)$ with $x_i, y_i \in \mathbb{F}_{2^\rho}$. Let $\zeta_1, \dots, \zeta_m \in \mathbb{F}_{2^\rho}$ be m distinct roots of $F(X)$, then we obtain the i -th triple by evaluating x, y and $x \cdot y$ over ζ_i , i.e., $x_i := x(\zeta_i)$, $y_i := y(\zeta_i)$ and $z_i := z(\zeta_i)$. Since this operation is linear, the parties locally compute the same evaluation when given shares $\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket z \rrbracket$ and obtain $\llbracket x_i \rrbracket, \llbracket y_i \rrbracket, \llbracket z_i \rrbracket$ with the correct correlation.

2.8 Almost Universal Linear Hash Functions

Definition 5 (Almost Universal Linear Hash Function [CDD⁺16]). We say that a family of linear hash functions $\mathcal{H}$ is ϵ -almost universal, if for every non-zero input vector $\mathbf{x} \in \mathbb{F}_{2^\rho}^m$, $\Pr_{\mathbf{H} \leftarrow \$ \mathcal{H}}[\mathbf{H}(\mathbf{x}) = 0] \leq \epsilon$, where $\mathbf{H} \leftarrow \$ \mathcal{H}$ denotes sampling a random function from the family.

The family is ϵ' -almost uniform, if for every non-zero vector $\mathbf{x} \in \mathbb{F}_{2^\rho}^m$ and any $\mathbf{v} \in \mathbb{F}_{2^\rho}$, $\Pr_{\mathbf{H} \leftarrow \mathcal{H}}[\mathbf{H}(\mathbf{x}) = \mathbf{v}] \leq \epsilon'$.

We can instantiate $\mathbf{H}$ as $\mathbf{H}_\chi(\mathbf{x}) = x_1\chi + x_2\chi^2 + \dots + x_m\chi^m$, where $\chi \leftarrow \$ \mathbb{F}_{2^\rho}$, as proposed in [NST17, HSS20, YWZ20]. This construction is $\epsilon' = m/2^s$ -uniform.

3 Multi-Party Garbling with Row Reduction

The parties agree on a circuit to compute and its topological order of gates. Each wire w in party P_i 's circuit has the following invariant

$$k_{w, \hat{w}}^i = k_{w, 0}^i \oplus (w \oplus \lambda_w) \Delta^i,$$

where w is the semantic value of the wire, λ_w is the secret wire mask and $\hat{w} = w \oplus \lambda_w$ is the public evaluation value of the wire. P_i knows Δ^i and $k_{w, 0}^i$. We write $k_{w, 1}^i = k_{w, 0}^i \oplus \Delta^i$. During evaluation, an evaluating party learns one of the two wire labels, namely $k_{w, \hat{w}}^i$.

The circuit is composed of inputs, XOR gates, AND gates and output gates where we assume for simplicity that each party receives distinct outputs (i.e., parties do not share output gates), and values that are output are not used as input to other gates. To prepare for the garbling, the parties step through the circuit in topological order, starting from the input wires. This ensures that when reaching an XOR/AND gate or an output gate, the parties have sufficient information on the incoming wires to that gate to proceed with the computation.

Functionality $\mathcal{F}_{\text{Prep}}$

The functionality runs with parties $P_1, \dots, P_n$ and the adversary.

PARAMETERS: Finite field $\mathbb{F}_{2^\kappa}$, number of parties n , set of corrupt parties $\mathcal{C}$.

Initialise: On input (Init) from all parties, receive $\Delta^i \in \mathbb{F}_{2^\kappa}$ for all $i \in \mathcal{C}$. Sample $\Delta^i \leftarrow \mathbb{F}_{2^\kappa}$, $i \notin \mathcal{C}$, and send it to P_i . Store all the values Δ^i , $i \in [n]$. On input (Init, P_2) from all parties, proceed as above except for P_2 . If $P_2 \notin \mathcal{C}$, sample $\Delta^2 \leftarrow \mathbb{F}_{2^\kappa}$ s.t. $\text{lsb}(\Delta^2) = 1$. If $P_2 \in \mathcal{C}$, receive Δ^2 from $\mathcal{A}$. Ignore all subsequent Init commands from P_i .

RandomBit: 1. On receiving (RandomBit, id_x, i) from all parties, generate m_i^i random bits as follows: if $i \in \mathcal{C}$, receive $\mathbf{x}^i$ from the adversary, otherwise sample $\mathbf{x}^i \in \mathbb{F}_{2^{m_i^i}}$ at random, and then run **Macro** $[\mathbf{x}^i, i]$.
 2. On receiving (RandomBit, id_x) from all parties, receive $\mathbf{x}^i \in \mathbb{F}_{2^m}$, $i \in \mathcal{C}$, from the $\mathcal{A}$, and sample $\mathbf{x}^i \in \mathbb{F}_{2^m}$, $i \notin \mathcal{C}$, at random. Run **Macro** $[\mathbf{x}^i, i]$, $i \in [n]$.
 3. Set $\mathbf{x} = \bigoplus \mathbf{x}^i$ and store $(\mathbf{x}, \text{id}_x)$.

Add: On input (Add, $\text{id}_w, \text{id}_u, \text{id}_v$) compute $\mathbf{w} = \mathbf{u} \oplus \mathbf{v}$, associate $\mathbf{w}$ with id_w .

AND_Output_Mask: On receiving (ANDOutputMask, $\text{id}_x, \text{id}_y, \text{id}_z$), retrieve values Δ^i , for all $i \in [n]$.

1. Compute $\mathbf{z} = \mathbf{x} \odot \mathbf{y}$
2. Receive $\langle \mathbf{z} \Delta^i \rangle_j, j \in \mathcal{C}$ from $\mathcal{A}$, $\forall i \in [n]$
3. Compute $\langle \mathbf{z} \Delta^i \rangle_j, j \notin \mathcal{C}$ such that $\sum_{i \in \mathcal{C}} \langle \mathbf{z} \Delta^i \rangle + \sum_{i \notin \mathcal{C}} \langle \mathbf{z} \Delta^i \rangle = \mathbf{z} \Delta^i$
4. Wait for an input from the adversary. If it sends **continue**, output $\langle \mathbf{z} \Delta^i \rangle_j$ to parties $j \notin \mathcal{C}$, $i \in [n]$, otherwise output $\perp$ and terminate.

OpenValue: On input (OpenValue, id_x, i) from all $i \notin \mathcal{C}$, wait for **continue** or **abort** from the adversary. If it replied **abort**, output $\perp$ to P_i . Otherwise, retrieve and output $\mathbf{x}$ to P_i .

Macro $[\cdot]$: On input (λ^i, i) :

- If $i \in \mathcal{C}$, receive $M_j^i[\mathbf{x}^i] \in \mathbb{F}_{2^\kappa}$ from the adversary and compute $K_i^j[\mathbf{x}^i] = M_j^i[\mathbf{x}^i] + \mathbf{x}^i \Delta^j, \forall j \neq i$.
- Else, receive $K_i^j \in \mathbb{F}_{2^\kappa}$, for $j \in \mathcal{C}$, sample $K_i^j \in \mathbb{F}_{2^\kappa}$, for $j \notin \mathcal{C}$, at random and compute $M_j^i = K_i^j + \mathbf{x}^i \Delta^j$ for $j \in \mathcal{C}$.
- Output $\{M_j^i, K_i^j\}_{j \neq i}$.

Figure 6: Functionality for function-independent preprocessing.

We use $\mathcal{F}_{\text{Prep}}$ in Figure 6 to capture the required preprocessing. Using $\mathcal{F}_{\text{Prep}}$, the parties can obtain shares of random bits $\llbracket \lambda \rrbracket$, or obtain $\llbracket \lambda \rrbracket$ and have a party learn λ as well. Parties can add shares $\llbracket \lambda_u \oplus \lambda_v \rrbracket := \llbracket \lambda_u \rrbracket + \llbracket \lambda_v \rrbracket$ and output a share $\llbracket \lambda \rrbracket$ to a specific party. $\mathcal{F}_{\text{Prep}}$ also allows parties to obtain additive shares $\langle \lambda_u \lambda_v \Delta^i \rangle$ for each $i \in [n]$ given $\llbracket \lambda_u \rrbracket$ and $\llbracket \lambda_v \rrbracket$ using the **ANDOutputMask** command.

Initially, the parties receive the FreeXOR offset $\Delta^i \in \mathbb{F}_{2^\kappa}$ from the preprocessing phase. Then, the parties traverse the circuit in topological order and perform the following for each type of gate.

- *Inputs.* For an input wire w of party P_i , all parties use the preprocessing to sample a random, secret-shared wire mask $\llbracket \lambda_w \rrbracket$, which they open to P_i . In addition, each party samples a random key $k_{w,0}^i \leftarrow \{0, 1\}^\kappa$.
- *XOR gates.* Let (u, v, w) be an XOR gate with incoming wires u and v , and outgoing wire w . The parties hold shares of the incoming wire masks $\llbracket \lambda_u \rrbracket, \llbracket \lambda_v \rrbracket$ and know their own keys $k_{u,0}^i, k_{v,0}^i$. To utilize FreeXOR, the parties compute $\llbracket \lambda_w \rrbracket := \llbracket \lambda_u \rrbracket + \llbracket \lambda_v \rrbracket$, and set $k_{w,0}^i := k_{u,0}^i \oplus k_{v,0}^i$.
- *AND gates.* Let (u, v, w) be an AND gate with incoming wires u and v , and outgoing wire w . The parties use the preprocessing to sample a random, secret-shared wire mask $\llbracket \lambda_w \rrbracket$ which will not be opened. Further each P_i samples the wire label for the outgoing wire, $k_{w,0}^i \leftarrow \{0, 1\}^\kappa$, at random.

- *Outputs.* Let w denote an output wire for P_i . Note that all parties already computed $\llbracket \lambda_w \rrbracket$ since it is either an input, an output of an XOR or an output of an AND gate. The parties open the wire mask λ_w to P_i .

After having obtained shares $\llbracket \lambda_w \rrbracket$ of each wire in the circuit, the garbling parties proceed to construct the garbled circuit ciphertexts including our new row reduction. The protocol in HSS17-style and authenticated garbling differ in the exact construction of the garbled circuit ciphertext, so we present HSS17-style next in Section 3.1, followed by the garbling protocol for authenticated garbling in Section 3.2.

3.1 Row Reduction in HSS17-style Garbling

For each AND gate (u, v, w) with incoming wires u and v , and outgoing wire w , we create unique indices g, g' using, e.g., counters⁵. Then, let $\text{prod}_1 = (u \oplus \lambda_u) \cdot v$ and $\text{prod}_2 = (v \oplus \lambda_v) \cdot \lambda_u$. We compute an AND gate in the following way

$$uv \oplus \lambda_w = (u \oplus \lambda_u)v \oplus (v \oplus \lambda_v)\lambda_u \oplus \lambda_u\lambda_v \oplus \lambda_w,$$

using prod_1 and prod_2 and some preprocessed data. The evaluating parties need to obtain circuits consisting of three rows instead of four like in the standard BMR garbling [BMR90, HSS20], where each row contains n ciphertexts, $(C^1, \dots, C^n)$, and for each $i \in [n]$ the ciphertext takes the form,

$$\begin{aligned} C_{1,g}^i &= \bigoplus_{j=1}^n F_{k_{u,0}^j}(i||g) \oplus \bigoplus_{j=1}^n F_{k_{u,1}^j}(i||g) \oplus k_{v,0}^i \oplus \lambda_v \Delta^i, \\ C_{2,g}^{i,a} &= \bigoplus_{j=1}^n F_{k_{v,a}^j}(i||g') \oplus \bigoplus_{j=1}^n F_{k_{u,0}^j}(i||g) \oplus k_{w,0}^i \oplus (\lambda_w \oplus \lambda_u\lambda_v \oplus a\lambda_u)\Delta^i \quad \text{for } a \in \{0, 1\}. \end{aligned}$$

The function F is a *ccrnd*-secure PRF (cf. Section 2.5).

The PRF construction $\bigoplus_{i=1}^n (F_{k_{u,0}^i} \oplus F_{k_{u,1}^i})$ serves two purposes. First, since P_i knows Δ^i and $k_{v,0}^i$ in the clear, the PRF output from other parties' keys $F_{k_{u,0}^j}$ and $F_{k_{u,1}^j}$ sufficiently hides λ_v . Second, all PRFs use a single key (as opposed to the two-key PRFs used in [BMR90, BLO16, HSS20]), so CCR instantiations [GKW⁺20, CT21] can be used which have high performance due to the use of fixed-key AES-NI. Having unique inputs for F_{k_u} and F_{k_v} for the same gate index g, g' allows instantiations with CCR instead of tweakable CCR (TCCR) due to the natural derived keys property (cf. [ZRE15, GKWY20]). Clearly PRF calls can always be computed locally, even though P_j does not know Δ^i when garbling P_i 's circuit.

The ciphertext $C_{1,g}^i$ can be viewed as a row-reduced half gate that computes prod_1 . This construction is not helpful by itself since the “zero label key” $k_{\text{prod}_1,0}^i$ is $\bigoplus_{j \in [n]} F_{k_{u,0}^j}(i||g)$, a value that is not known to any party at the time of garbling. Conveniently, we can include $k_{\text{prod}_1,0}^i$ into both ciphertexts $C_{2,g}^{i,0}$ and $C_{2,g}^{i,1}$ to cancel it out, leaving only the proper output key $k_{w,0}^i$ as the zero label key. Since $k_{w,0}^i$ is known to P_i beforehand, subsequent keys don't depend on the ciphertexts and therefore we can garble all AND gates in parallel. This ensures the constant-round nature of the garbling phase. The other two ciphertexts $C_{2,g}^{i,0}$ and $C_{2,g}^{i,1}$ serve two purposes. They compute the *unary* gate $\text{prod}_2(v)$ and also correct the half-gate's zero label key. They are constructed in the same way as in HSS17 [HSS20] but encrypted with our single key PRF.

⁵ For example, for the i -th AND gate in the circuit to garble, $g = 2i$ and $g' = 2i + 1$.

Protocol Π_{Garble}

PARAMETERS: Finite field $\mathbb{F}_{2^\kappa}$.

COMMON INPUT: A boolean circuit $\mathbf{C}$ with a fixed topological order on the gates.

Given a gate, we denote by u (resp. v) its left (resp. right) input wire, and by w its output wire. If it is an AND gate, let (g, g') be the unique gate indices. Let $F : \{0, 1\}^\kappa \times [|G|] \times [n] \rightarrow \{0, 1\}^\kappa$ be a **ccrnd** PRF.

Init: All parties call **Init** on $\mathcal{F}_{\text{Prep}}$ and receive $\Delta^i \leftarrow \mathbb{F}_{2^\kappa}$.

Garbling: The parties proceed in four steps.

Circuit input wire masks and keys. For a circuit input wire $w \in \mathcal{I}_i$ of party P_i ,

1. Parties call **(RandomBit, i)** on $\mathcal{F}_{\text{Prep}}$ with input $\lambda_w \leftarrow \{0, 1\}$ from P_i . All parties receive $\llbracket \lambda_w \rrbracket$.
2. P_i samples a key $k_{w,0}^i \leftarrow \{0, 1\}^\kappa$ and sets $k_{w,1}^i := k_{w,0}^i \oplus \Delta^i$.

Intermediate wire masks and keys.

- If $w \in W_\wedge$:
 1. P_i calls $\mathcal{F}_{\text{Prep}}$ on command **RandomBit**. It receives $\llbracket \lambda_w \rrbracket$, where $\lambda_w = \bigoplus_{i \in [n]} \lambda_w^i$.
 2. P_i samples a key $k_{w,0}^i \leftarrow \{0, 1\}^\kappa$ and sets $k_{w,1}^i$ accordingly.
- If the wire $w \in W_\oplus$:
 1. P_i calls $\mathcal{F}_{\text{Prep}}$ on command **Add** with input $\llbracket \lambda_u \rrbracket$ and $\llbracket \lambda_v \rrbracket$, and receives $\llbracket \lambda_u \oplus \lambda_v \rrbracket$.
 2. P_i sets $k_{w,0}^i := k_{u,0}^i \oplus k_{v,0}^i$ and $k_{w,1}^i := k_{u,0}^i \oplus \Delta^i$.

Generate garbled gates. For each AND gate g with input wires u and v and output wire w :

1. All parties P_i , call **ANDOutputMask** on $\mathcal{F}_{\text{Prep}}$ with input $\llbracket \lambda_u \rrbracket, \llbracket \lambda_v \rrbracket$, and obtain $\{\langle \lambda_u \lambda_v \Delta^j \rangle\}_{j \in [n]}$. The parties locally compute $\langle \lambda_u \Delta^j \rangle$ from $\llbracket \lambda_u \rrbracket$, $\langle \lambda_v \Delta^j \rangle$ from $\llbracket \lambda_v \rrbracket$ and $\langle \lambda_w \Delta^j \rangle$ from $\llbracket \lambda_w \rrbracket$ for each $j \in [n]$.
2. For every AND gate identified by (g, g') , each P_i computes

$$\begin{aligned} \langle C_{1,g}^j \rangle_i &:= F_{k_{u,0}^i}(j||g) \oplus F_{k_{u,1}^i}(j||g) \oplus \langle \lambda_v \Delta^j \rangle_i, \\ \langle C_{2,g}^{j,a} \rangle_i &:= F_{k_{v,a}^i}(j||g') \oplus F_{k_{u,0}^i}(j||g) \oplus \langle (\lambda_u \lambda_v) \Delta^j \rangle_i \oplus \langle \lambda_w \Delta^j \rangle_i \oplus a \cdot \langle \lambda_u \Delta^j \rangle_i, \end{aligned}$$

for $j \in [n] \setminus \{i\}$, $a \in \{0, 1\}$, and for $a \in \{0, 1\}$

$$\begin{aligned} \langle C_{1,g}^i \rangle_i &:= F_{k_{u,0}^i}(i||g) \oplus F_{k_{u,1}^i}(i||g) \oplus \langle \lambda_v \Delta^i \rangle_i \oplus k_{v,0}^i, \\ \langle C_{2,g}^{i,a} \rangle_i &:= F_{k_{v,a}^i}(i||g') \oplus F_{k_{u,0}^i}(i||g) \oplus \langle (\lambda_u \lambda_v) \Delta^i \rangle_i \oplus \langle \lambda_w \Delta^i \rangle_i \oplus k_{w,0}^i \oplus a \cdot \langle \lambda_u \Delta^i \rangle_i. \end{aligned}$$

Output wire masks. For output wires $w \in \mathcal{O}_i$ of P_i , all parties call **OpenValue** on $\mathcal{F}_{\text{Prep}}$ with input $\llbracket \lambda_w \rrbracket$. P_i receives λ_w .

Open Garbling: Let $\{\langle C_{1,g}^j \rangle_i, \langle C_{2,g}^{j,0} \rangle_i, \langle C_{2,g}^{j,1} \rangle_i\}_{j \in [n], g \in W_\wedge}$ be P_i 's share computed in the garbling phase.

1. For $j \in [n]$ and all AND gates g , P_i broadcasts $\langle C_{1,g}^j \rangle_i, \langle C_{2,g}^{j,0} \rangle_i, \langle C_{2,g}^{j,1} \rangle_i$.
2. For $j \in [n]$, the parties reconstruct $C_{1,g}^j := \bigoplus_{i=1}^n \langle C_{1,g}^j \rangle_i$,
 $C_{2,g}^{j,0} := \bigoplus_{i=1}^n \langle C_{2,g}^{j,0} \rangle_i$ and $C_{2,g}^{j,1} := \bigoplus_{i=1}^n \langle C_{2,g}^{j,1} \rangle_i$.

Figure 7: Function-dependent preprocessing in HSS17-style garbling.

Constructing the Circuit. As observed in Section 2.4, the parties can obtain additive shares $\langle \lambda_u \Delta^j \rangle$ from $\llbracket \lambda_u \rrbracket$ and $\langle \lambda_v \Delta^j \rangle$ from $\llbracket \lambda_v \rrbracket$. The parties use $\mathcal{F}_{\text{Prep}}$ to obtain $\langle \lambda_u \lambda_v \Delta^j \rangle$. With the necessary material, each P_i , $i \in [n]$, computes,

$$\begin{aligned} \langle C_{1,g}^j \rangle_i &\leftarrow F_{k_{u,0}^i}(j||g) \oplus F_{k_{u,1}^i}(j||g) \oplus \langle \lambda_v \Delta^j \rangle_i & \text{if } i \neq j, \\ \langle C_{1,g}^i \rangle_i &\leftarrow F_{k_{u,0}^i}(i||g) \oplus F_{k_{u,1}^i}(i||g) \oplus \langle \lambda_v \Delta^i \rangle_i \oplus k_{v,0}^i & \text{if } i = j, \end{aligned}$$

for shares of $C_{1,g}^j$. P_i continues for $a \in \{0, 1\}$ if $i \neq j$,

$$\langle C_{2,g}^{j,a} \rangle_i \leftarrow F_{k_{v,a}^i}(j||g') \oplus F_{k_{u,0}^i}(j||g) \oplus \langle (\lambda_u \lambda_v) \Delta^j \rangle_i \oplus \langle \lambda_w \Delta^j \rangle_i \oplus a \cdot \langle \lambda_u \Delta^j \rangle_i,$$

and if $i = j$,

$$\langle C_{2,g}^{i,a} \rangle_i \leftarrow F_{k_{v,a}^i}(i||g') \oplus F_{k_{u,0}^i}(i||g) \oplus k_{w,0}^i \oplus \langle (\lambda_u \lambda_v) \Delta^i \rangle_i \oplus \langle \lambda_w \Delta^i \rangle_i \oplus a \cdot \langle \lambda_u \Delta^i \rangle_i.$$

After garbling the whole circuit, the parties reconstruct the garbled circuit for each party such that all parties obtain all circuits. Now each party P_i has n circuits $\{C_{1,g}^j, C_{2,g}^{j,0}, C_{2,g}^{j,1}\}_{j \in n, g \in W_\wedge}$ and the output keys $k_{w,0}^i$ of their own circuit.

Protocol Π_{Garble} presents a self-contained description in Figure 7. We formally prove security of Π_{Garble} in Theorem 1 which realizes the ideal functionality for the garbling phase $\mathcal{F}_{\text{Garble}}$ in Figure 8.

Theorem 1. *The protocol Π_{Garble} securely realizes $\mathcal{F}_{\text{Garble}}$ in the $\mathcal{F}_{\text{Prep}}$ -hybrid model against an active adversary corrupting at most $n - 1$ parties.*

Proof. We construct a simulator $\mathcal{S}$ that simulates the honest parties in Π_{Garble} to the adversary $\mathcal{A}$ in the ideal world with access to $\mathcal{F}_{\text{Garble}}$. We assume the adversary is deterministic and uses a random tape r as input for randomness. Note that $\mathcal{A}$ may ignore r and use a different randomness source instead. We use r to compute the wire label keys of the corrupted parties to obtain the values that they would sample if they followed the protocol.

The simulator emulates $\mathcal{F}_{\text{Prep}}$ to $\mathcal{A}$. We will now describe the different phases of the protocol.

Init. $\mathcal{S}$ emulates $\mathcal{F}_{\text{Prep}}$ and receives Δ^j for $j \in \mathcal{C}$ from $\mathcal{A}$. It chooses $\Delta^i \leftarrow \mathbb{F}_{2^\kappa}$ for all $i \notin \mathcal{C}$.

Circuit-input wire masks and keys. $\mathcal{S}$ emulates **RandomBit** of $\mathcal{F}_{\text{Prep}}$ and receives λ_w for each input wire of a corrupted party. The simulator uses r to sample the corresponding $k_{w,0}$. For input wires of honest parties, $\mathcal{S}$ picks random λ_w and $k_{w,0}$ and emulates **RandomBit** with input λ_w for $\mathcal{F}_{\text{Prep}}$.

Intermediate wire masks and keys. For an AND gate, $\mathcal{S}$ emulates **RandomBit** of $\mathcal{F}_{\text{Prep}}$: It samples $\lambda_w \leftarrow \mathbb{F}_{2^\kappa}$, runs **Macro** $[\lambda_w]$ and sends $\llbracket \lambda_w \rrbracket$ to the corrupted parties. The simulator uses r to sample the corresponding $k_{w,0}^j$ for corrupted parties and picks a random $k_{w,0}^i$ for honest parties.

For an XOR gate, it emulates **Add** of $\mathcal{F}_{\text{Prep}}$ and computes the output keys as in the protocol.

Generate garbled gates. For each AND gate g , $\mathcal{S}$ emulates **ANDOutputMask** of $\mathcal{F}_{\text{Prep}}$ and computes for each $j \in \mathcal{C}$ and $i \in [n]$, the ciphertext shares $\langle C_{1,g}^i \rangle_j, \langle C_{2,g}^{i,0} \rangle_j, \langle C_{2,g}^{i,1} \rangle_j$ according to the protocol. Note that $\mathcal{S}$ has all the necessary values. The output shares $\langle \lambda_u \lambda_v \Delta^i \rangle$ were sent by $\mathcal{A}$ during **ANDOutputMask**.

Output wire masks. The simulator calls **Garbling** on $\mathcal{F}_{\text{Garble}}$ with input from each corrupted party P_j : Δ^j , keys $k_{w,0}^j$ for each input wire of P_j and AND gate output wire and λ_w for each input

Functionality $\mathcal{F}_{\text{Garble}}$

Parameters: Finite field $\mathbb{F}_{2^\kappa}$, circuit C with pre-defined topological order.

Garbling: On input (Garbling) from all parties,

1. $\mathcal{F}_{\text{Garble}}$ receives from the adversary $\mathcal{A}$, the wire label offset Δ^j , keys $k_{w,0}^j$ for $w \in \mathcal{O}_j \cup W_\wedge$ for corrupted parties $j \in \mathcal{C}$.
2. $\mathcal{F}_{\text{Garble}}$ chooses $\Delta^i \leftarrow \mathbb{F}_{2^\kappa}$, $k_{w,0}^i \leftarrow \{0,1\}^\kappa$ for $w \in \mathcal{O}_i \cup W_\wedge$ for honest parties $i \notin \mathcal{C}$.
3. $\mathcal{F}_{\text{Garble}}$ receives from $\mathcal{A}$ input wire masks λ_w for $w \in \mathcal{I}_j$, $j \in \mathcal{C}$.
4. $\mathcal{F}_{\text{Garble}}$ chooses input wires masks $\lambda_w \leftarrow \{0,1\}$ for $w \in \mathcal{I}_i$, $i \notin \mathcal{C}$.
5. $\mathcal{F}_{\text{Garble}}$ chooses remaining wire masks for AND gates $\lambda_w \leftarrow \{0,1\}$ for $w \in W_\wedge$, and computes wire masks for XOR gates as $\lambda_w := \lambda_u \oplus \lambda_v$ for XOR gate with input wires u and v and output wire w in the topological order.
6. $\mathcal{F}_{\text{Garble}}$ computes the garbled circuit by stepping through the circuit C in topological order and for every gate (g, g') with input wires u, v and output wire w
 - if it is an XOR gate, $k_{w,0}^i := k_{u,0}^i \oplus k_{v,0}^i$, $i \in [n]$
 - if it is an AND gate,

$$C_{1,g}^i := \bigoplus_{l=1}^n F_{k_{u,0}^l} (i||g) \oplus \bigoplus_{l=1}^n F_{k_{u,1}^l} (i||g) \oplus k_{v,0}^i \oplus \lambda_v \Delta^i,$$

$$C_{2,g}^{i,a} := \bigoplus_{l=1}^n F_{k_{v,a}^l} (i||g') \oplus \bigoplus_{l=1}^n F_{k_{u,0}^l} (i||g) \oplus k_{w,0}^i \oplus (\lambda_w \oplus \lambda_u \lambda_v \oplus a \cdot \lambda_u) \Delta^i,$$

for $a \in \{0,1\}$, $i \in [n]$.

and stores the ciphertexts.

7. $\mathcal{F}_{\text{Garble}}$ sends λ_w , $w \in \mathcal{O}_j$ to $\mathcal{A}$ for $j \in \mathcal{C}$. When $\mathcal{A}$ replies with continue, the functionality outputs Δ^i , $\{\lambda_w\}_{w \in \mathcal{I}_i \cup \mathcal{O}_i}$, and $\{k_{w,0}^i\}_{w \in W}$ to honest party P_i , $i \notin \mathcal{C}$. Otherwise, $\mathcal{F}_{\text{Garble}}$ aborts.

Open Garbling: On input (OpenGarbling) from all parties, and if $\mathcal{F}_{\text{Garble}}$ executed the command Garbling before without aborting,

1. $\mathcal{F}_{\text{Garble}}$ outputs the garbled circuit $\{C_{1,g}^i, C_{2,g}^{i,0}, C_{2,g}^{i,1}\}_{i \in [n], g \in W_\wedge}$ to $\mathcal{A}$. If $\mathcal{A}$ aborts, send abort to the honest parties, otherwise receive the additive error $\{e_{1,g}^i, e_{2,g}^{i,0}, e_{2,g}^{i,1}\}_{i \in [n], g \in W_\wedge}$ from $\mathcal{A}$.
2. $\mathcal{F}_{\text{Garble}}$ outputs the garbled circuit $\{C_{1,g}^i \oplus e_{1,g}^i, C_{2,g}^{i,0} \oplus e_{2,g}^{i,0}, C_{2,g}^{i,1} \oplus e_{2,g}^{i,1}\}_{i \in [n], g \in W_\wedge}$ to each honest party P_i , $i \notin \mathcal{C}$.

Figure 8: Functionality for function-dependent preprocessing.

wire of P_j . $\mathcal{S}$ receives $\tilde{\lambda}_w$ for the output wires of each P_j . Then, $\mathcal{S}$ emulates **OpenValue** of $\mathcal{F}_{\text{Prep}}$ towards $\mathcal{A}$ making it output $\tilde{\lambda}_w$. If $\mathcal{A}$ aborts during interaction with $\mathcal{F}_{\text{Prep}}$, $\mathcal{S}$ sends abort to $\mathcal{F}_{\text{Garble}}$, otherwise it sends continue.

Open Garbling. The simulator calls **OpenGarbling** on $\mathcal{F}_{\text{Garble}}$ and receives the garbled circuit $\{C_{1,g}^i, C_{2,g}^{i,0}, C_{2,g}^{i,1}\}_{i \in [n], g \in W_\wedge}$. For each $i \in [n]$, $g \in W_\wedge$, $\mathcal{S}$ creates random shares $\{\langle C_{1,g}^i \rangle_j, \langle C_{2,g}^{i,0} \rangle_j, \langle C_{2,g}^{i,1} \rangle_j\}_{j \notin \mathcal{C}}$ subject to

$$\bigoplus_{j \in \mathcal{C}} \langle C_{1,g}^i \rangle_j + \bigoplus_{j \notin \mathcal{C}} \langle C_{1,g}^i \rangle_j = C_{1,g}^i, \quad \bigoplus_{j \in \mathcal{C}} \langle C_{2,g}^{i,a} \rangle_j + \bigoplus_{j \notin \mathcal{C}} \langle C_{2,g}^{i,a} \rangle_j = C_{2,g}^{i,a} \text{ for } a \in \{0,1\}.$$

It sends $\{\langle C_{1,g}^i \rangle_j, \langle C_{2,g}^{i,0} \rangle_j, \langle C_{2,g}^{i,1} \rangle_j\}_{j \notin \mathcal{C}}$ on behalf of the honest parties $j \notin \mathcal{C}$ to $\mathcal{A}$. The simulator receives $\{\langle \tilde{C}_{1,g}^i \rangle_j, \langle \tilde{C}_{2,g}^{i,0} \rangle_j, \langle \tilde{C}_{2,g}^{i,1} \rangle_j\}_{j \in \mathcal{C}}$ from the corrupted parties. $\mathcal{S}$ computes the additive error for

each $i \in [n]$, $g \in W_\wedge$ as

$$e_{1,g}^i = \bigoplus_{j \in \mathcal{C}} \langle C_{1,g}^i \rangle_j \oplus \langle \tilde{C}_{1,g}^i \rangle_j, \quad e_{2,g}^{i,a} = \bigoplus_{j \in \mathcal{C}} \langle C_{2,a}^{i,0} \rangle_j \oplus \langle \tilde{C}_{2,a}^{i,0} \rangle_j \text{ for } a \in \{0, 1\},$$

and sends them to $\mathcal{F}_{\text{Garble}}$ if $\mathcal{A}$ did not abort.

This concludes the simulator. We now argue indistinguishability to the real protocol execution. Correctness of the garbled circuit computed in the last step of Π_{Garble} compared to the circuit output by $\mathcal{F}_{\text{Garble}}$ is easy to see. Note that all interactions with $\mathcal{F}_{\text{Prep}}$ are perfectly simulated by $\mathcal{S}$. The output wire masks that a corrupt P_j obtains from **OpenValue** are *identically* distributed since they are computed as in the protocol (the honest parties' inputs are random bits and the corrupted parties' inputs are included). Further, the shares of $\{C_{1,g}^i, C_{2,g}^{i,0}, C_{2,g}^{i,1}\}_{i \in [n], g \in W_\wedge}$ that are sent to $\mathcal{A}$ from the honest parties (or $\mathcal{S}$) are indistinguishable due to the randomness in $\langle \lambda_v \Delta^i \rangle$, $\langle \lambda_u \lambda_v \Delta^i \rangle$ and $\langle \lambda_u \Delta^i \rangle$. $\square$

3.2 Row Reduction in Authenticated Garbling

We will now detail how the same idea can be used to achieve full row reduction also in the authenticated garbling approach. In our construction, each garbling party P_i , $i \neq 1$, sends $3(n-1)\kappa + 1$ bits per AND gate whereas in YWZ20 [YWZ20] P_i sends $(4n-6)\kappa + 1$ bits and in WRK17 [WRK17b] P_i sends $4n\kappa + 4$ bits. Note that unlike in HSS17 garbling, we require $\text{lsb}(\Delta^2) = 1$, same as in YWZ20. This simply means that an honest P_2 chooses Δ^2 s.t. $\text{lsb}(\Delta^2) = 1$. Should P_2 be malicious and choose $\text{lsb}(\Delta^2) = 0$, then this will be caught in the check phase.

Before describing the protocol, we first define our encryption function that is constructed from a ccrrnd secure single-key PRF.

Definition 6 (Concatenated ccrrnd single-key PRF). *Let F be a ccrrnd single-key PRF (as in Definition 1). We define the concatenated version $\hat{F} : \{0, 1\}^\kappa \times \{0, 1\}^{\kappa - \lceil \log_2(n) \rceil} \times ([n] \setminus \{1\}) \rightarrow \{0, 1\}^{(n-1)\kappa}$ as*

$$\hat{F}_k(g, i) = F_k(1 \| g) \| \{F_k(j \| g)\}_{j \neq i, 1},$$

where by $x \| \{y^j\}_{j \neq i, 1}$ we mean the concatenation $x \| y^2 \| \dots \| y^{i-1} \| y^{i+1} \| \dots \| y^n$ (x concatenated with all y^j in ascending order except for $j = i$).

To garble the AND gate (g, g') with incoming wires u and v , and outgoing wire w , the parties first compute the following authenticated bit-shares locally,

$$\begin{aligned} \llbracket r_1 \rrbracket &:= \llbracket \lambda_v \rrbracket, \\ \llbracket r_{2,a} \rrbracket &:= \llbracket \lambda_w \rrbracket + \llbracket \lambda_u \lambda_v \rrbracket + a \cdot \llbracket \lambda_u \rrbracket, \end{aligned} \quad \text{for } a \in \{0, 1\}.$$

Each garbler P_i , $i \neq 1$ creates three ciphertexts of length $3(n-1)\kappa$ bit,

$$\begin{aligned} C_1^i &:= \hat{F}_{k_{u,0}^i}(g, i) \oplus \hat{F}_{k_{u,1}^i}(g, i) \oplus \left(k_{v,0}^i \oplus \langle r_1 \Delta^i \rangle_i \| \{ \langle r_1 \Delta^j \rangle_i \}_{j \neq i, 1} \right), \\ C_2^{i,a} &:= \hat{F}_{k_{u,0}^i}(g, i) \oplus \hat{F}_{k_{v,a}^i}(g', i) \oplus \left(k_{w,0}^i \oplus \langle r_{2,a} \Delta^i \rangle_i \| \{ \langle r_{2,a} \Delta^j \rangle_i \}_{j \neq i, 1} \right) \text{ for } a \in \{0, 1\}, \end{aligned}$$

where all $\langle r_1 \Delta^j \rangle_i, \langle r_{2,a} \Delta^j \rangle_i$ for $j \neq 1$ can be derived locally from $\llbracket r_1 \rrbracket, \llbracket r_{2,a} \rrbracket$. Each garbler sends the three ciphertexts to the evaluator P_1 . In addition, P_2 will send $b := \text{lsb}(k_{w,0}^2)$ to P_1 . The full protocol, $\Pi_{\text{AuthGarble}}$, is depicted in Figure 9.

Protocol $\Pi_{\text{AuthGarble}}$

PARAMETERS: Finite field $\mathbb{F}_{2^\kappa}$.

COMMON INPUT: A boolean circuit $\mathbf{C}$ with a fixed topological order on the gates.

Given a gate, we denote by u (resp. v) its left (resp. right) input wire, and by w its output wire. If it is an AND gate, let (g, g') be the unique gate indices. Let $\hat{F}$ be the concatenated ccrnd PRF (cf. Definition 6).

Init: All parties call (Init, P_2) on $\mathcal{F}_{\text{Prep}}$ and receive $\Delta^i \leftarrow \mathbb{F}_{2^\kappa}$ where $\text{lsb}(\Delta^2) = 1$.

Garbling: The parties proceed in four steps.

Circuit input wire masks and keys. For a circuit input wire $w \in \mathcal{I}_i$ of party P_i ,

1. Parties call $(\text{RandomBit}, i)$ on $\mathcal{F}_{\text{Prep}}$ with input $\lambda_w \leftarrow \{0, 1\}$ from P_i . All parties receive $\llbracket \lambda_w \rrbracket$.
2. Each garbler P_j , $j \neq 1$, samples a key $k_{w,0}^j \leftarrow \{0, 1\}^\kappa$ and sets $k_{w,1}^j := k_{w,0}^j \oplus \Delta^j$.

Intermediate wire masks and keys.

- If $w \in W_\wedge$:
 1. All parties call $\mathcal{F}_{\text{Prep}}$ on command **RandomBit**. P_i receives $\llbracket \lambda_w \rrbracket$, where $\lambda_w = \bigoplus_{i \in [n]} \lambda_w^i$.
 2. Each garbler P_i , $i \neq 1$ samples a key $k_{w,0}^i \leftarrow \{0, 1\}^\kappa$ and sets $k_{w,1}^i$ accordingly.
 3. Party P_2 computes $b_w := \text{lsb}(k_{w,0}^2)$.
- If the wire $w \in W_\oplus$:
 1. All parties call $\mathcal{F}_{\text{Prep}}$ on command **Add** with input $\llbracket \lambda_u \rrbracket$ and $\llbracket \lambda_v \rrbracket$, and P_i receives $\llbracket \lambda_u \oplus \lambda_v \rrbracket$.
 2. Each garbler P_i , $i \neq 1$, sets $k_{w,0}^i := k_{u,0}^i \oplus k_{v,0}^i$ and $k_{w,1}^i := k_{u,1}^i \oplus k_{v,1}^i$.

Generate garbled gates. For each AND gate g with input wires u and v and output wire w :

1. All parties P_i , call **ANDOutputMask** on $\mathcal{F}_{\text{Prep}}$ with input $\llbracket \lambda_u \rrbracket$, $\llbracket \lambda_v \rrbracket$, and obtain $\{\langle \lambda_u \lambda_v \Delta^j \rangle\}_{j \in [n]}$. The parties locally compute $\langle \lambda_u \Delta^j \rangle$ from $\llbracket \lambda_u \rrbracket$, $\langle \lambda_v \Delta^j \rangle$ from $\llbracket \lambda_v \rrbracket$ and $\langle \lambda_w \Delta^j \rangle$ from $\llbracket \lambda_w \rrbracket$ for each $j \in [n]$.
2. For every AND gate identified by (g, g') , each P_i , $i \neq 1$, computes

$$\begin{aligned} \llbracket r_1 \rrbracket &:= \llbracket \lambda_v \rrbracket, \\ \llbracket r_{2,a} \rrbracket &:= \llbracket \lambda_w \rrbracket \oplus \llbracket \lambda_u \lambda_v \rrbracket \oplus a \cdot \llbracket \lambda_u \rrbracket, \\ C_{1,g}^i &:= \hat{F}_{k_{u,0}^i}(g, i) \oplus \hat{F}_{k_{u,1}^i}(g, i) \oplus \left(k_{v,0}^i \oplus \langle r_1 \Delta^i \rangle_i \parallel \{ \langle r_1 \Delta^j \rangle_i \}_{j \neq i, 1} \right), \\ C_{2,g}^{i,a} &:= \hat{F}_{k_{v,a}^i}(g', i) \oplus \hat{F}_{k_{u,0}^i}(g, i) \oplus \left(k_{w,0}^i \oplus \langle r_{2,a} \Delta^i \rangle_i \parallel \{ \langle r_{2,a} \Delta^j \rangle_i \}_{j \neq i, 1} \right), \end{aligned}$$

for $a \in \{0, 1\}$.

Output wire masks and circuit. For output wires $w \in \mathcal{O}_i$ of P_i , all parties call **OpenValue** on $\mathcal{F}_{\text{Prep}}$ with input $\llbracket \lambda_w \rrbracket$.

1. P_i receives λ_w .
2. Each party P_i , $i \neq 1$, sends $\{C_{1,g}^i, C_{2,g}^{i,0}, C_{2,g}^{i,1}\}_{g \in W_\wedge}$ to P_1 .
3. Party P_2 sends $\{b_w\}_{w \in W_\wedge}$ to P_1 .

Figure 9: Function-dependent preprocessing using authenticated garbling.

4 Online Evaluation

We will now detail the function- and input-dependent phase of our protocols. Again, the HSS17-style and authenticated garbling protocol follow the same pattern but differ in the details how AND gates are evaluated. When using the term evaluators or evaluating parties, we mean all parties in case of HSS17 and P_1 in case of authenticated garbling. We discuss the common pattern first and detail AND gate evaluation separately in Sections 4.1 and 4.2 for HSS17 and authenticated garbling, respectively.

Let x be the input of P_i for an input wire w . Since P_i obtained λ_w in the clear from the garbling phase, it broadcasts $\hat{w} = \lambda_w \oplus x$ to all parties. All garblers P_j then broadcast $k_{w,\hat{w}}^j$, or in case of authenticated garbling, send $k_{w,\hat{w}}^j$ to P_1 . After the input phase, the evaluating parties have active keys for all input wires.

Now, the parties traverse the circuit in topological order. Each evaluating party evaluates an XOR gate (u, v, w) as $k_{w,\hat{w}}^j := k_{u,\hat{u}}^j \oplus k_{v,\hat{v}}^j$ for each active key and sets $\hat{w} = \hat{u} \oplus \hat{v}$. Note that this results in the correct active key for wire w due to FreeXOR,

$$k_{w,\hat{w}}^j = k_{u,\hat{u}}^j \oplus k_{v,\hat{v}}^j = k_{u,0}^j \oplus k_{v,0}^j \oplus ((u \oplus v) \oplus (\lambda_u \oplus \lambda_v)) \Delta^i.$$

4.1 AND Gate Evaluation in HSS17-style Garbling

Each evaluating party P_i received the garbled circuit $\{C_{1,g}^j, C_{2,g}^{j,0}, C_{2,g}^{j,1}\}_{j \in [n], g \in W_\wedge}$ from Π_{Garble} . For an AND gate indexed by (g, g') with wire values (u, v, w) , P_i evaluates P_j 's circuit using the three ciphertexts $C_{1,g}^j, C_{2,g}^{j,0}, C_{2,g}^{j,1}$ to obtain $k_{w,\hat{w}}^j$ as,

$$k_{w,\hat{w}}^j = \bigoplus_{l=1}^n F_{k_{u,\hat{u}}^l}(j||g) \oplus \bigoplus_{l=1}^n F_{k_{v,\hat{v}}^l}(j||g') \oplus \hat{u}(C_{1,g}^j \oplus k_{v,\hat{v}}^j) \oplus C_{2,g}^{j,\hat{v}}. \quad (2)$$

Since P_i also evaluates its own circuit to obtain $k_{w,\hat{w}}^i$, it can figure out $\hat{w}$ for subsequent gates by looking at the obtained labels of its own circuit. P_i sets $\hat{w}$ s.t. $k_{w,\hat{w}}^i = k_{w,0}^i \oplus \hat{w} \Delta^i$, and otherwise aborts.

Let λ_w be the wire mask for P_i 's output wire. Since P_i obtained $\hat{w}$, and is sure about its correctness due to the evaluation of its own circuit, it can output $y := \lambda_w \oplus \hat{w}$ without further checks. Figure 10 details the evaluation protocol Π_{Eval} .

Correctness of the Row Reduction. Since the correctness of input, output and XOR gate evaluation follow easily from inspection, we now show how evaluating Equation (2) using a correctly garbled circuit indeed computes an AND gate.

Lemma 1. *Let $\hat{x} = x \oplus \lambda_x$ for $x \in \{u, v, w\}$, and $w = uv$, then $\lambda_w \oplus \lambda_u \lambda_v \oplus \hat{v} \lambda_u = \hat{w} \oplus \hat{u} v$.*

Proof. This claim follows by inspection:

$$\begin{aligned} \lambda_w \oplus \lambda_u \lambda_v \oplus \hat{v} \lambda_u &= \lambda_w \oplus \lambda_u (\lambda_v \oplus \hat{v}) = \lambda_w \oplus \lambda_u v \\ &= \lambda_w \oplus uv \oplus uv \oplus \lambda_u v = \hat{w} \oplus (u \oplus \lambda_u) v = \hat{w} \oplus \hat{u} v. \end{aligned}$$

□

Protocol Π_{Eval}

PARAMETERS: Finite field $\mathbb{F}_{2^\kappa}$, F a ccrrnd secure PRF.

PUBLIC INPUT: Circuit C

PRIVATE INPUT: Each party P_i inputs $x_i \in \mathcal{I}_i$

1. All parties P_i call $\mathcal{F}_{\text{Garble}}$ on command **Garbling** and obtain $\Delta^i, \{\lambda_w\}_{w \in \mathcal{I}_i \cup \mathcal{O}_i}$ and $\{k_{w,0}^i\}_{w \in W}$.
2. Let x be an input of party P_i for wire $w \in \mathcal{I}_i$. P_i sets $\hat{w} = x \oplus \lambda_w$ and broadcasts $\hat{w}$.
3. All parties P_j broadcast $k_{w,\hat{w}}^j$. P_i stores $(\{k_{w,\hat{w}}^j\}_{j \in [n]}, \hat{w})$.
4. The parties call the command **OpenGarbling** on $\mathcal{F}_{\text{Garble}}$ and obtain the garbled circuit $\{C_{1,g}^j, C_{2,g}^j, C_{3,g}^j\}_{j \in [n], g \in W_\wedge}$.
5. The parties traverse the circuit in topological order.
 - (a) For every XOR gate (u, v, w) , party P_i computes $k_{w,\hat{w}}^j = k_{u,\hat{u}}^j \oplus k_{v,\hat{v}}^j$ for $j \in [n]$ and sets $\hat{w} = \hat{u} \oplus \hat{v}$.
If the obtained $k_{w,\hat{w}}^i \neq k_{w,0}^i \oplus \hat{w} \Delta^i$, party P_i aborts.
 - (b) For every AND gate g with wires (u, v, w) , party P_i computes for $j \in [n]$,

$$k_{w,\hat{w}}^j = \bigoplus_{l=1}^n F_{k_{u,\hat{u}}^l}(j||g) \oplus \bigoplus_{l=1}^n F_{k_{v,\hat{v}}^l}(j||g') \oplus \hat{u}(C_{1,g}^j \oplus k_{v,\hat{v}}^j) \oplus C_{2,g}^j.$$

If the obtained $k_{w,\hat{w}}^i$ equals $k_{w,0}^i$ or $k_{w,1}^i$, P_i sets $\hat{w}$ to 0 or 1, respectively. Otherwise, P_i aborts.

6. For every output wire $w \in \mathcal{O}_i$ of party P_i , it sets the output $y = \lambda_w \oplus \hat{w}$.

Figure 10: Protocol for Online Circuit Evaluation in HSS17-style garbling.

We check the evaluation of P_j 's circuit $C_{1,g}^j, C_{2,g}^{j,0}, C_{2,g}^{j,1}$ for AND gate (g, g') with wires (u, v, w) .

Lemma 2. *Given correctly formed ciphertexts $C_{1,g}^j, C_{2,g}^{j,0}, C_{2,g}^{j,1}$ from P_j 's circuit and input keys $\{k_{u,\hat{u}}^i, k_{v,\hat{v}}^i\}_{i \in [n]}$, the AND gate evaluation from Equation (2) correctly outputs $k_{w,\hat{w}}^j$, i.e.,*

$$\bigoplus_{l=1}^n F_{k_{u,\hat{u}}^l}(j||g) \oplus \bigoplus_{l=1}^n F_{k_{v,\hat{v}}^l}(j||g') \oplus \hat{u}(C_{1,g}^j \oplus k_{v,\hat{v}}^j) \oplus C_{2,g}^{j,\hat{v}} = k_{w,\hat{w}}^j.$$

Proof. We first present the evaluation of the row-reduced half-gate that uses $C_{1,g}^j$ and afterwards turn to the evaluation of the other unary gate from $C_{2,g}^{j,0}, C_{2,g}^{j,1}$.

The row-reduced half-gate evaluates to

$$\begin{aligned} & \bigoplus_{l=1}^n F_{k_{u,\hat{u}}^l}(j||g) \oplus \hat{u}(C_{1,g}^j \oplus k_{v,\hat{v}}^j) \\ &= \bigoplus_{l=1}^n F_{k_{u,\hat{u}}^l}(j||g) \oplus \hat{u} \left(\bigoplus_{l=1}^n F_{k_{u,0}^l}(j||g) \oplus \bigoplus_{l=1}^n F_{k_{u,1}^l}(j||g) \oplus k_{v,0}^j \oplus \lambda_v \Delta^j \oplus k_{v,\hat{v}}^j \right) \\ &= \bigoplus_{l=1}^n F_{k_{u,\hat{u}}^l}(j||g) \oplus \hat{u} \left(\bigoplus_{l=1}^n F_{k_{u,0}^l}(j||g) \oplus \bigoplus_{l=1}^n F_{k_{u,1}^l}(j||g) \right) \oplus \hat{u}(\lambda_v \oplus \hat{v}) \Delta^j \\ &= \bigoplus_{l=1}^n F_{k_{u,0}^l}(j||g) \oplus (\hat{u}v) \Delta^j. \end{aligned} \tag{3}$$

Note that this corresponds to $\text{prod}_1(u, v)$ with $\bigoplus_{l=1}^n F_{k_{u,0}^l}(j||g)$ as zero wire label. The evaluation of the remaining unary gate from $C_{2,g}^{j,0}, C_{2,g}^{j,1}$ evaluates to

$$\begin{aligned}
& \bigoplus_{l=1}^n F_{k_{v,\hat{v}}^l}(j||g') \oplus C_{2,g}^{j,\hat{v}} \\
&= \bigoplus_{l=1}^n F_{k_{v,\hat{v}}^l}(j||g') \oplus \bigoplus_{l=1}^n F_{k_{v,\hat{v}}^l}(j||g') \oplus \bigoplus_{l=1}^n F_{k_{u,0}^l}(j||g) \oplus k_{w,0}^j \oplus (\lambda_w \oplus \lambda_u \lambda_v \oplus \hat{v} \lambda_u) \Delta^j \\
&= \bigoplus_{l=1}^n F_{k_{u,0}^l}(j||g) \oplus k_{w,0}^j \oplus (\lambda_w \oplus \lambda_u \lambda_v \oplus \hat{v} \lambda_u) \Delta^j \\
&= \bigoplus_{l=1}^n F_{k_{u,0}^l}(j||g) \oplus k_{w,0}^j \oplus (\hat{w} \oplus \hat{u} v) \Delta^j, \tag{4}
\end{aligned}$$

where the last step used Lemma 1. This corresponds to prod_2 with zero wire label $k_{w,0}^j$ but also contains the zero wire label of prod_1 .

Combining Equation (3) and Equation (4), as during evaluation, we obtain

$$\begin{aligned}
k_{w,\hat{w}}^j &= \bigoplus_{l=1}^n F_{k_{u,0}^l}(j||g) \oplus (\hat{u} v) \Delta^j \oplus \bigoplus_{l=1}^n F_{k_{u,0}^l}(j||g) \oplus k_{w,0}^j \oplus (\hat{w} \oplus \hat{u} v) \Delta^j \\
&= k_{w,0}^j \oplus \hat{w} \Delta^j,
\end{aligned}$$

as desired. $\square$

We now prove that given a secure circuit-dependent preprocessing, e.g., Π_{Garble} (Figure 7), together with the online phase evaluation Π_{Eval} (Figure 10), we can securely compute any function f .

Theorem 2. *Let f be any function represented by circuit C_f , then Π_{Eval} securely computes f in the $\mathcal{F}_{\text{Garble}}$ -hybrid model against a static, malicious adversary corrupting up to $n - 1$ parties, if F is a ccrnd secure PRF for naturally derived keys (see Definition 1).*

Proof. The proof follows [HSS20, Theorem 5.1] with the same strategy. We show that the view of any adversary $\mathcal{A}$ when interacting with Π_{Eval} is computationally indistinguishable from an interaction of $\mathcal{A}$ with the simulator $\mathcal{S}$ (given below) in the ideal world with access to f . The simulator internally plays $\mathcal{A}$ and simulates the honest parties in the interaction of Π_{Eval} towards $\mathcal{A}$. In the $\mathcal{F}_{\text{Garble}}$ -hybrid model, both worlds also interact with $\mathcal{F}_{\text{Garble}}$. In the ideal world, $\mathcal{F}_{\text{Garble}}$ is simulated by $\mathcal{S}$. To keep the proof concise, we first revisit the strategy of [HSS20] adapted to our protocol and then focus on the parts that are different, namely the treatment of AND gates. The simulation/security of XOR gates is unchanged and already given in previous works. We define the following distributions/games which are detailed later.

- $\text{Ideal}_{C_f, \mathcal{S}, \mathcal{Z}}$ denotes the output of the simulator $\mathcal{S}$ where $\mathcal{S}$ only knows the set of active keys $k_{w,\hat{w}}^i$ for every wire w and party $i \in [n]$. The simulator aborts if evaluation of the modified garbled circuit returned by $\mathcal{A}$ for some honest party P_i does not match $k_{w,\hat{w}}^i$ for some wire w .

- $\widetilde{\text{Ideal}}_{C_f, \mathcal{S}, \mathcal{Z}}$ denotes the output of the simulator $\mathcal{S}$ as in Ideal with the modification that before evaluating the modified garbled circuit, $\mathcal{S}$ samples $\Delta^i \leftarrow_{\$} \{0, 1\}^\kappa$ for all honest parties P_i , defines $k_{w,0}^i$ and $k_{w,1}^i$ s.t. $k_{w,\hat{w}}^i = k_{w,0}^i \oplus \hat{w} \Delta^i$. $\mathcal{S}$ aborts if the evaluation of the modified garbled circuit does not yield $k_{w,0}^i$ or $k_{w,1}^i$ for some honest party P_i and wire w .
- $\widetilde{\text{Hyb}}_{\Pi, \mathcal{A}, \mathcal{Z}}^{\mathcal{F}_{\text{Prep}}}$ denotes the distribution that is observed by $\mathcal{A}$ during the real protocol execution but with two changes. First, the garbled circuit creation is moved to after input phase (in step 2). The input $\{x_w\}_{w \in \mathcal{I}_i}$ of all parties P_i is used to evaluate the circuit C_f obtaining all actual values $\{x_w\}_{w \in W}$ on all wires. Each party chooses $k_{w,0}^i, k_{w,1}^i \leftarrow_{\$} \{0, 1\}^\kappa$ and λ_w^i (where the adversary is allowed to choose arbitrarily). We set $\lambda_w := \bigoplus \lambda_w^i$ and set the active wire key to $k_{w, x_w \oplus \lambda_w}$.
- $\text{Hyb}_{\Pi, \mathcal{A}, \mathcal{Z}}^{\mathcal{F}_{\text{Prep}}}$ denotes the real protocol execution.

The view that is generated by $\mathcal{S}$ (Ideal) is proven to be computationally indistinguishable from the real world (Hyb) in two steps. First, we step from Ideal to $\widetilde{\text{Ideal}}$ where the distinguishing difference is the abort event that errors introduced by $\mathcal{A}$ during opening of the circuit cause some honest party to obtain $k_{w, \hat{w} \oplus 1}^i$ for a wire w . This event, named Flip, would effectively cause a party to compute incorrect values. We show that it only happens with negligible probability below. Next, we show that $\widetilde{\text{Ideal}}$ and $\widetilde{\text{Hyb}}$ are indistinguishable by reducing to the ccrnd property of the PRF F . Assuming Flip does not occur in $\widetilde{\text{Hyb}}$, then $\widetilde{\text{Hyb}}$ and Hyb only differ in the time when the garbled circuit is created, so they produce an identically distributed view for $\mathcal{A}$.

Simulator and Ideal. The simulator proceeds as the simulator of [HSS20, Theorem 5.1]. In the garbling phase, the simulator runs steps 1, 3, 4, 5 and 7 (of Figure 8) without output to honest parties playing as the functionality towards $\mathcal{A}$. Note that here, $\mathcal{A}$ chooses all keys and Δ s for the corrupted parties while the simulator *does not* choose any secret data of honest parties it simulates. $\mathcal{S}$ only chooses the secret wire masks λ .

To compute the set of active keys, the simulator first extracts the corrupted party's inputs and obtains the output values for each output wire of the circuit from the functionality that computes C_f . The active key $\hat{w}$ for wire w is chosen in the standard way where $\hat{w}$ is sampled uniformly at random under the constraints that $\mathcal{A}$'s view is consistent (e.g., w.r.t XOR gates) and that $\hat{w} \oplus \lambda_w = y_w$ for $w \in \mathcal{O}$ and y_w denotes the output wire value from the functionality f . In particular, AND gates (for all P_i 's circuit) are simulated as follows, where $\hat{u}, \hat{v}, \hat{w}$ are the external values of the active keys for gate g ,

$$\begin{aligned}
C_{1,g}^i &\leftarrow_{\$} \{0, 1\}^\kappa, \\
C_{2,g}^{i,\hat{v}} &= \bigoplus_{l=1}^n F_{k_{u,\hat{u}}^l}(i||g) \oplus \bigoplus_{l=1}^n F_{k_{v,\hat{v}}^l}(i||g') \oplus \hat{u}(C_{1,g}^i \oplus k_{v,\hat{v}}^i) \oplus k_{w,\hat{w}}^i, \\
C_{2,g}^{i,\hat{v} \oplus 1} &\leftarrow_{\$} \{0, 1\}^\kappa.
\end{aligned} \tag{5}$$

The simulator hands $\mathcal{A}$ the garbled circuit. It is easy to see that if $\mathcal{A}$ evaluates the received circuit properly, it only sees the set of active keys and obtains $k_{w,\hat{w}}^i$ when evaluating the AND gate. $\mathcal{S}$ receives the additive error $\{e_{1,g}^i, e_{2,g}^{i,0}, e_{2,g}^{i,1}\}_{i \in [n]}$ for each AND gate g from $\mathcal{A}$ and evaluates the modified circuit $\{C_{1,g}^i \oplus e_{1,g}^i, C_{2,g}^{i,0} \oplus e_{2,g}^{i,0}, C_{2,g}^{i,0} \oplus e_{2,g}^{i,1}\}_{i \in [n]}$ with the information that the simulated honest parties received from $\mathcal{A}$ during the input phase. If any honest party observes an output wire of any XOR/AND gate that is not $k_{w,\hat{w}}^i$, the simulator aborts on behalf of the honest parties. This concludes the simulator.

Flip and Ideal. Let Flip be the event that an honest party would obtain $k_{w,\hat{w}\oplus 1}$ during the evaluation of the garbled circuit that was created by $\mathcal{S}$ with the error added by $\mathcal{A}$. Let (g, g') be the first AND gate output of P_j where P_j obtains $k_{w,\hat{w}\oplus 1}^j$. Let $e_{1,g}^j, e_{2,g}^{j,0}, e_{2,g}^{j,1}$ be the additive error added by $\mathcal{A}$, then

$$k_{w,\hat{w}\oplus 1}^j = \bigoplus_{l=1}^n F_{k_{u,\hat{u}}^l}(j||g) \oplus \bigoplus_{l=1}^n F_{k_{v,\hat{v}}^l}(j||g') \oplus \hat{u}(C_{1,g}^j \oplus e_{1,g}^j \oplus k_{v,\hat{v}}^j) \oplus C_{2,g}^{j,\hat{v}} \oplus e_{2,g}^{j,\hat{v}}.$$

And therefore

$$\begin{aligned} \hat{u}e_{1,g}^j \oplus e_{2,g}^{j,\hat{v}} &= k_{w,\hat{w}\oplus 1}^j \oplus \bigoplus_{l=1}^n F_{k_{u,\hat{u}}^l}(j||g) \oplus \bigoplus_{l=1}^n F_{k_{v,\hat{v}}^l}(j||g') \oplus \hat{u}(C_{1,g}^j \oplus k_{v,\hat{v}}^j) \oplus C_{2,g}^{j,\hat{v}} \\ &= k_{w,\hat{w}\oplus 1}^j \oplus k_{w,\hat{w}}^j \quad (\text{from Lemma 2}) \\ &= \Delta^j. \end{aligned}$$

To choose matching errors $e_{1,g}^j, e_{2,g}^{j,0}, e_{2,g}^{j,1}$, $\mathcal{A}$ must guess Δ^j which was chosen independently and uniformly at random by an honest P_j . Therefore the probability of Flip is $2^{-\kappa}$ and thus negligible.

The simulator always aborts when the error added to the garbled circuit makes a party compute a wrong output key while in the real world, the parties abort with probability of the complement event to Flip. Since Flip's probability is negligible, this cannot be used to distinguish the two worlds.

Reduction to ccnd Security. We construct the same distinguisher $\mathcal{D}'$ as [HSS20, Lemma 5.1] and only highlight how $\mathcal{D}'$ produces ciphertexts for garbled AND gates. In short, we assume for contradiction an environment $\mathcal{Z}$ and adversary $\mathcal{A}$ that can distinguish between the real world $\widetilde{\text{Hyb}}$ and the ideal world $\widetilde{\text{Ideal}}$ with non-negligible probability. Using $\mathcal{Z}$ and $\mathcal{A}$, we construct $\mathcal{D}'$ for the ccnd security game. Let z be the input to $\mathcal{Z}$, then $\mathcal{D}'$ internally runs $\mathcal{Z}$ simulating the functionality that computes C_f and runs the simulator $\mathcal{S}$ that internally runs $\mathcal{A}$. $\mathcal{D}'$ receives inputs for the honest parties from $\mathcal{Z}$ during simulation of the C_f functionality and extracts the inputs of the corrupted parties in the same way as the simulator does. Therefore, $\mathcal{D}'$ knows all inputs and can compute all wire values. Let P_{i_0} be the honest party that is chosen by $\mathcal{D}'$, then it chooses Δ^i for all $P_i \neq P_{i_0}$. Further, it also received $k_{u,0}^j, k_{v,0}^j, k_{w,0}^j$ and Δ^j of each corrupted party P_j from $\mathcal{A}$ during the simulated interaction of $C_f/\mathcal{F}_{\text{Prep}}$. Now, $\mathcal{D}'$ chooses $\hat{u}, \hat{v}, \hat{w}$ at random and computes $\lambda_u, \lambda_v, \lambda_w$ accordingly (since the true value on each wire in the circuit is known).

For each AND gate g , $\mathcal{D}'$ computes the following using the ccnd oracle $\mathcal{O}(k, i, g, b)$,

$$\begin{aligned} h_{u,i} &= \begin{cases} \mathcal{O}(k_{u,\hat{u}}^{i_0}, i, g, 0) \oplus k_{v,0}^i \oplus \lambda_v \Delta^i & \text{if } i \neq i_0 \\ \mathcal{O}(k_{u,\hat{u}}^{i_0}, i_0, g, \hat{v} \oplus \lambda_v) \oplus k_{v,\hat{v}}^{i_0} & \text{if } i = i_0 \end{cases}, \\ h_{v,i} &= \begin{cases} \mathcal{O}(k_{v,\hat{v}}^{i_0}, i, g', 0) \oplus \lambda_u \Delta^i & \text{if } i \neq i_0 \\ \mathcal{O}(k_{v,\hat{v}}^{i_0}, i_0, g', \lambda_u) & \text{if } i = i_0 \end{cases}. \end{aligned} \quad (6)$$

Clearly, the oracle queries are legal since inputs are unique, and satisfy the natural derived key requirement.

The distinguisher $\mathcal{D}'$ now creates the ciphertexts for each garbled AND gate g and party $i \in [n]$ as follows:

$$\tilde{C}_1^i = \bigoplus_{j=1; j \neq i_0}^n F_{k_{u,0}^j}(i||g) \oplus \bigoplus_{j=1; j \neq i_0}^n F_{k_{u,1}^j}(i||g) \oplus F_{k_{u,\hat{u}}^{i_0}}(i||g) \oplus h_{u,i},$$

$$\begin{aligned}
\tilde{C}_2^{i,\hat{v}} &= \bigoplus_{j=1}^n F_{k_{v,\hat{v}}}^j(i||g') \oplus \bigoplus_{j=1;j \neq i_0}^n F_{k_{u,0}}^j(i||g) \\
&\quad \oplus (1 \oplus \hat{u})F_{k_{u,\hat{u}}}^{i_0}(i||g) \oplus k_{w,\hat{w}}^i \oplus \hat{u}(h_{u,i} \oplus k_{v,\hat{v}}^i), \\
\tilde{C}_2^{i,\hat{v} \oplus 1} &= \bigoplus_{j=1;j \neq i_0}^n F_{k_{v,\hat{v} \oplus 1}}^j(i||g') \oplus \bigoplus_{j=1;j \neq i_0}^n F_{k_{u,0}}^j(i||g) \\
&\quad \oplus (1 \oplus \hat{u})F_{k_{u,\hat{u}}}^{i_0}(i||g) \oplus k_{w,\hat{w}}^i \oplus \hat{u}(h_{u,i} \oplus k_{v,\hat{v}}^i) \oplus h_{v,i}.
\end{aligned}$$

If $O(k, i, g, b) = F_{k \oplus \Delta^{i_0}}(i||g) \oplus b\Delta^{i_0}$, then $\tilde{C}_1^i, \tilde{C}_2^{i,0}$ and $\tilde{C}_2^{i,1}$ are identical to $C_1^i, C_2^{i,0}$ and $C_2^{i,1}$ generated by the garbling protocol. First note that in this case, $h_{u,i} = F_{k_{u,\hat{u} \oplus 1}}^{i_0}(i||g) \oplus k_{v,\lambda_v}^i$ and $h_{v,i} = F_{k_{v,\hat{v} \oplus 1}}^{i_0}(i||g') \oplus \lambda_u \Delta^i$. Thus for $\hat{u}(h_{u,i} \oplus k_{v,\hat{v}}^i) = \hat{u}F_{k_{u,\hat{u} \oplus 1}}^{i_0}(i||g) \oplus (\hat{u}v)\Delta^i$ which we use in the following derivations:

$$\begin{aligned}
\tilde{C}_1^i &= \bigoplus_{j=1;j \neq i_0}^n F_{k_{u,0}}^j(i||g) \oplus \bigoplus_{j=1;j \neq i_0}^n F_{k_{u,1}}^j(i||g) \oplus F_{k_{u,\hat{u}}}^{i_0}(i||g) \oplus F_{k_{u,\hat{u} \oplus 1}}^{i_0}(i||g) \oplus k_{v,\lambda_v}^i \\
&= \bigoplus_{j=1}^n F_{k_{u,0}}^j(i||g) \oplus \bigoplus_{j=1}^n F_{k_{u,1}}^j(i||g) \oplus k_{v,0}^i \oplus \lambda_v \Delta^i = C_1^i.
\end{aligned}$$

For $\tilde{C}_2^{i,\hat{v}} = C_2^{i,\hat{v}}$ we have,

$$\begin{aligned}
\tilde{C}_2^{i,\hat{v}} &= \bigoplus_{j=1}^n F_{k_{v,\hat{v}}}^j(i||g') \oplus \bigoplus_{j=1;j \neq i_0}^n F_{k_{u,0}}^j(i||g) \oplus (1 \oplus \hat{u})F_{k_{u,\hat{u}}}^{i_0}(i||g) \\
&\quad \oplus k_{w,\hat{w}}^i \oplus \hat{u}F_{k_{u,\hat{u} \oplus 1}}^{i_0}(i||g) \oplus (\hat{u}v)\Delta^i \\
&= \bigoplus_{j=1}^n F_{k_{v,\hat{v}}}^j(i||g') \oplus \bigoplus_{j=1}^n F_{k_{u,0}}^j(i||g) \oplus k_{w,0}^i \oplus (\hat{w} \oplus \hat{u}v)\Delta^i \\
&= \bigoplus_{j=1}^n F_{k_{v,\hat{v}}}^j(i||g') \oplus \bigoplus_{j=1}^n F_{k_{u,0}}^j(i||g) \oplus k_{w,0}^i \oplus (\lambda_w \oplus \lambda_u \lambda_v \oplus \hat{v} \lambda_u) \\
&= C_2^{i,\hat{v}}. \tag{from Lemma 1}
\end{aligned}$$

And for $\tilde{C}_2^{i,\hat{v} \oplus 1} = C_2^{i,\hat{v} \oplus 1}$, the same holds:

$$\begin{aligned}
\tilde{C}_2^{i,\hat{v} \oplus 1} &= \bigoplus_{j=1;j \neq i_0}^n F_{k_{v,\hat{v} \oplus 1}}^j(i||g') \oplus \bigoplus_{j=1;j \neq i_0}^n F_{k_{u,0}}^j(i||g) \oplus (1 \oplus \hat{u})F_{k_{u,\hat{u}}}^{i_0}(i||g) \oplus k_{w,\hat{w}}^i \\
&\quad \oplus \hat{u}F_{k_{u,\hat{u} \oplus 1}}^{i_0}(i||g) \oplus (\hat{u}v)\Delta^i \oplus F_{k_{v,\hat{v} \oplus 1}}^{i_0}(i||g') \oplus \lambda_u \Delta^i \\
&= \bigoplus_{j=1}^n F_{k_{v,\hat{v} \oplus 1}}^j(i||g') \oplus \bigoplus_{j=1}^n F_{k_{u,0}}^j(i||g) \oplus k_{w,0}^i \oplus (\hat{w} \oplus \hat{u}v)\Delta^i \oplus \lambda_u \Delta^i
\end{aligned}$$

$$\begin{aligned}
&= \bigoplus_{j=1}^n F_{k_{v,\hat{v} \oplus 1}}^j(i||g') \oplus \bigoplus_{j=1}^n F_{k_{u,0}}^j(i||g) \oplus k_{w,0}^i \oplus (\lambda_w \oplus \lambda_u \lambda_v \oplus \hat{v} \lambda_u) \Delta^i \oplus \lambda_u \Delta^i \\
&= C_2^{i,\hat{v} \oplus 1}. \quad (\text{from Lemma 1})
\end{aligned}$$

Next, we show that if $\mathcal{O}(k, i, g, b) = \text{Rand}$, the distribution of $(\tilde{C}_1^i, \tilde{C}_2^{i,0}, \tilde{C}_2^{i,1})$ is indistinguishable from the simulated garbled circuit in Equation (5). Since now $h_{u,i}$ and $h_{v,i}$ are fresh uniformly random values, $\tilde{C}_1^i$ and $\tilde{C}_2^{i,\hat{v} \oplus 1}$ are indistinguishable from C_1^i and $C_2^{i,\hat{v} \oplus 1}$. Further, we express $h_{u,i} = \tilde{C}_1^i \oplus \bigoplus_{j=1; j \neq i_0}^n F_{k_{u,0}}^j(i||g) \oplus \bigoplus_{j=1; j \neq i_0}^n F_{k_{u,1}}^j(i||g) \oplus F_{k_{u,\hat{u}}}^{i_0}(i||g)$, so

$$\begin{aligned}
\tilde{C}_2^{i,\hat{v}} &= \bigoplus_{j=1}^n F_{k_{v,\hat{v}}}^j(i||g') \oplus \bigoplus_{j=1; j \neq i_0}^n F_{k_{u,0}}^j(i||g) \oplus (1 \oplus \hat{u}) F_{k_{u,\hat{u}}}^{i_0}(i||g) \oplus k_{w,\hat{w}}^i \\
&\quad \oplus \hat{u} \left(\tilde{C}_1^i \oplus \bigoplus_{j=1; j \neq i_0}^n F_{k_{u,0}}^j(i||g) \oplus \bigoplus_{j=1; j \neq i_0}^n F_{k_{u,1}}^j(i||g) \oplus F_{k_{u,\hat{u}}}^{i_0}(i||g) \oplus k_{v,\hat{v}}^i \right) \\
&= \bigoplus_{j=1}^n F_{k_{v,\hat{v}}}^j(i||g') \oplus \bigoplus_{j=1}^n F_{k_{u,\hat{u}}}^j(i||g) \oplus k_{w,\hat{w}}^i \oplus \hat{u}(\tilde{C}_1^i \oplus k_{v,\hat{v}}^i),
\end{aligned}$$

where the last step used $(1 \oplus \hat{u}) F_{k_{u,0}}^j(i||g) \oplus \hat{u} F_{k_{u,1}}^j(i||g) = F_{k_{u,\hat{u}}}^j(i||g)$ for all $j \in [n]$. So $\tilde{C}_2^{i,\hat{v}}$ is identical to $C_2^{i,\hat{v}}$ of the simulated circuit.

In conclusion, if F is ccrnd , the step between $\widetilde{\text{Ideal}}$ and $\widetilde{\text{Hyb}}$ is indistinguishable.

Flip in the Real Protocol. For the final step, we need to show that the Flip event also occurs with negligible probability in the real protocol execution. We build a distinguisher $\mathcal{D}$ for ccrnd which works the same as $\mathcal{D}'$ from above except that it picks i_0 at random. $\mathcal{A}$ receives the simulated garbled circuit from $\mathcal{D}'$ and returns the modified version. Assuming $\mathcal{A}$ found additive errors to cause the Flip event for some gate (g, g') in party P_i 's circuit, then with probability $1/n$, $i = i_0$. $\mathcal{D}$ can now evaluate the circuit and obtain the difference between the active output key $k_{w,\hat{w}}^{i_0}$ it constructed and the output key $k_{w,\hat{w} \oplus 1}^{i_0}$ it obtained during evaluation of the modified circuit as $\Delta_g^* := k_{w,\hat{w}}^{i_0} \oplus k_{w,\hat{w} \oplus 1}^{i_0}$. $\mathcal{D}$ computes $F_{k_{w,\hat{w}}^{i_0} \oplus \Delta_g^*}^{i_0}(i_0||g^*) \oplus p^* \Delta_g^*$ where g^* denotes the gate where the output wire of (g, g') is either left or right input and p^* denotes the appropriate input from the oracle call in Equation (6) for gate g^* , i.e., p^* is either $\hat{v} \oplus \lambda_v$ or λ_u . If the returned value is indeed the same $\mathcal{O}(k_{w,\hat{w}}^{i_0}, i_0, g^*, p^*)$ as was returned by the oracle during the construction phase of the garbled circuit in Equation (6), then $\mathcal{D}$ concludes that $\mathcal{O}(k, i, g, b) = F_{k \oplus \Delta_{i_0}}(i||g) \oplus b \Delta^{i_0}$. If $\mathcal{D}$ cannot conclude this for any gate with a difference Δ_g^* , it returns $\mathcal{O}(k, i, g, b) = \text{Rand}$. Let $\text{Adv}_{\text{Flip}}^{\mathcal{A}}(\kappa)$ be the probability that $\mathcal{A}$ creates the Flip event, then $\mathcal{D}$ succeeds with probability $(1/n) \cdot \text{Adv}_{\text{Flip}}^{\mathcal{A}}(\kappa)$ since if $\mathcal{O} \neq \text{Rand}$,

$$F_{k_{w,\hat{w}}^{i_0} \oplus \Delta_g^*}^{i_0}(i_0||g^*) \oplus p^* \Delta_g^* = F_{k_{w,\hat{w}}^{i_0} \oplus \Delta^{i_0}}^{i_0}(i_0||g^*) \oplus p^* \Delta^{i_0} = \mathcal{O}(k_{w,\hat{w}}^{i_0}, i_0, g^*, p^*),$$

since $\Delta_g^* = \Delta^{i_0}$. Further, if $\mathcal{O} = \text{Rand}$, $\mathcal{A}$'s view is as in $\widetilde{\text{Ideal}}$ and $\mathcal{D}$ outputs the wrong decision only with negligible probability. If $\text{Adv}_{\text{Flip}}^{\mathcal{A}}(\kappa)$ is non-negligible in κ , then $\mathcal{D}$ has a non-negligible advantage against the ccrnd PRF F . Since we assume F is ccrnd secure, the Flip event can only happen with negligible probability.

This concludes the proof. $\square$

4.2 AND Gate Evaluation in Authenticated Garbling

The evaluator P_1 received the garbled circuit $\{C_{1,g}^i, C_{2,g}^{i,0}, C_{2,g}^{i,1}\}_{i \neq 1, g \in W_\wedge}$ from the garbling phase $\Pi_{\text{AuthGarble}}$.

Let $\hat{u}, \hat{v}$ be the external values for the input wires and $\{k_{u,\hat{u}}^i, k_{v,\hat{v}}^i\}_{i \neq 1}$ be the active keys during evaluation. P_1 computes,

$$(k^i \| \{ \langle r \Delta^j \rangle_i \}_{j \neq i, 1}) := \hat{F}_{k_{u,\hat{u}}^i}(g, i) \oplus \hat{u} C_1^i \oplus \hat{F}_{k_{v,\hat{v}}^i}(g', i) \oplus C_{2,\hat{v}}^i, \quad \text{for each } i \neq 1.$$

Followed by obtaining the evaluation key via,

$$k_{w,\hat{w}}^i := k^i \oplus \hat{u} k_{v,\hat{v}}^i \oplus \bigoplus_{j \neq i} \langle r \Delta^j \rangle_j, \quad \text{for each } i \neq 1,$$

where $\langle r \Delta^i \rangle_1 := \hat{u} \langle r_1 \Delta^i \rangle_1 \oplus \langle r_{2,\hat{v}} \Delta^i \rangle_1$, for each $i \neq 1$. Note that the shares of $r \Delta^i$ come from decryptions of other party's circuits. The external value is computed as $\hat{w} := b_w \oplus \text{lsb}(k_{w,\hat{w}}^2)$. Now P_1 can evaluate the next gate using $(\{k_{w,\hat{w}}^i\}_{i \neq 1}, \hat{w})$.

Reaching an output gate, P_1 sends $\hat{w}$ to the respective party. Then the output can be obtained as $y := \hat{w} \oplus \lambda_w$ where λ_w is the wire mask that was opened to the party receiving output. Before accepting the outputs, we need a few checks to protect against malicious behaviour.

Checks. So far, P_1 cannot be sure that the AND gate was correctly garbled and the garblers P_i , $i \neq 1$, cannot be sure that the external value of an output wire that P_1 broadcasts is obtained via evaluation of the circuit.

The latter is checked by having P_1 send in addition all wire labels $k_{w,\hat{w}}^i$ of P_i encountered during evaluation to P_i . This can be compressed to $\mathcal{O}(\kappa)$ bits of communication via a collision resistant hash function.

The former check uses P_1 's Δ^1 . The garblers send $\langle t \Delta^1 \rangle_i$ to P_1 , where $t = (\hat{u} \oplus \lambda_u)(\hat{v} \oplus \lambda_v) \oplus \lambda_u \lambda_v \oplus \lambda_w$, which should be $t = 0$, if $uv = w$. For this, the parties compute,

$$\begin{aligned} \langle t \Delta^1 \rangle_1 &:= \hat{u} \langle r_1 \Delta^1 \rangle_1 \oplus \langle r_{2,\hat{v}} \Delta^1 \rangle_1 \oplus (\hat{u} \hat{v} \oplus \hat{w}) \Delta^1, \\ \langle t \Delta^1 \rangle_i &:= \hat{u} \langle r_1 \Delta^1 \rangle_i \oplus \langle r_{2,\hat{v}} \Delta^1 \rangle_i, \end{aligned} \quad \text{for } i \neq 1.$$

P_1 can sum the received shares together with $\langle t \Delta^1 \rangle_1$ and check if $t \Delta^1 = 0$. This can be compressed via an almost universal linear hash function. The evaluation protocol Π_{AuthEval} is detailed in Figure 11.

Correctness. We will sketch correctness of the obtained active key $k_{w,\hat{w}}^i$ and the external value $\hat{w}$ given an honestly generated circuit.

Note first that

$$\begin{aligned} \langle r \Delta^j \rangle_i &= \hat{u} \langle r_1 \Delta^j \rangle_i \oplus \langle r_{2,\hat{v}} \Delta^j \rangle_i = \langle (\hat{u} \lambda_v \oplus \lambda_w \oplus \lambda_u \lambda_v \oplus \hat{v} \lambda_u) \Delta^j \rangle_i, \\ k_{w,\hat{w}}^i &= k_{w,0}^i \oplus \hat{u} \hat{v} \Delta^i \oplus \hat{u} \langle r_1 \Delta^i \rangle_i \oplus \langle r_{2,\hat{v}} \Delta^i \rangle_i \oplus \bigoplus_{j \neq i} \langle r \Delta^j \rangle_j \\ &= k_{w,0}^i \oplus (\hat{u} \hat{v} \oplus \hat{u} \lambda_v \oplus \lambda_w \oplus \lambda_u \lambda_v \oplus \hat{v} \lambda_u) \Delta^i, \end{aligned}$$

which is $k_{w,\hat{w}}^i = k_{w,0}^i \oplus \hat{w} \Delta^i$ (cf. Lemma 1). Then the external value is

$$\hat{w} = b_w \oplus \text{lsb}(k_{w,\hat{w}}^2) = \text{lsb}(k_{w,0}^2) \oplus \text{lsb}(k_{w,0}^2 \oplus \hat{w} \Delta^2) = \hat{w} \text{lsb}(\Delta_2) = \hat{w}.$$

Protocol Π_{AuthEval}

PARAMETERS: Finite field $\mathbb{F}_{2^\kappa}$, $\hat{F}$ a concatenated ccrrnd PRF (cf. Definition 6), $\mathbf{H}$ an almost universal linear hash function, H a collision resistant hash function.

PUBLIC INPUT: Circuit C

PRIVATE INPUT: Each party P_i inputs $x_i \in \mathcal{I}_i$

Inputs: Let x be an input of party P_i for wire $w \in \mathcal{I}_i$.

1. P_i sets $\hat{w} = x \oplus \lambda_w$ and broadcasts $\hat{w}$.
2. All parties P_i , $i \neq 1$, send $k_{w,\hat{w}}^i$ to P_1 . P_1 stores $(\{k_{w,\hat{w}}^i\}_{i \neq 1}, \hat{w})$.

Evaluation: P_1 traverses the circuit in topological order.

1. For every XOR gate (u, v, w) , party P_1 computes $k_{w,\hat{w}}^i = k_{u,\hat{w}}^i \oplus k_{v,\hat{w}}^i$ for $i \neq 1$ and sets $\hat{w} = \hat{u} \oplus \hat{v}$.
2. For every AND gate g with wires (u, v, w) , party P_1 computes
 - (a) $(k^i || \{r_j^i\}_{j \neq i, 1}) := \hat{F}_{k_{u,\hat{w}}^i}^i(g, i) \oplus \hat{F}_{k_{v,\hat{w}}^i}^i(g', i) \oplus \hat{u}C_{1,g}^{i,\hat{w}} \oplus C_{2,g}^{i,\hat{w}}$ for $i \neq 1$.
 - (b) $k_{w,\hat{w}}^i := k^i \oplus \hat{u}k_{v,\hat{w}}^i \oplus \bigoplus_{j \neq i} r_j^i$ where $r_1^i := \hat{u}\langle \lambda_v \Delta^i \rangle_1 \oplus \langle \lambda_u \lambda_v \Delta^i \rangle_1 \oplus \langle \lambda_w \Delta^i \rangle_1 \oplus \hat{v}\langle \lambda_u \Delta^i \rangle_1$ is computed from P_1 's shares.
 - (c) Party P_1 sets $\hat{w} := b_w \oplus \text{lsb}(k_{w,\hat{w}}^2)$, and stores $(\{k_{w,\hat{w}}^i\}_{i \neq 1}, \hat{w})$.

Check: P_1 samples a seed $\chi \leftarrow \mathbb{F}_{2^\kappa}$ and computes $h^i := H(\{k_{w,\hat{w}}^i\}_{w \in W_\wedge})$ for each $i \neq 1$.

1. P_1 broadcasts $\chi, \{\hat{w}\}_{w \in W_\wedge}$ and sends h^i to each P_i , $i \neq 1$.
2. Each P_i , $i \neq 1$, checks $h^i = H(\{k_{w,0}^i \oplus \hat{w} \Delta^i\}_{w \in W_\wedge})$ and aborts if the check does not hold.
3. Each P_i traverses the circuit in topological order and computes
 - For every XOR gate (u, v, w) , $\hat{w} := \hat{u} \oplus \hat{v}$.
 - For every AND gate g with wires (u, v, w) , $\langle t_g \Delta^1 \rangle_i := \hat{u}\langle r_1 \Delta^1 \rangle_i \oplus \langle r_{2,\hat{v}} \Delta^1 \rangle_i$.
Each P_i , $i \neq 1$, sends $z^i := \mathbf{H}_\chi(\{\langle t_g \Delta^1 \rangle_i\}_{g \in W_\wedge})$ to P_1 .
4. P_1 computes $\langle t_g \Delta^1 \rangle_1 := \hat{u}\langle r_1 \Delta^1 \rangle_1 \oplus \langle r_{2,\hat{v}} \Delta^1 \rangle_1 \oplus (\hat{u}\hat{v} \oplus \hat{w})\Delta^1$ and sets $z^1 := \mathbf{H}_\chi(\{\langle t_g \Delta^1 \rangle_1\}_{g \in W_\wedge})$. P_1 aborts if $\sum_{i=1}^n z^i \neq 0$.

Output: For every output wire $w \in \mathcal{O}_i$ of party P_i , it sets the output $y = \lambda_w \oplus \hat{w}$.

Figure 11: Protocol for Online Circuit Evaluation using authenticated garbling.

Security. We prove security in the following Theorem 3. Note that unlike before, we don't explicitly separate garbling and evaluation phase, thus there is no equivalent $\mathcal{F}_{\text{Garble}}$ functionality, and we prove that both protocols $\Pi_{\text{AuthGarble}}$ and Π_{AuthEval} together realize secure computation of any function f .

Theorem 3. *Let f be any function represented by circuit C_f , then $\Pi_{\text{AuthGarble}}$ followed by Π_{AuthEval} securely computes f in the $\mathcal{F}_{\text{Prep}}$ -hybrid model against a static, malicious adversary corrupting up to $n - 1$ parties, if F is a ccrrnd secure PRF for naturally derived keys (see Definition 1), $\mathbf{H}$ is a $|W_\wedge|/2^\kappa$ -universal linear hash function and H is a collision-resistant hash function.*

We first prove two related lemmas before moving on to the main proof.

Lemma 3. *In an execution of $\Pi_{\text{AuthGarble}}$ and Π_{AuthEval} (cf. Figures 9 and 11) where P_1 is honest and the adversary $\mathcal{A}$ corrupts any number of garblers, let $x_w^i = \hat{w} \oplus \lambda_w$ for each input wire $w \in \mathcal{I}_i$, for $i \in [n]$. Then either, P_1 outputs $y^1 = f_1(x^1, \dots, x^n)$ or aborts, except with probability negligible in κ if $\mathbf{H}$ is $|W_\wedge|/2^\kappa$ -universal. The function f_1 denotes P_1 's output of f .*

Proof. If the adversary $\mathcal{A}$ does not supply

- a different input wire label $k_{w,\hat{w}}^j$ (in Step 2 in **Inputs**, Figure 11),
- a different b_w value in case P_2 is corrupted (in Step 3 in **Output wire masks and circuit**, Figure 9), or

- a different garbled circuit $\{C_{1,g}^j, C_{2,g}^{j,0}, C_{2,g}^{j,1}\}$ (in Step 2 in **Output wire masks and circuit**, Figure 9),

then no cheating has occurred and P_1 outputs y^1 since our scheme is correct.

Thus assume that $\mathcal{A}$ supplied some incorrect values. Let $\{\hat{w}\}_{w \in W_\wedge}$ denote the active external values where we only consider external values of AND gates since external values of XOR gates are always computed from the outputs of AND gates, so an error of $\hat{w}, w \in W_\oplus$ always originates in some $\hat{w}', w' \in W_\wedge$. If honest evaluation still yields the same active external values, P_1 outputs the correct y^1 . Therefore, we can finally assume that the incorrect values of $\mathcal{A}$ made P_1 obtain the active external values $\{\hat{w} \oplus e_w\}_{w \in W_\oplus \cup W_\wedge}$ for at least one w where $e_w = 1$.

We will show that P_1 aborts in Step 4 during the check phase in Π_{AuthEval} (Figure 11), except with negligible probability. P_1 computes

$$\begin{aligned} \sum_{i=1}^n z^i &= \mathbf{H}_\chi \left(\left\{ \langle t\Delta^1 \rangle_1 + e_w \Delta^1 + \sum_{i \neq 1} \langle t\Delta^1 \rangle_i + \sum_{j \in \mathcal{C}} E_w^j \right\}_{w \in W_\wedge} \right) \\ &= \mathbf{H}_\chi \left(\left\{ e_w \Delta^1 + \sum_{j \in \mathcal{C}} E_w^j \right\}_{w \in W_\wedge} \right), \end{aligned}$$

which is a non-zero input to $\mathbf{H}$, except if $\mathcal{A}$ guesses the independently uniformly random secret Δ^1 and sets $\sum_{j \in \mathcal{C}} E^j = \Delta^1$ with probability $2^{-\kappa}$. Then using the $|W_\wedge|$ -universal property of $\mathbf{H}$, we conclude that P_1 aborts except with probability $(|W_\wedge| + 1)/2^\kappa$. $\square$

Lemma 4. *In an execution of $\Pi_{\text{AuthGarble}}$ and $\Pi_{\text{AuthGarble}}$ (cf. Figures 9 and 11) where P_1 is corrupted and the adversary $\mathcal{A}$ corrupts at most $n - 2$ garblers, let $x_w^i = \hat{w} \oplus \lambda_w$ for each input wire $w \in \mathcal{I}_i$, for $i \in [n]$. Then either, $P_i, i \notin \mathcal{C}$, outputs $y^i = f_i(x^1, \dots, x^n)$ or aborts, except with probability negligible in κ if H is a collision-resistant hash function. The function f_i denotes P_i 's output of f .*

Proof. Let $\{\hat{w} \oplus e_w\}_{w \in W_\wedge}$ be the active external values that the corrupted P_1 broadcasts to honest garblers $P_i, i \notin \mathcal{C}$, where $\hat{w}$ denotes the value in an honest execution for the same inputs. Clearly, if all $e_w = 0$, then P_i will accept h^i sent by P_1 if it is honestly computed.

Suppose, P_1 sends active external values where $e_{w'} = 1$ for at least one wire w' and some h' to make P_i accept. P_1 can compute h' honestly by guessing the missing key $k_{w', \hat{w}'}^i \oplus \Delta^i$ which succeeds only with probability $1/2^\kappa$ since Δ^i is sampled uniformly and independently from $k_{w', \hat{w}'}^i$. Otherwise, P_1 must produce $h' = H(X) = h^i$. Since H is collision resistant, $\mathcal{A}$ will only be able to find a $H(X)$ with probability negligible in κ . $\square$

We move on to the the proof of Theorem 3.

Proof. We prove a slightly modified variant of $\Pi_{\text{AuthGarble}}$ and Π_{AuthEval} where output masks and circuit (last step in $\Pi_{\text{AuthGarble}}$) are revealed after the input phase in Π_{AuthEval} . This clearly preserves correctness but allows us the simulation.

Let $\mathcal{A}$ be the adversary. The simulator (given below) internally runs $\mathcal{A}$, simulates the honest parties towards $\mathcal{A}$ and forwards all communication between $\mathcal{A}$ and the environment $\mathcal{Z}$. In the $\mathcal{F}_{\text{Prep}}$ -hybrid model, the simulator also emulates $\mathcal{F}_{\text{Prep}}$ towards $\mathcal{A}$.

We divide the proof into two parts: first where we assume P_1 is honest and second where P_1 is corrupted.

Case I: P_1 is honest. We construct the simulator as follows.

Init: $\mathcal{S}$ emulates the Init command on $\mathcal{F}_{\text{Prep}}$.

Garbling: $\mathcal{S}$ emulates all RandomBit and ANDOutputMask commands on $\mathcal{F}_{\text{Prep}}$.

Inputs: $\mathcal{S}$ sets the input for honest parties to 0, otherwise runs the protocol honestly.

1. $\mathcal{S}$ receives $\hat{w}$ and then extracts $x^j := \hat{w} \oplus \lambda_w$ for all $w \in \mathcal{I}_j$ for $j \in \mathcal{C}$.
2. $\mathcal{S}$ receives $\tilde{k}_{w,\hat{w}}^j$ for $j \in \mathcal{C}$ from $\mathcal{A}$. It stores $\hat{w}, \{\tilde{k}_{w,\hat{w}}^j\}_{j \in \mathcal{C}}$.

Output wire masks and circuit: The simulator provides the extracted input to f and obtains the corrupted parties' output y_w for $w \in \mathcal{O}_j, j \in \mathcal{C}$. The simulator samples the active external values $\hat{w}$, choosing $\hat{w} \leftarrow \$\{0, 1\}$ for AND gates and $\hat{w} := \hat{u} \oplus \hat{v}$ for XOR gates.

1. The simulator computes $\lambda_w := \hat{w} \oplus y_w$ for $w \in \mathcal{O}_j$ for $j \in \mathcal{C}$, and sends λ_w to $\mathcal{A}$ (via emulation of OpenValue of $\mathcal{F}_{\text{Prep}}$).
2. The simulator receives the garbled circuit $\{\tilde{C}_{1,g}^j, \tilde{C}_{2,g}^{j,0}, \tilde{C}_{2,g}^{j,1}\}_{j \in \mathcal{C}}$ from $\mathcal{A}$.
3. If $2 \in \mathcal{C}$, the simulator receives $\{\tilde{b}_w\}_{w \in W_\wedge}$ from $\mathcal{A}$.

Evaluation: The simulator first computes the values w^* for each wire in the circuit computing $f(x^1, \dots, x^n)$ where $x^i = 0$ for $i \notin \mathcal{C}$ and then obtains the corresponding wire label mask $\lambda_w := \hat{w} \oplus w^*$. The simulator computes the active path $\hat{w} \oplus e_w$ by evaluating the corrupted garbler's ciphertexts according to the protocol where it sets r_i^j during P_j 's circuit evaluation to be a share of $(\hat{u}\lambda_v \oplus \lambda_w \oplus \lambda_u\lambda_v \oplus \hat{v}\lambda_u)\Delta^j$ (consistent with random values given by $\mathcal{F}_{\text{Prep}}$ to $\mathcal{A}$).

Check: The simulator runs the check phase according to the protocol but also aborts if the obtained active external values $\hat{w} \oplus e_w$ do not correspond to the chosen active external values $\hat{w}$, i.e., if there is a $e_w = 1$.

Output: Nothing to simulate.

We will now argue that the view of $\mathcal{A}$ induced by $\mathcal{S}$ is computationally indistinguishable from the real protocol execution. If $\mathcal{S}$ does not abort, the computation is correct. Further, all interaction with $\mathcal{F}_{\text{Prep}}$ yields identical views for $\mathcal{A}$. In case that $\mathcal{A}$ supplies incorrectly computed garbled circuit values or wire labels that lead to incorrect computation, $\mathcal{S}$ always aborts while in the real protocol, P_1 runs the check $t\Delta^1 = 0$. Lemma 3 shows that the distinguishing advantage between these two cases is negligible in κ . This concludes the proof for case I.

Case II: P_1 is corrupted. We construct the simulator as follows.

Init $\mathcal{S}$ emulates the Init command on $\mathcal{F}_{\text{Prep}}$.

Garbling:

Circuit input wire masks and keys.

1. $\mathcal{S}$ emulates the RandomBit command on $\mathcal{F}_{\text{Prep}}$.
2. $\mathcal{S}$ samples $k_{w,\hat{w}}^i \leftarrow \$\{0, 1\}^\kappa$, for $i \notin \mathcal{C}$.

Intermediate wire masks and keys. For an AND gate, $\mathcal{S}$ emulates the RandomBit command on $\mathcal{F}_{\text{Prep}}$ and samples $k_{w,\hat{w}}^i \leftarrow \$\{0, 1\}^\kappa$ for $i \notin \mathcal{C}$. For an XOR gate, $\mathcal{S}$ emulates the Add command on $\mathcal{F}_{\text{Prep}}$ and computes $k_{w,\hat{w}}^i := k_{u,\hat{u}}^i \oplus k_{v,\hat{v}}^i$ for $i \notin \mathcal{C}$.

Generate garbled gates. For each AND gate (g, g') with input wires u and v and output wire w :

1. $\mathcal{S}$ emulates the ANDOutputMask command on $\mathcal{F}_{\text{Prep}}$. Note that now $\mathcal{S}$ knows $\langle \lambda_u \Delta^i \rangle_j, \langle \lambda_v \Delta^i \rangle_j, \langle \lambda_w \Delta^i \rangle_j, \langle \lambda_u \lambda_v \Delta^i \rangle_j$ for $i \in [n], j \in \mathcal{C}$.

2. $\mathcal{S}$ does nothing.

Inputs: $\mathcal{S}$ sets the input for honest parties to 0, otherwise runs the protocol honestly.

1. $\mathcal{S}$ receives $\hat{w}$ and then extracts $x := \hat{w} \oplus \lambda_w$ for all $w \in \mathcal{I}_j$ for $j \in \mathcal{C}$.
2. $\mathcal{S}$ sends $k_{w,\hat{w}}^i$, $i \notin \mathcal{C}$ to $\mathcal{A}$.

Output wire masks and circuit: The simulator provides the extracted input to f and obtains the corrupted parties' output y_w for $w \in \mathcal{O}_j$, $j \in \mathcal{C}$. $\mathcal{S}$ first computes the values w^* for each wire in the circuit computing $f(x^1, \dots, x^n)$ where $x^i = 0$ for $i \notin \mathcal{C}$ and then obtains the corresponding wire label mask $\lambda_w := \hat{w} \oplus w^*$. It can now compute the active path $(\hat{w}, \{k_{w,\hat{w}}^i\}_{i \notin \mathcal{C}})$. Note that $\hat{w}, k_{w,\hat{w}}^i$ for input wires have already been obtained in the previous step. $\mathcal{S}$ steps through the circuit and computes, if w is the output wire of an XOR gate, $\hat{w} := \hat{u} \oplus \hat{v}$, otherwise, if w is the output of an AND gate, $\hat{w} \leftarrow \{0, 1\}$.

1. The simulator computes $\lambda_w := \hat{w} \oplus y_w$ for $w \in \mathcal{O}_j$ for $j \in \mathcal{C}$, and sends λ_w to $\mathcal{A}$ (via emulation of `OpenValue` of $\mathcal{F}_{\text{Prep}}$).
2. The simulator chooses for $i \notin \mathcal{C}$,

$$\begin{aligned} k_1^i &\leftarrow \{0, 1\}^\kappa, \\ \langle r_1 \Delta^j \rangle_i &\leftarrow \{0, 1\}^\kappa && \text{for } j \neq i, 1, \\ \langle r_{2,\hat{v}} \Delta^j \rangle_i &\leftarrow \{0, 1\}^\kappa && \text{for } j \neq i, 1, \\ C_{2,g}^{i,\hat{v} \oplus 1} &\leftarrow \{0, 1\}^{(n-1)\kappa}, \end{aligned}$$

and sets

$$\begin{aligned} C_{1,g}^i &:= k_1^i \| \{ \langle r_1 \Delta^j \rangle_i \}_{j \neq i, 1}, \\ C_{2,g}^{i,\hat{v}} &:= \hat{F}_{k_{u,\hat{u}}^i}(g, i) \oplus \hat{F}_{k_{v,\hat{v}}^i}(g', i) \\ &\quad \oplus \left(k_{w,\hat{w}}^i \oplus \hat{u}(k_1^i \oplus k_{v,\hat{v}}^i) \oplus \bigoplus_{j \neq i} (\hat{u} \langle r_1 \Delta^i \rangle_j \oplus \langle r_{2,\hat{v}} \Delta^i \rangle_j) \| \{ \langle r_{2,\hat{v}} \Delta^j \rangle_i \}_{j \neq i, 1} \right). \end{aligned}$$

Note that $\langle r_1 \Delta^i \rangle_j$ and $\langle r_{2,\hat{v}} \Delta^i \rangle_j$ for $j \in \mathcal{C}$ can be derived by $\mathcal{S}$ based on the outputs of $\mathcal{F}_{\text{Prep}}$ to $\mathcal{A}$. The simulator $\mathcal{S}$ sends $\{C_{1,g}^i, C_{2,g}^{i,0}, C_{2,g}^{i,1}\}_{g \in W_\wedge}$ on behalf of honest garbler P_i , $i \notin \mathcal{C}$ to $\mathcal{A}$.

3. If $2 \notin \mathcal{C}$, $\mathcal{S}$ computes $b_w := \text{lsb}(k_{w,\hat{w}}^2) \oplus \hat{w}$ for every $w \in W_\wedge$ and sends $\{b_w\}_{w \in W_\wedge}$ to $\mathcal{A}$.

Evaluation: Nothing to simulate.

Check:

1. Let $\chi, \{\hat{w} \oplus e_w\}_{w \in W_\wedge}, h^i$ be the values sent by $\mathcal{A}$ to an honest garbler P_i , $i \notin \mathcal{C}$.
2. The simulator compares the active external values $\{\hat{w} \oplus e_w\}_{w \in W_\wedge}$ with the chosen ones and aborts if any $e_w = 1$.
3. Let $\langle t_g \Delta^1 \rangle_j$ be the honestly computed values of the corrupted parties $j \in \mathcal{C}$. $\mathcal{S}$ can obtain these values based on the output of the emulated $\mathcal{F}_{\text{Prep}}$ to $\mathcal{A}$. $\mathcal{S}$ samples $\langle t_g \Delta^1 \rangle_i \leftarrow \mathbb{F}_{2^\kappa}$ subject to $\sum_{i=1}^n \langle t_g \Delta^1 \rangle_i = 0$. $\mathcal{S}$ sends $h^i := \mathbf{H}_\chi(\{\langle t_g \Delta^1 \rangle_i\}_{g \in W_\wedge})$ on behalf of honest garbler P_i , $i \notin \mathcal{C}$ to $\mathcal{A}$.

Output: Nothing to simulate.

We will first show that if $\mathcal{A}$ chooses to evaluate the circuit returned by the simulated honest garblers, it will learn the correct wire label. The correctness of XOR gates is evident by construction of $k_{w,\hat{w}}^i$ for $w \in W_\oplus$. Let $\hat{u}, \{k_{u,\hat{u}}^i\}_{i \notin \mathcal{C}}$ and $\hat{v}, \{k_{v,\hat{v}}^i\}_{i \notin \mathcal{C}}$ be the external values and active labels for an AND gate.

$$\begin{aligned} & \hat{F}_{k_{u,\hat{u}}^i}(g, i) \oplus \hat{F}_{k_{v,\hat{v}}^i}(g', i) \oplus \hat{u}C_{1,g}^i \oplus C_{2,g}^{i,\hat{v}} \\ &= k_{w,\hat{w}}^i \oplus \hat{u}k_{v,\hat{v}}^i \oplus \bigoplus_{j \neq i} \langle \hat{u}r_1\Delta^i \oplus r_{2,\hat{v}}\Delta^i \rangle_j \| \{ \langle \hat{u}r_1\Delta^j \rangle_i \oplus \langle r_{2,\hat{v}}\Delta^j \rangle_i \}_{j \neq i,1}, \end{aligned}$$

which yields the desired active key

$$k_{w,\hat{w}}^i \oplus \hat{u}k_{v,\hat{v}}^i \oplus \bigoplus_{j \neq i} \langle \hat{u}r_1\Delta^i \oplus r_{2,\hat{v}}\Delta^i \rangle_j \oplus \hat{u}k_{v,\hat{v}}^i \oplus \bigoplus_{j \neq i} r_j^i = k_{w,\hat{w}}^i.$$

With the correct active key $k_{w,\hat{w}}^2$, $\hat{w} = b_w \oplus \text{lsb}(k_{w,\hat{w}}^2)$, by construction.

We will now argue, via a series of hybrid transformations, that the view of $\mathcal{A}$ induced by the simulator is computationally indistinguishable from the real protocol execution. Let $1 \leq m < n$ denote the number of honest parties, we will use the following hybrids Hyb^0 (the simulator given above), $\text{Hyb}^1, \dots, \text{Hyb}^m$ (Hyb^ℓ denotes the game where we switched ℓ honest garblers to create the circuit according to the protocol), Hyb^{m+1} where (simulated) honest parties run the check in Step 2 in the check phase of Π_{AuthEval} (Figure 11) according to the protocol, and finally Hyb^{m+2} which denotes the real protocol execution.

Hyb⁰ to Hyb^m. We show that stepping from Hyb^ℓ to $\text{Hyb}^{\ell+1}$ for $0 \leq \ell < m$ is indistinguishable for $\mathcal{A}$ and $\mathcal{Z}$ if F is ccrrnd secure (cf. Definition 1). Let $i_1, \dots, i_m$ be some order of the honest garblers, we will show this by induction over ℓ where parties P_{i_j} , $j < \ell$ already create the circuit according to the protocol, while P_{i_j} , $j > \ell$ create the circuit like the simulator $\mathcal{S}$. Assume for contradiction that there exists an environment $\mathcal{Z}$ with input z and $\mathcal{A}$ that distinguishes the difference between Hyb^ℓ and $\text{Hyb}^{\ell+1}$ for $0 \leq \ell < m$ with an advantage non-negligible in κ . We will show that this breaks ccrrnd security of F by constructing a distinguisher $\mathcal{D}$ as follows.

$\mathcal{D}$ is given oracle access to $\mathcal{O}(k, i, g, b)$, and internally runs $\mathcal{Z}$ on input z , and $\mathcal{S}'$ (that internally runs $\mathcal{A}$). $\mathcal{S}'$ denotes the simulator $\mathcal{S}$ with a modified **input** and **output wire masks and circuit** step. $\mathcal{S}'$ computes the active values w^* according to the computation $f(x^1, \dots, x^n)$ where x^i denotes the real input values of honest parties $i \notin \mathcal{C}$ (chosen by $\mathcal{Z}$). Then, $\mathcal{S}'$ obtains $\hat{w} := \lambda_w \oplus w^*$ where now $\lambda_w \leftarrow_{\$} \{0,1\}$. The input phase is run according to the protocol and the garbled circuits are computed as follows. The honest garbler P_{i_j} 's circuit for $j < \ell$ is computed according to the protocol, and $\mathcal{S}'$ uses Δ^{i_j} to return valid shares of $\langle \lambda_u \Delta^{i_j} \rangle$, etc. in $\mathcal{F}_{\text{Prep}}$. For $j > \ell$, the circuit is computed as in $\mathcal{S}$, i.e., $\mathcal{S}'$ does not use Δ^{i_j} and returns random shares in $\mathcal{F}_{\text{Prep}}$.

For garbler P_{i_ℓ} (now denoted P_ℓ or ℓ for brevity) is computed as

$$\begin{aligned} \tilde{C}_{1,g}^\ell &:= h_u \oplus \hat{F}_{k_{u,\hat{u}}^\ell}(g, \ell) \oplus \left(k_{v,\hat{v}}^\ell \oplus \bigoplus_{j \neq \ell} \langle r_1 \Delta^\ell \rangle_j \| \{ \langle r_1 \Delta^j \rangle_\ell \}_{j \neq \ell,1} \right), \\ \tilde{C}_{2,g}^{\ell,\hat{v}} &:= \hat{F}_{k_{v,\hat{v}}^\ell}(g', \ell) \oplus (1 \oplus \hat{u}) \hat{F}_{k_{u,\hat{u}}^\ell}(g, \ell) \oplus \hat{u}h_u \oplus \left(k_{w,\hat{w}}^\ell \oplus \bigoplus_{j \neq \ell} \langle r_{2,\hat{v}} \Delta^\ell \rangle_j \| \{ \langle r_{2,\hat{v}} \Delta^j \rangle_\ell \}_{j \neq \ell,1} \right), \end{aligned}$$

$$\tilde{C}_{2,g}^{\ell,\hat{v}\oplus 1} := h_v \oplus (\hat{u} \oplus 1) \hat{F}_{k_{u,\hat{u}}}^{\ell}(g, \ell) \oplus \hat{u} h_u \oplus \left(k_{w,\hat{w}}^{\ell} \oplus \bigoplus_{j \neq \ell} \langle r_{2,\hat{v}\oplus 1} \Delta^{\ell} \rangle_j \| \{ \langle r_{2,\hat{v}\oplus 1} \Delta^j \rangle_{\ell} \}_{j \neq \ell, 1} \right),$$

where

$$\begin{aligned} h_u &= \mathcal{O}(k_{u,\hat{u}}^{\ell}, 1, g, \hat{v} \oplus r_1) \| \{ \mathcal{O}(k_{u,\hat{u}}^{\ell}, j, g, 0) \}_{j \neq \ell, 1}, \\ h_v &= \mathcal{O}(k_{v,\hat{v}}^{\ell}, 1, g', \lambda_u) \| \{ \mathcal{O}(k_{v,\hat{v}}^{\ell}, j, g', 0) \}_{j \neq \ell, 1}. \end{aligned}$$

Note that the oracle queries are all legal and satisfy the natural derived keys requirement since per key, $k_{u,\hat{u}}^{\ell}$ and $k_{v,\hat{v}}^{\ell}$, the inputs $1, \{j\}_{j \neq \ell, 1}$ are unique. The distinguisher $\mathcal{D}$ outputs as guess regarding the oracle whatever $\mathcal{Z}$ outputs as guess regarding the world Hyb^{ℓ} or $\text{Hyb}^{\ell+1}$.

We will now show that if $\mathcal{O} = \text{Rand}$, the constructed ciphertexts $\{\tilde{C}_{1,g}^{\ell}, \tilde{C}_{2,g}^{\ell,0}, \tilde{C}_{2,g}^{\ell,1}\}$ correspond to the ciphertexts from $\mathcal{S}$ (in other words: the output for party P_{ℓ} in Hyb^{ℓ}).

It is clear that $\tilde{C}_{1,g}^{\ell}$ and $\tilde{C}_{2,g}^{\ell,\hat{v}\oplus 1}$ are uniform random strings due to h_u and h_v , respectively, and as such indistinguishable from the ciphertexts from $\mathcal{S}$. Next, note that $\tilde{C}_{2,g}^{\ell,\hat{v}}$ behaves identical, since evaluation still returns the correct key,

$$\begin{aligned} &\hat{F}_{k_{u,\hat{u}}}^{\ell}(g, \ell) \oplus \hat{F}_{k_{v,\hat{v}}}^{\ell}(g', \ell) \oplus \hat{u} \tilde{C}_{1,g}^{\ell} \oplus \tilde{C}_{2,g}^{\ell,\hat{v}} \\ &= k_{w,\hat{w}}^{\ell} \oplus \hat{u} k_{v,\hat{v}}^{\ell} \oplus \bigoplus_{j \neq \ell} \langle \hat{u} r_1 \Delta^{\ell} \rangle_j \oplus \langle r_{2,\hat{v}} \Delta^{\ell} \rangle_j \| \{ \langle \hat{u} r_1 \Delta^j \rangle_{\ell} \oplus \langle r_{2,\hat{v}} \Delta^j \rangle_{\ell} \}_{j \neq \ell, 1}, \end{aligned}$$

thus, $k_{w,\hat{w}}^{\ell} \oplus \hat{u} k_{v,\hat{v}}^{\ell} \oplus \bigoplus_{j \neq \ell} \langle \hat{u} r_1 \Delta^{\ell} \rangle_j \oplus \langle r_{2,\hat{v}} \Delta^{\ell} \rangle_j \oplus \hat{u} k_{v,\hat{v}}^{\ell} \oplus \bigoplus_{j \neq \ell} r_j^{\ell} = k_{w,\hat{w}}^{\ell}$.

If $\mathcal{O}(k, i, g, b) = F_{k \oplus \Delta}(i \| g) \oplus b \Delta$, then all ciphertexts correspond to those in the protocol, i.e., the output for party P_{ℓ} in $\text{Hyb}^{\ell+1}$. Note first that

$$\begin{aligned} h_u &= F_{k_{u,\hat{u}\oplus 1}}^{\ell}(1 \| g) \oplus (\hat{v} \oplus r_1) \Delta^{\ell} \| \{ F_{k_{u,\hat{u}\oplus 1}}^{\ell}(j \| g) \}_{j \neq \ell, 1} \\ &= \hat{F}_{k_{u,\hat{u}\oplus 1}}^{\ell}(g, \ell) \oplus \left((\hat{v} \oplus r_1) \Delta^{\ell} \| \{ 0 \}_{j \neq \ell, 1} \right), \\ h_v &= F_{k_{v,\hat{v}\oplus 1}}^{\ell}(1 \| g') \oplus \lambda_u \Delta^{\ell} \| \{ F_{k_{v,\hat{v}\oplus 1}}^{\ell}(j \| g') \}_{j \neq \ell, 1} \\ &= \hat{F}_{k_{v,\hat{v}\oplus 1}}^{\ell}(g', \ell) \oplus \left(\lambda_u \Delta^{\ell} \| \{ 0 \}_{j \neq \ell, 1} \right). \end{aligned}$$

The first ciphertext is

$$\begin{aligned} \tilde{C}_{1,g}^{\ell} &= \hat{F}_{k_{u,\hat{u}\oplus 1}}^{\ell}(g, \ell) \oplus \hat{F}_{k_{u,\hat{u}}}^{\ell}(g, \ell) \oplus \left(k_{v,\hat{v}}^{\ell} \oplus \hat{v} \Delta^{\ell} \oplus r_1 \Delta^{\ell} \oplus \bigoplus_{j \neq \ell} \langle r_1 \Delta^{\ell} \rangle_j \| \{ \langle r_1 \Delta^j \rangle_{\ell} \}_{j \neq \ell, 1} \right) \\ &= \hat{F}_{k_{u,0}}^{\ell}(g, \ell) \oplus \hat{F}_{k_{u,1}}^{\ell}(g, \ell) \oplus \left(k_{v,0}^{\ell} \oplus \langle r_1 \Delta^{\ell} \rangle_{\ell} \| \{ \langle r_1 \Delta^j \rangle_{\ell} \}_{j \neq \ell, 1} \right), \end{aligned}$$

as in the protocol $\Pi_{\text{AuthGarble}}$. The other ciphertexts are

$$\begin{aligned} \tilde{C}_{2,g}^{\ell,\hat{v}} &= \hat{F}_{k_{v,\hat{v}}}^{\ell}(g', \ell) \oplus (1 \oplus \hat{u}) F_{k_{u,\hat{u}}}^{\ell}(g, \ell) \oplus \hat{u} \hat{F}_{k_{u,\hat{u}\oplus 1}}^{\ell}(g, \ell) \\ &\quad \oplus \left(\hat{u} (\hat{v} \oplus r_1) \Delta^{\ell} \oplus k_{w,\hat{w}}^{\ell} \oplus \bigoplus_{j \neq \ell} \langle r_{2,\hat{v}} \Delta^{\ell} \rangle_j \| \{ \langle r_{2,\hat{v}} \Delta^j \rangle_{\ell} \}_{j \neq \ell, 1} \right) \end{aligned}$$

$$\begin{aligned}
&= \hat{F}_{k_{v,\hat{v}}}^\ell(g', \ell) \oplus F_{k_{u,0}}^\ell(g, \ell) \\
&\quad \oplus \left(k_{w,0}^\ell \oplus (\hat{w} \oplus \hat{u}(\hat{v} \oplus r_1) \oplus r_{2,\hat{v}}) \Delta^\ell \oplus \langle r_{2,\hat{v}} \Delta^\ell \rangle_\ell \| \{ \langle r_{2,\hat{v}} \Delta^j \rangle_\ell \}_{j \neq \ell, 1} \right) \\
&= \hat{F}_{k_{v,\hat{v}}}^\ell(g', \ell) \oplus F_{k_{u,0}}^\ell(g, \ell) \oplus \left(k_{w,0}^\ell \oplus \langle r_{2,\hat{v}} \Delta^\ell \rangle_\ell \| \{ \langle r_{2,\hat{v}} \Delta^j \rangle_\ell \}_{j \neq \ell, 1} \right),
\end{aligned}$$

since $\hat{w} \oplus \hat{u}(\hat{v} \oplus r_1) \oplus r_{2,\hat{v}} = 0$. And,

$$\begin{aligned}
\tilde{C}_{2,g}^{\ell, \hat{v} \oplus 1} &= \hat{F}_{k_{v,\hat{v} \oplus 1}}^\ell(g', \ell) \oplus (\hat{u} \oplus 1) \hat{F}_{k_{u,\hat{u}}}^\ell(g, \ell) \oplus \hat{u} \hat{F}_{k_{u,\hat{u} \oplus 1}}^\ell(g, \ell) \\
&\quad \oplus \left(\lambda_u \Delta^\ell \oplus \hat{u}(\hat{v} \oplus r_1) \Delta^\ell \oplus k_{w,\hat{w}}^\ell \oplus \bigoplus_{j \neq \ell} \langle r_{2,\hat{v} \oplus 1} \Delta^\ell \rangle_j \| \{ \langle r_{2,\hat{v} \oplus 1} \Delta^j \rangle_\ell \}_{j \neq \ell, 1} \right) \\
&= \hat{F}_{k_{v,\hat{v} \oplus 1}}^\ell(g', \ell) \oplus \hat{F}_{k_{u,0}}^\ell(g, \ell) \oplus (k_{w,0}^\ell \oplus \langle r_{2,\hat{v} \oplus 1} \Delta^\ell \rangle_\ell \\
&\quad \oplus (\hat{w} \oplus \lambda_u \oplus \hat{u}(\hat{v} \oplus r_1) \oplus r_{2,\hat{v} \oplus 1}) \Delta^\ell \| \{ \langle r_{2,\hat{v} \oplus 1} \Delta^j \rangle_\ell \}_{j \neq \ell, 1}) \\
&= \hat{F}_{k_{v,\hat{v} \oplus 1}}^\ell(g', \ell) \oplus \hat{F}_{k_{u,0}}^\ell(g, \ell) \oplus \left(k_{w,0}^\ell \oplus \langle r_{2,\hat{v} \oplus 1} \Delta^\ell \rangle_\ell \| \{ \langle r_{2,\hat{v} \oplus 1} \Delta^j \rangle_\ell \}_{j \neq \ell, 1} \right),
\end{aligned}$$

since $\hat{w} \oplus \lambda_u \oplus \hat{u}(\hat{v} \oplus r_1) \oplus r_{2,\hat{v} \oplus 1} = 0$. Now we set $\hat{v} = a$ and obtain the ciphertexts $C_{2,g}^{\ell, a}$, $a \in \{0, 1\}$ from $\Pi_{\text{AuthGarble}}$.

Hyb^{m+1}. In this hybrid game, each (simulated) honest garbler runs the check in Step 2 in the check phase of Π_{AuthEval} (Figure 11) according to the protocol. This is now possible since all honest parties use Δ^i , $i \in \mathcal{C}$. Using Lemma 4, the distinguishing gap between the check in the real protocol and always aborting when an incorrect active path was sent by P_1 (as in Hyb^m) is negligible in κ .

Hyb^{m+2}. In the final step, we let each honest garbler compute h^i (in Step 4 in the check phase of Π_{AuthEval} , Figure 11) according to the protocol. Further, all emulation of $\mathcal{F}_{\text{Prep}}$ will now behave as the real functionality. The view of $\mathcal{A}$ induced now is identical to the real world protocol execution.

This concludes the proof for case II. $\square$

4.3 Implementation and Performance

We give a prototype implementation of the online phase of our authenticated garbling protocol in Rust using the swanky framework [Gal19]. Our implementation is available on GitHub.⁶ The experiment is executed on a Linux machine with Intel(R) Core(TM) i9-9900 CPU using a simulated LAN and WAN with a bandwidth of 100 Mbps, and the results are given in Table 4. Since additional rounds of communication is required in our protocol due to the Check phase of Figure 11, our WAN results are slower than WRK17 [WRK17b], especially when there are only a few parties as the garbled circuits are smaller. On the other hand, our LAN results are better since the additional rounds do not add a lot of latency and our bandwidth consumption is lower.

⁶ <https://github.com/zama-ai/copz25-code>

Table 4: Experimental results of execution time of garbling followed by evaluation. All numbers are in milliseconds averaged after 10 repetitions.

Circuit	AES-128								AES-256							
Network	LAN				WAN				LAN				WAN			
Parties	4	8	12	16	4	8	12	16	4	8	12	16	4	8	12	16
WRK17 [WRK17b]	223	423	629	845	452	654	887	1097	304	553	860	1159	533	805	1103	1410
Ours v3	168	359	540	725	503	679	879	1069	233	474	713	982	561	804	1066	1340

5 Generating Correlations for n -Party Garbling

We use the preprocessing protocol Π_{Prep} , described in Figure 13, to obtain all the necessary material that we need for the new garbling procedures, that is random masks $\llbracket \lambda_w \rrbracket$ used for the circuit-input and circuit-output wires and the output wires of AND gates as well as the values $\langle \lambda_u \cdot \Delta^i \rangle$, $\langle \lambda_v \cdot \Delta^i \rangle$ and $\langle (\lambda_u \lambda_v \oplus \lambda_w) \cdot \Delta^i \rangle$, for each $i \in [n]$. Since the n -party VOLE functionality (Figure 4) already provides us with shares of $\lambda_u \cdot \Delta^i$, $\lambda_v \cdot \Delta^i$ and $\lambda_w \cdot \Delta^i$, the main challenge is to compute shares of $\lambda_u \lambda_v \Delta^i$.

The protocol Π_{Prep} , which makes use of $\mathcal{F}_{\text{nVOLE}}$ and $\mathcal{F}_{\text{nRandomTriple}}$ is discussed in Section 5.1. In Section 5.2 we show how to instantiate $\mathcal{F}_{\text{nRandomTriple}}$.

5.1 The Preprocessing Protocol

Here we describe Π_{Prep} in full detail. As mentioned earlier, the protocol uses two functionalities, $\mathcal{F}_{\text{nVOLE}}$ and $\mathcal{F}_{\text{nRandomTriple}}$ (Figure 12).

The $\mathcal{F}_{\text{nRandomTriple}}$ Functionality. While $\mathcal{F}_{\text{nVOLE}}$, given in Section 2.3, enables the generation of random authenticated bits $\llbracket \lambda \rrbracket$, $\mathcal{F}_{\text{nRandomTriple}}$ provides each party P_i with values $\mathbf{x}^i, \mathbf{y}^i, \mathbf{z}^i \in \mathbb{F}_{2^\rho}^m$, where $m = m_T + \ell$, as well as, $\{\langle \mathbf{x} \alpha^j \rangle_i, \langle \mathbf{y} \alpha^j \rangle_i, \langle \mathbf{z} \alpha^j \rangle_i\}_{j \neq i}$, with $\alpha^j \in \mathbb{F}_{2^\rho}$ and $\mathbf{z} = \mathbf{x} \odot \mathbf{y}$. Here m_T is the number of triples generated to be used in subsequent protocols and ℓ is a small constant number (e.g., ρ) used for generating masking values during our protocol. In Section 5.2, we show how to instantiate $\mathcal{F}_{\text{nRandomTriple}}$ with *sublinear* communication cost for the parties, which is why our circuit-independent preprocessing phase is also *sublinear*. Moreover, these triples allow us to compute the required shares of $\lambda_u \lambda_v \Delta^i$ with only 2ρ bits of communication.

The preprocessing protocol Π_{Prep} is divided in the following steps.

Initialization. In this step, each party P_i initializes $\mathcal{F}_{\text{nVOLE}}$ and $\mathcal{F}_{\text{nRandomTriple}}$, obtaining two random keys: $\Delta^i \in \mathbb{F}_{2^\kappa}$, which will be used in the garbling process, and $\alpha^i \in \mathbb{F}_{2^\rho}$, used only in the function-independent preprocessing as an authentication key to ensure correctness. This is done using the **OpenToZero** macro, which detects malicious behavior and ensures consistency of the values opened during the protocol. The main idea is for the parties to compute a random linear combination of the values being opened and verify that the result is zero. This consistency check is similarly used in HSS17 and subsequent works.

Random Bits. This command does not require any interaction among the n parties, but only calls to $\mathcal{F}_{\text{nVOLE}}$. More specifically, when a party P_i needs random bit masks for its m_I^i circuit-input wires, parties call the command on input $(\text{RandomBit}, i)$ and use $\mathcal{F}_{\text{nVOLE}}.\text{Extend}$ to obtain $\llbracket \lambda \rrbracket$, where

Functionality $\mathcal{F}_{\text{nRandomTriple}}$

PARAMETERS: Finite field $\mathbb{F}_{2^\rho}$, number of triples m .

Initialise: On input (Init) from all parties, the functionality samples $\alpha^i \leftarrow \mathbb{F}_{2^\rho}$ for each honest party P_i , $i \notin \mathcal{C}$ and sends α^i to P_i . It receives $\alpha^j \in \mathbb{F}_{2^\rho}$ from $\mathcal{A}$ for $j \in \mathcal{C}$.

Triple: On input (Triple) from all parties, the functionality

1. Samples $\mathbf{x}, \mathbf{y} \leftarrow \mathbb{F}_{2^\rho}^m$ and computes $\mathbf{z} := \mathbf{x} \odot \mathbf{y}$.
2. Samples for each $j \in [n]$, random additive n -party shares of $\mathbf{x}, \mathbf{y}, \mathbf{z}$ and $\mathbf{x}\alpha^j, \mathbf{y}\alpha^j, \mathbf{z}\alpha^j$ denoted as $\mathbf{x}^i, \mathbf{y}^i, \mathbf{z}^i$ and $\langle \mathbf{x}\alpha^j \rangle_i, \langle \mathbf{y}\alpha^j \rangle_i, \langle \mathbf{z}\alpha^j \rangle_i$, respectively.
3. Returns to P_i : $(\mathbf{x}^i, \mathbf{y}^i, \mathbf{z}^i)$ and $\{\langle \mathbf{x}\alpha^j \rangle_i, \langle \mathbf{y}\alpha^j \rangle_i, \langle \mathbf{z}\alpha^j \rangle_i\}_{j \in [n]}$.

Corrupt parties: A corrupt P_i can choose $\alpha \in \mathbb{F}_{2^\rho}$ and all output shares $(\mathbf{x}^i, \mathbf{y}^i, \mathbf{z}^i)$ and $\{\langle \mathbf{x}\alpha^j \rangle_i, \langle \mathbf{y}\alpha^j \rangle_i, \langle \mathbf{z}\alpha^j \rangle_i\}_{j \in [n]}$. The chosen values are sent to $\mathcal{F}_{\text{nRandomTriple}}$ and shares output to honest parties are recomputed accordingly.

Figure 12: Functionality for our multiplication triples.

$\lambda \in \mathbb{F}_2^{m_I^i}$ is known to P_i . To generate random masks for AND and output gates, parties call the command on input (RandomBit) and again use $\mathcal{F}_{\text{nVOLE}}.\text{Extend}$ to generate $\llbracket \lambda \rrbracket$, where $\lambda = \bigoplus_{i \in [n]} \lambda^i$ are unknown to the parties.

AND Output Masks. First, parties receive $\mathbf{x}^i, \mathbf{y}^i, \mathbf{z}^i \in \mathbb{F}_{2^\rho}^m$, along with values $\{\langle \mathbf{x}\alpha^j \rangle_i, \langle \mathbf{y}\alpha^j \rangle_i, \langle \mathbf{z}\alpha^j \rangle_i\}_{j \neq i}$ from $\mathcal{F}_{\text{nRandomTriple}}$. Then, they apply Beaver's trick to generate $\langle \lambda_{uv} \rangle := \langle \lambda_u \odot \lambda_v \rangle$ and $\langle \lambda_{uv} \alpha^i \rangle, i \in [n]$. Note that this requires opening two values $\mathbf{e}, \mathbf{f} \in \mathbb{F}_{2^\rho}^m$. Moreover, the result $\langle \lambda_{uv} \rangle$ is not an element in $\mathbb{F}_2$, but rather is a value in $\mathbb{F}_{2^\rho}$ that is either 0 or 1. Since all our values are additively shared, taking the least significant bit $\text{lsb}(\cdot)$, locally converts a $\mathbb{F}_2$ -sharing embedded in $\mathbb{F}_{2^\rho}$ into a proper $\mathbb{F}_2$ -sharing. While this works for the value λ_{uv} , it unfortunately does not hold for the bit-string multiplication $\lambda_{uv} \cdot \alpha^i, i \in [n]$. This issue is solved in *Extend labels*. This step has two main goals: first, to obtain correct shares of the bit-string product $\lambda_{uv} \cdot \alpha^i$, with $\lambda_{uv} \in \mathbb{F}_2^m$, and second to switch to the longer key Δ^i in order to derive the final shared values $\lambda_{uv} \cdot \Delta^i$ that are needed in garbling.

Consistency Check. When parties open values $\mathbf{e}$ and $\mathbf{f}$, each party P_i holds shares of $\llbracket \lambda_u \rrbracket^\kappa, \llbracket \lambda_v \rrbracket^\kappa$ authenticated with global keys $\{\Delta^j\}_{j \in [n]}$ and $\{\langle \mathbf{x}\alpha^j \rangle_i, \langle \mathbf{y}\alpha^j \rangle_i, \langle \mathbf{z}\alpha^j \rangle_i\}_{j \neq i}$. To perform the check, parties first open $\delta = \lfloor \Delta^i \rfloor - \alpha^i \in \mathbb{F}_{2^\rho}$ and then locally compute $\langle \lambda_u \alpha \rangle_i$ and $\langle \lambda_v \alpha \rangle_i$. The adversary can deviate from the protocol in different ways, either by sending incorrect shares in the opening of $\mathbf{e}$ and $\mathbf{f}$ as mentioned before, and/or sending incorrect sharing of δ in the switching-key step. We show that the probability that the check passes in case of any malicious behaviour is negligible. After the check, parties use another call to $\mathcal{F}_{\text{nVOLE}}$ to “extend” the bit-string multiplication $\langle \lambda_{uv} \cdot \alpha^i \rangle$ to $\langle \lambda_{uv} \cdot \Delta^i \rangle$, for all $i \in [n]$. This step requires an additional opening of values in $\mathbb{F}_2^m$, therefore another check is needed. Again, we prove that the probability of passing the test is negligible if the opening values are incorrect.

Protocol Π_{Prep}

PARAMETERS: Finite fields $\mathbb{F}_{2^\rho}, \mathbb{F}_{2^\kappa}$, m_i^j : number of inputs of party P_i , m_T : number of AND gates, m_R : random values, m_O : number of outputs, ℓ : number of additional random values used in the zero check, and expansion function $\text{Expand} : S \rightarrow \mathbb{F}_2^m$ with seed space S .

The parties use a macro **OpenToZero** defined as follows.

OpenToZero: To open and check if the vector $\langle \mathbf{v} \rangle$, $\mathbf{v} \in \mathbb{F}_{2^\rho}^m$, for any m is all zero,

1. Parties call $\mathcal{F}_{\text{Coin}}$ to sample an ϵ -almost uniform hash function $\mathbf{H} \leftarrow \mathcal{H}$, $\mathbf{H} \in \mathbb{F}_2^{\rho \times m}$.
2. Each P_i computes and, using $\mathcal{F}_{\text{Commit}}$, sends $h^i := \mathbf{H}(\langle \mathbf{v} \rangle_i) + \langle \bar{\mathbf{v}} \rangle$, where $\bar{\mathbf{v}}$ is an additional opening mask.
3. Each P_i opens the commitments and checks if $\sum_{j=1}^n h^j = 0$. If not, it aborts.

Initialise: All parties P_i invoke $\mathcal{F}_{\text{nVOLE}}^{2,\kappa}$ and $\mathcal{F}_{\text{nRandomTriple}}$ with command **Init**, receiving $\Delta^i \leftarrow \mathbb{F}_{2^\kappa}$ and $\alpha^i \leftarrow \mathbb{F}_{2^\rho}$, respectively.

RandomBit: This command can be called in two ways: **(RandomBit, i)** and **(RandomBit)**. If **(RandomBit, i)**:

1. Party P_i samples a seed seed_λ^i and invokes $\mathcal{F}_{\text{nVOLE}}.\text{Extend}(m_T + \ell)$ P_i receives $(\lambda, \{M_j^i[\lambda]\})$, P_j receives $K_j^j[\lambda]$, $j \neq i$.
2. Parties set $[\lambda]_i = (\lambda, \{M_j^i[\lambda], 0\}_{j \neq i})$ and $[\lambda]_j = (0, \{0, K_j^j[\lambda]\})$.

If **(RandomBit)**: Each party P_i calls $\mathcal{F}_{\text{nVOLE}}.\text{Extend}(2m_T + m_O + \ell)$ with some seed seed_{λ^i} receiving $(\lambda^i, \{M_j^i[\lambda^i], K_j^i[\lambda^j]\}_{i \neq j})$. Each party P_i sets $[\lambda]_i := (\lambda^i, \{M_j^i[\lambda], K_j^i[\lambda]\}_{j \neq i})$ where $\lambda = \bigoplus_{i \in [n]} \lambda^i$.

Add: Given $[\lambda_u], [\lambda_v]$, parties locally compute $[\lambda_u \oplus \lambda_v]$.

ANDOutputMask:

1. Each P_i calls the functionality $\mathcal{F}_{\text{nRandomTriple}}$ obtaining $\mathbf{x}^i, \mathbf{y}^i, \mathbf{z}^i$ and $\{\langle \mathbf{x}\alpha^j \rangle_i, \langle \mathbf{y}\alpha^j \rangle_i, \langle \mathbf{z}\alpha^j \rangle_i\}_{j \in [n]}$.
2. Parties broadcast their shares $\mathbf{e}^i := \lambda_u^i + \mathbf{x}^i$ and $\mathbf{f}^i := \lambda_v^i + \mathbf{y}^i$ where $\mathbf{e}^i, \mathbf{f}^i \in \mathbb{F}_{2^\rho}^{m_T}$. and reconstruct $\mathbf{e} := \sum_{j=1}^n \mathbf{e}^j$ and $\mathbf{f} := \sum_{j=1}^n \mathbf{f}^j$.
3. Each party P_i computes

$$\begin{aligned} \lambda_{uv}^i &:= \mathbf{f} \odot \mathbf{x}^i + \mathbf{e} \odot \mathbf{y}^i + \mathbf{z}^i + \mathbf{e} \odot \mathbf{f} & \text{if } i = 1, \\ \lambda_{uv}^i &:= \mathbf{f} \odot \mathbf{x}^i + \mathbf{e} \odot \mathbf{y}^i + \mathbf{z}^i & \text{if } i \neq 1, \\ \langle \lambda_{uv} \alpha^i \rangle_i &:= \mathbf{f} \odot \langle \mathbf{x} \alpha^i \rangle_i + \mathbf{e} \odot \langle \mathbf{y} \alpha^i \rangle_i + \langle \mathbf{z} \alpha^i \rangle_i + (\mathbf{e} \odot \mathbf{f}) \alpha^i, \\ \langle \lambda_{uv} \alpha^j \rangle_i &:= \mathbf{f} \odot \langle \mathbf{x} \alpha^j \rangle_i + \mathbf{e} \odot \langle \mathbf{y} \alpha^j \rangle_i + \langle \mathbf{z} \alpha^j \rangle_i & \text{for all } j \neq i. \end{aligned}$$

Note $\lambda_{uv}^i \in \mathbb{F}_{2^\rho}^m$.

4. *Check openings:*

- (a) Each P_i computes $\delta^i := \lfloor \Delta^i \rfloor - \alpha^i \in \mathbb{F}_{2^\rho}$ and broadcasts it. The parties compute $\delta := \sum_{i=1}^n \delta^i$.
- (b) Each P_i locally computes $\langle \lambda_u \alpha \rangle_i := \lambda_u^i \lfloor \Delta^i \rfloor + \sum_{j \neq i} [K_j^i[\lambda_u]] + [M_j^i[\lambda_u]] - \lambda_u^i \delta$, $\langle \lambda_v \alpha \rangle_i := \lambda_v^i \lfloor \Delta^i \rfloor + \sum_{j \neq i} [K_j^i[\lambda_v]] + [M_j^i[\lambda_v]] - \lambda_v^i \delta$, $\langle \mathbf{x} \alpha \rangle_i := \sum_{j=1}^n \langle \mathbf{x} \alpha^j \rangle_i$ and $\langle \mathbf{y} \alpha \rangle_i := \sum_{j=1}^n \langle \mathbf{y} \alpha^j \rangle_i$.
- (c) Parties call **OpenToZero** to check all the opened values. Let $\mathbf{a} = (\mathbf{e}, \mathbf{f}) \in \mathbb{F}_2^{2m_T}$, $\lambda = (\lambda_u, \lambda_v)$, $\mathbf{s} = (\mathbf{x}, \mathbf{y})$ and $\bar{\mathbf{a}} \in \mathbb{F}_2^{\rho}$ an additional mask of random authenticated opened values, parties run the macro with input $\mathbf{v}_i = \langle (\mathbf{s} + \lambda) \alpha \rangle_i - \mathbf{a} \cdot \alpha^i$ and $\langle \bar{\mathbf{a}} \alpha \rangle_i - \bar{\mathbf{a}} \alpha^i$.

5. *Extend labels:*

- (a) Parties call $\mathcal{F}_{\text{nVOLE}}.\text{Extend}(m_T)$ with some seed $\text{seed}_{\mathbf{r}^i}$ receiving $(\mathbf{r}^i, \{M_j^i[\mathbf{r}], K_j^i[\mathbf{r}]\}_{j \neq i})$.
- (b) Parties open $\mathbf{d}^i = \mathbf{r}^i - \text{lsb}(\lambda_{uv}^i)$.
- (c) Each P_i computes $\langle \lambda_{uv} \Delta^i \rangle_i := \text{lsb}(\lambda_{uv}^i) \Delta^i + \sum_{j \neq i} (K_j^i[\mathbf{r}] - \mathbf{d}^j \Delta^i)$ and $\langle \lambda_{uv} \Delta^j \rangle_i := M_j^i[\mathbf{r}]$.
- (d) P_i computes $\langle \lambda_{uv} \lfloor \Delta \rfloor \rangle_i := \sum_{j=1}^n \lfloor \langle \lambda_{uv} \Delta^j \rangle_i \rfloor$ and $\langle \lambda_{uv} \alpha \rangle_i := \sum_{j=1}^n \langle \lambda_{uv} \alpha^j \rangle_i$.
- (e) Parties call **OpenToZero** with input $\langle \mathbf{v} \rangle_i = \langle \lambda_{uv} \lfloor \Delta \rfloor \rangle_i - \langle \lambda_{uv} \alpha \rangle_i - \text{lsb}(\lambda_{uv}^i) \delta$.

6. Each party outputs $\{\langle \lambda_{uv} \Delta^j \rangle\}_{j \in [n]}$.

OpenValue: To output a value $[\mathbf{v}]$ to a single party P_i (where $\mathbf{v} \in \mathbb{F}_2^{m_O}$),

1. Each P_j , $j \neq i$, sends $\mathbf{v}^j$ to P_i . which sets $\mathbf{v} := \sum_{j=1}^n \mathbf{v}^j$.
2. Parties call **OpenToZero** with input $\langle \mathbf{v} \rangle_i := \lfloor \langle \mathbf{v} \Delta^i \rangle_i \rfloor - \mathbf{v} \lfloor \Delta^i \rfloor$ and additional mask $\bar{\mathbf{v}}_i \in \mathbb{F}_2^{\rho}$ from P_i and $\langle \mathbf{v} \rangle_j := \lfloor \langle \mathbf{v} \Delta^j \rangle_j \rfloor$ and addition mask $\bar{\mathbf{v}}_j$ from P_j , $j \neq i$.

Figure 13: Protocol Π_{Prep} for preprocessing.

Theorem 4. *The protocol Π_{Prep} given in Figure 13 securely realizes $\mathcal{F}_{\text{Prep}}$ in the $(\mathcal{F}_{\text{nRandomTriple}}, \mathcal{F}_{\text{nVOLE}}, \mathcal{F}_{\text{Coin}}, \mathcal{F}_{\text{Commit}})$ -hybrid model against a static, malicious adversary corrupting up to $n - 1$ parties.*

We first analyse the probability of passing the **OpenToZero** tests in case of malicious behaviour.

Lemma 5. *If the Step 4 succeeds, the opened values $\mathbf{e}$, $\mathbf{f}$, δ are correct except with probability at most $\frac{m+2}{2^\rho} + \frac{1}{2^m}$, where m is the number of AND gates.*

Proof. The adversary might deviate from the protocol in different ways: by sending incorrect sharings in the openings of $\mathbf{a}$, and by sending incorrect shares in the key-switching step, i.e., by sending incorrect shares of δ . We note first that the adversary must provide $\{\hat{h}^j\}_{j \in \mathcal{C}}$ without learning the honest parties' shares $\{h^i\}$ due to the commit-then-reveal steps. Let $\hat{\mathbf{a}} := \mathbf{a} + \mathbf{E}_a$ be the possibly modified opened values that the honest parties compute. And let $\hat{\delta}^i = \lfloor \Delta^i \rfloor - \alpha^i + E_\delta^i$ and $E_\delta = \sum_{i \in \mathcal{C}} E_\delta^i$. In addition, let E_m be the additive error on the input to **OpenToZero** and $E_h^j := \hat{h}^j - h^j$ another possible additive error on the check term.

Suppose $E_a, E_\delta \neq 0$. To pass the test the adversary must choose $\mathbf{E}_m$ and $\{E_h^j\}$ such that the input vector to **OpenToZero** reconstructs to the all-zero vector (and then the check passes by definition) or that $\sum h^i + \sum \hat{h}^j = 0$. In particular, using the linearity of $\mathbf{H}$, it should hold:

$$\begin{aligned} & \mathbf{H}\left((\lambda + s)\alpha - (\mathbf{a} + E_a)\alpha - \lambda E_\delta + E_m\right) + E_{\bar{a}}\alpha + E_h = \\ & \mathbf{H}\left(E_a\alpha - \lambda E_\delta + E_m\right) + E_{\bar{a}}\alpha + E_h = 0. \end{aligned}$$

By the fact that $\mathbf{H}$ is a ϵ -almost uniform hash function this happens with probability less than ϵ , assuming that all the errors are non-zero. The adversary can also pass the check if $E_a\alpha - \lambda E_\delta + E_m$ is zero and if $E_{\bar{a}}\alpha + E_h = 0$. The probability that $E_a\alpha - \lambda E_\delta + E_m = 0$ if at least one of E_a, E_δ, E_m is non-zero, is bounded by $\frac{1}{2^\rho} + \frac{1}{2^m}$; and the probability that $E_{\bar{a}}\alpha + E_h = 0$ is equivalent to the probability of guessing α . Overall, the probability of passing the check is at most $\epsilon + \frac{2}{2^\rho} + \frac{1}{2^m}$. $\square$

Lemma 6. *If the call to **OpenToZero** succeeds in Extend labels, then the value $\mathbf{d}$ is correctly opened except with probability at most $\epsilon = \frac{m}{2^\rho}$.*

Proof. Let $\hat{\mathbf{d}} = \mathbf{r} - \text{lsb}(\lambda_{uv}) + \mathbf{E}_d$. If $\mathbf{E}_d \neq 0$, the check will pass if:

$$\mathbf{H}\left(\chi^t(E_{d,t}\delta)\right) + E_h = 0,$$

which happens with probability at most ϵ . $\square$

Lemma 7. *When executing **OpenToZero** in the OpenValue phase of Π_{Prep} , honest parties abort if and only if the adversary sends a different output bit-share λ^j for some corrupted party P_j except with probability $\frac{m_O^i}{2^\rho}$ (where m_O^i denotes the number of output bits to the honest party P_i).*

Proof. Let P_i be the honest party that receives output and let P_j denote a corrupted party. Let λ^j be P_j 's share during an honest protocol execution and $\hat{\lambda}^j$ be the value P_j sent to P_i during the output phase. If $\hat{\lambda}^j = \lambda^j + \mathbf{E}$ with $\mathbf{E} \neq 0$, to make the check still pass, the adversary should have

$$\mathbf{H}(\mathbf{E}\alpha^i) + E_h = 0.$$

Since H is an ϵ -uniform hash function, this happens with probability $\epsilon = \frac{m_O^i}{2^\rho}$. $\square$

Finally, we can use these results, to prove security of the preprocessing.

Let $\mathcal{C}$ be the set of corrupt parties. We construct a simulator $\mathcal{S}$ as follows. For each $i \in \mathcal{C}$, $\mathcal{S}$ receives Δ^i, α^i from $\mathcal{A}$ and forwards them to the functionality. The **RandomBit** step does not require interaction between parties, the simulator emulates $\mathcal{F}_{\text{nVole}}$ with input seed^i from $\mathcal{A}$ and it replies with $\{M_j^i, K_j^i\}_{i \in \mathcal{C}}$. It forwards these values to the functionality. In the **ANDOutputMask**, $\mathcal{S}$ emulates the functionality $\mathcal{F}_{\text{nRandomTriple}}$ and sends the corresponding output shares $x^i, y^i, z^i, \langle x\alpha^i \rangle, \langle y\alpha^i \rangle, \langle z\alpha^i \rangle$ to $\mathcal{A}$. The only step requiring interactions between parties are during the openings of e, f, δ and d . The strategy is similar in these cases, $\mathcal{S}$ forwards random elements to the adversary on behalf of the honest parties and receives corrupted parties' shares from $\mathcal{A}$. Given the output of the emulated functionalities, $\mathcal{S}$ can also compute the correct values the adversary could have sent. If all the error values E are zero, $\mathcal{S}$ runs the **CheckToZero** step honestly on behalf of the honest parties. If some additive error is not zero, $\mathcal{S}$ samples $h \leftarrow \mathbb{F}_{2^\rho}$ and returns random $\{h^i\}_{i \notin \mathcal{C}}$ subject to $h = \sum_{i \notin \mathcal{C}} h^i$ during the commit-and-open phase. $\mathcal{S}$ then aborts. In **OpenValue**, if the party that receives output is honest, $\mathcal{S}$ receives $\hat{\lambda}^j$ from all corrupted parties. Let $E := \sum_{j \in \mathcal{C}} \hat{\lambda}^j - \lambda^j$ be the added error. If $E \neq 0$, $\mathcal{S}$ sends abort to $\mathcal{F}_{\text{Prep}}$, samples a random $h \leftarrow \mathbb{F}_{2^\rho}$ and simulates the honest parties in **OpenToZero** to reconstruct h (as in the previous step). If $E = 0$, $\mathcal{S}$ replies continue to $\mathcal{F}_{\text{Prep}}$. To prove indistinguishability between the real and ideal execution it is enough to note that the opened values are uniformly distributed in both executions. The only point where the ideal and real world differ is during the check: the simulator always abort in case of malicious behaviour, while the real protocol it does the same except with negligible probabilities as we have shown with the previous lemmata.

This concludes the proof.

The communication cost per party to multiply m bits is $m(2\rho + 1) + \mathcal{O}(\kappa + \rho)$ where the small constant cost $\mathcal{O}(\kappa + \rho)$ depends on the implementation of $\mathcal{F}_{\text{Coin}}$ and $\mathcal{F}_{\text{Commit}}$. For example, we can realize $\mathcal{F}_{\text{Coin}}$ by having every party commit-then-open a random seed $s^i \in \mathbb{F}_{2^\kappa}$ and define the output as $s = \bigoplus_{i \in [n]} s^i$. And further, we realize $\mathcal{F}_{\text{Commit}}$ using a collision and pre-image resistant hash function H where $\tau := H(m||r) \in \mathbb{F}_{2^\rho}$ is the commitment to message m with local randomness $r \in \mathbb{F}_{2^\rho}$ and (m, r) are the opening information. Then, the constant cost is $\kappa + 5\rho$ bit since all calls to **OpenToZero** can be batched into a single larger one.

5.2 Instantiating $\mathcal{F}_{\text{nRandomTriple}}$

We instantiate $\mathcal{F}_{\text{nRandomTriple}}$ with sublinear communication using the protocol $\Pi_{\text{nRandomTriple}}$ (see Figure 14). We modify the SPDZ-style multiplication triple PCG described in [AS22] to output triples in our secret-sharing representation instead. First, we briefly summarize the PCG from [AS22].

Let $R := \mathbb{F}_{2^\rho}[X]/(F(X))$ be an appropriate ring where $F(X)$ has degree m . The Module-LPN assumption allows compression of a random element $x \in R$ to a description of size independent of the size of R via c t -sparse polynomials. Let $\omega_1, \dots, \omega_t$ denote the non-zero positions of the values $\beta_1, \dots, \beta_t$ where $\omega_i \in [m]$ and $\beta_i \in \mathbb{F}_{2^\rho}$. From this, to obtain a representation of $x\alpha$ with $\alpha \in \mathbb{F}_{2^\rho}$, it suffices to compute $\alpha \cdot \beta_1, \dots, \alpha \cdot \beta_t$. Further, to obtain $x \cdot y$ where $x, y \in R$, we compute a t^2 -sparse polynomial with values $\beta_i \cdot \gamma_j$ at non-zero position $\omega_i + \eta_j$. Here, ω and β denote the position and value of x , respectively, while η and γ denote the same for y .

Now, in the multi-party setting, we will let each party choose their share of x and y as $x^i, y^i \in R$ by choosing $\omega^i, \beta^i, \eta^i, \gamma^i$ and also their key $\alpha^i \in \mathbb{F}_{2^\rho}$. [AS22] compute $\langle x\alpha \rangle_i$ where $x \in R$ and $\alpha = \sum_{i=1}^n \alpha^i$ as $\langle x\alpha \rangle_i := x^i \alpha^i + \sum_{j \neq i} \langle x^i \alpha^j \rangle_i + \langle x^j \alpha^i \rangle_i$. The term $x^i \alpha^i$ is local to party P_i . Each

$\langle x^i \alpha^j \rangle_i$ and $\langle x^j \alpha^i \rangle_i$ is computed via a 2-party distributed sum of point functions (DSPF) between P_i and P_j . A share of the product $\langle x \cdot y \rangle_i$ is computed as $\langle x \cdot y \rangle := x^i \cdot y^i + \sum_{j \neq i} \langle x^i \cdot y^j \rangle + \langle x^j \cdot y^i \rangle$. Again, $x^i \cdot y^i$ is a local term and $\langle x^i \cdot y^j \rangle_i$ and $\langle x^j \cdot y^i \rangle_i$ are computed via a 2-DSPF between P_i and P_j . Finally, the share of the authentication on the product, $\langle x \cdot y \cdot \alpha \rangle_i$, is computed as $\langle x \cdot y \cdot \alpha \rangle_i := x^i \cdot y^i \cdot \alpha^i + \sum_{(j,k) \neq (i,i)} \langle x^i \cdot y^j \cdot \alpha^k \rangle_i + \langle x^j \cdot y^i \cdot \alpha^i \rangle_i + \langle x^k \cdot y^i \cdot \alpha^j \rangle_i$ where $x^i \cdot y^i \cdot \alpha^i$ is the local term and $\langle x^i \cdot y^j \cdot \alpha^k \rangle_i$ for any i, j, k is computed using a 3-DSPF among parties P_i, P_j and P_k .

In essence, [AS22] computes an almost pairwise authenticated value first which is then summed up. Thus, to obtain our authenticated triples, we omit the summation in the right spots as shown by $\Pi_{\text{nRandomTriple}}$ in Figure 14.

Note. Creating random $\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket x \cdot y \rrbracket$ triples, i.e., with pairwise MACs, instead is more involved using the techniques from [AS22]. In the two-party correlation $M_j^i[z] = K_i^j[z] + z^i \alpha^j$, we observe that computing $z^i \alpha^j$ necessitates obtaining cross-terms $\langle x^i y^k \alpha^j \rangle$ with a third-party P_k . However, the 3-DPF approach returns a three-party sharing and thus P_k would also hold a share of the former two-party VOLE relation between P_i and P_j ! It is not clear how this situation can be resolved with sublinear communication. Moreover, modifying the 3-DPF to return a two-party sharing of $x^i y^k \alpha^j$ to P_i and P_j raises security issues: if P_i and P_j collude, they would have full knowledge of $x^i y^k \alpha^j$, allowing them to potentially infer information about y^k .

Instantiating $\mathcal{F}_{\text{DSPF}}$. The functionality offers generic bit-MPC functionality, i.e., input, multiplication and integer addition, which can be instantiated using any actively secure MPC protocol. Note that since the computed circuits in $\Pi_{\text{nRandomTriple}}$ depend only on the LPN parameters c and t , the selected bit-MPC protocol does not need to be constant-round. The DSPFs can be simply instantiated as the sum of t DPF instances (in parallel) whose output is summed during the full evaluation. [BGIK22] give a constant-round distributed setup for a 2-party DPF which can be made actively secure using actively secure OT and techniques from [BCG⁺20]. To instantiate the 3-party DPF, we use [BK KO20] (since we also work in $\mathbb{F}_{2^k}$) for which the actively secure distributed setup of [AS22] can be adapted.

$\Pi_{\text{nRandomTriple}}$ uses the same number of 2- and 3-party DSPFs as [AS22] that is $2n(n-1)c$ 2-DSPFs with t points and domain size m , $4n(n-1)c^2$ 2-DSPFs with t^2 points and $2m$ domain size and $n(n-1)(n-2)c^2$ 3-DSPFs with t^2 points and domain size $2m$. The asymptotic communication cost for 2-DPFs and 3-DPFs is $O(\log m)$ and $O(\sqrt{m})$, respectively. We use the Ring-LPN parameters from [BCG⁺20] for $\kappa = 128$: $c = 8$ and $t = 4$. Since they consider rings whereas we have fields, this gives conservative parameters as supported by recent cryptanalysis [LWYY24] suggesting that the security level for fields is even higher than what the authors of [BCG⁺20] assumed.

Protocol $\Pi_{\text{nRandomTriple}}$

PARAMETERS: Module-LPN parameters c and t . Let m be the number of triples and n the number of parties. Let $R := \mathbb{F}_{2^\rho}/(F(X))$ be a ring where the polynomial $F(X)$ has degree m .

To generate m triples, all parties do the following.

1. Each party P_i samples $\alpha^i \leftarrow \mathbb{F}_{2^\rho}$ and for all $r \in [c]$, shares $\omega_r^i, \eta_r^i \leftarrow [m]^t$ and $\beta_r^i, \gamma_r^i \leftarrow \mathbb{F}_{2^\rho}^t$.
2. Each P_i calls for all $r \in [c]$ **Input** on $\mathcal{F}_{\text{DSPF}}$ for $\omega_r^i, \eta_r^i, \beta_r^i, \gamma_r^i$ and all parties obtain $[\omega_r^i], [\eta_r^i], [\beta_r^i], [\gamma_r^i]$. Each P_i calls **Input** on $\mathcal{F}_{\text{DSPF}}$ for α^i and all parties obtain $[\alpha^i]$.
3. For each $i, j \in [n]$, $i \neq j$ and for all $r \in [c]$,
 - (a) The parties call **Multiply** on $\mathcal{F}_{\text{DSPF}}$ for $([\beta_r^i], [\alpha^j])$ and for $([\gamma_r^i], [\alpha^j])$, and all parties obtain $[\beta_r^i \alpha^j]$ and $[\gamma_r^i \alpha^j]$.
 - (b) The parties call 2-DSPF on $\mathcal{F}_{\text{DSPF}}$ for $(t, m, i, j, [\omega_r^i], [\beta_r^i \alpha^j])$ and P_i and P_j obtain $\langle \tilde{u}_{i,j,r} \rangle \in \mathbb{F}_{2^\rho}^m$. The parties interpret $\tilde{u}_{i,j,r}$ as an element in R .
 - (c) The parties call 2-DSPF on $\mathcal{F}_{\text{DSPF}}$ for $(t, m, i, j, [\eta_r^i], [\gamma_r^i \alpha^j])$ and P_i and P_j obtain $\langle \tilde{v}_{i,j,r} \rangle \in \mathbb{F}_{2^\rho}^m$. The parties interpret $\tilde{v}_{i,j,r}$ as an element in R .
4. For each $i, j \in [n]$, $i \neq j$ and for all $r, s \in [c]$,
 - (a) The parties call **OuterSum** on $\mathcal{F}_{\text{DSPF}}$ for $([\omega_r^i], [\eta_s^j])$ such that all parties obtain $[\omega_r^i \boxplus \eta_s^j]$.
 - (b) The parties call **OuterProd** on $\mathcal{F}_{\text{DSPF}}$ for $([\beta_r^i], [\gamma_s^j])$ such that all parties obtain $[\beta_r^i \otimes \gamma_s^j]$.
 - (c) The parties call 2-DSPF on $\mathcal{F}_{\text{DSPF}}$ for $(t^2, 2m, i, j, [\omega_r^i \boxplus \eta_s^j], [\beta_r^i \otimes \gamma_s^j])$ such that P_i obtains $\langle w_{i,j,r,s} \rangle_i \in \mathbb{F}_{2^\rho}^{2m}$ and P_j obtains $\langle w_{i,j,r,s} \rangle_j \in \mathbb{F}_{2^\rho}^{2m}$. The parties interpret $w_{i,j,r,s}$ as an element in R (by reducing modulo $F(X)$ locally).
5. For each $i, j, k \in [n]$, (i, j, k) not all equal and for all $r, s \in [c]$, the parties call **Multiply** on $\mathcal{F}_{\text{DSPF}}$ for $([\beta_r^i \otimes \gamma_r^j], [\alpha^k])$ and all parties obtain $[(\beta_r^i \otimes \gamma_r^j) \alpha^k]$.
6. For each $i, j \in [n]$, $i \neq j$ and for all $r, s \in [c]$, the parties call 2-DSPF on $\mathcal{F}_{\text{DSPF}}$
 - (a) for $(t^2, 2m, i, j, [\omega_r^i \boxplus \eta_s^i], [(\beta_r^i \otimes \gamma_s^i) \alpha^j])$,
 - (b) for $(t^2, 2m, i, j, [\omega_r^i \boxplus \eta_s^j], [(\beta_r^i \otimes \gamma_s^j) \alpha^j])$ and
 - (c) for $(t^2, 2m, i, j, [\omega_r^j \boxplus \eta_s^i], [(\beta_r^j \otimes \gamma_s^i) \alpha^j])$,
 and P_i and P_j obtain $\langle \tilde{w}_{i,i,j,r,s} \rangle, \langle \tilde{w}_{i,j,j,r,s} \rangle, \langle \tilde{w}_{j,i,j,r,s} \rangle \in \mathbb{F}_{2^\rho}^{2m}$, respectively. The parties interpret the shares as elements in R (by reducing modulo $F(X)$ locally).
7. For each $i, j, k \in [n]$, $i \neq j, k$, $j \neq k$ and for all $r, s \in [c]$, the parties call 3-DSPF on $\mathcal{F}_{\text{DSPF}}$ for $(t^2, 2m, i, j, k, [\omega_r^i \boxplus \eta_s^j], [(\beta_r^i \otimes \gamma_r^j) \alpha^k])$ such that P_i, P_j and P_k obtain $\langle \tilde{w}_{i,j,k,r,s} \rangle_i, \langle \tilde{w}_{i,j,k,r,s} \rangle_j, \langle \tilde{w}_{i,j,k,r,s} \rangle_k \in \mathbb{F}_{2^\rho}^{2m}$, respectively. The parties interpret $\tilde{w}_{i,j,k,r,s}$ as an element in R (by reducing modulo $F(X)$ locally).
8. The parties agree on a public, random $\mathbf{a} \in R^c$ (using a seed from $\mathcal{F}_{\text{Coin}}$).
9. Each party P_i computes the following locally
 - (a) for $r \in [c]$, $u_r^i := \sum_{\ell=1}^t \beta_r^i[\ell] X^{\omega_r^i[\ell]} \in R$ and $v_r^i := \sum_{\ell=1}^t \gamma_r^i[\ell] X^{\eta_r^i[\ell]} \in R$.
 - (b) compute $x^i := \sum_{r=1}^c a_r u_r^i$ and $y^i := \sum_{r=1}^c a_r v_r^i$.
 - (c) compute $z^i := x^i \cdot y^i + \sum_{r=1}^c \sum_{s=1}^c \sum_{j \neq i} a_r a_s (\langle w_{i,j,r,s} \rangle_i + \langle w_{j,i,r,s} \rangle_i) \in R$.
 - (d) compute $\langle x \alpha^j \rangle_i := x^i \alpha^j + \sum_{j \neq i} \sum_{r=1}^c a_r \langle \tilde{u}_{j,i,r} \rangle_i$ and $\langle y \alpha^j \rangle_i := y^i \alpha^j + \sum_{j \neq i} \sum_{r=1}^c a_r \langle \tilde{v}_{j,i,r} \rangle_i$.
Compute for all $j \in [n]$ $j \neq i$: $\langle x \alpha^j \rangle_i := \sum_{r=1}^c a_r \langle \tilde{u}_{i,j,r} \rangle_i$ and $\langle y \alpha^j \rangle_i := \sum_{r=1}^c a_r \langle \tilde{v}_{i,j,r} \rangle_i$.
 - (e) compute $\langle z \alpha^i \rangle_i := x^i y^i \alpha^i + \sum_{r=1}^c \sum_{s=1}^c a_r a_s \sum_{(j,k) \neq (i,i)}^n \langle \tilde{w}_{j,k,i,r,s} \rangle_i$, and for all $j \in [n]$, $j \neq i$:

$$\langle z \alpha^j \rangle_i := \sum_{r=1}^c \sum_{s=1}^c a_r a_s \left(\langle \tilde{w}_{i,i,j,r,s} \rangle_i + \langle \tilde{w}_{j,i,j,r,s} \rangle_i + \sum_{k=1}^n \langle \tilde{w}_{i,k,j,r,s} \rangle_i \right).$$
- (f) Convert into m correlations (see Section 2.7):
 - $x^i, y^i, z^i \in R$ to $\mathbf{x}^i, \mathbf{y}^i, \mathbf{z}^i \in \mathbb{F}_{2^\rho}^m$;
 - for each $j \in [n]$ and $\tau \in \{x, y, z\}$: convert $\langle \tau \alpha^j \rangle_i \in R$ to $\langle \tau \alpha^j \rangle_i \in \mathbb{F}_{2^\rho}^m$.
10. Each party P_i returns $(\tau^i, \{\langle \tau \alpha^j \rangle\}_{j \in [n]})$ for $\tau \in \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$.

Figure 14: Protocol to generate multiplication triples.

Theorem 5. Let $R := \mathbb{F}_{2^\rho}[X]/(F(X))$ be a ring defined by the degree- m polynomial $F(X)$. If the R^c -LPN $_t$ problem is hard for R, c, t , then the protocol $\Pi_{\text{nRandomTriple}}$ securely realizes $\mathcal{F}_{\text{nRandomTriple}}$ in the $(\mathcal{F}_{\text{DSPF}}, \mathcal{F}_{\text{Coin}})$ -hybrid model against an active adversary that corrupts at most $n-1$ parties. The asymptotic communication cost is $O(n^2 ct \log m + n^2 c^2 t^2 \log(2m) + n^3 c^2 t^2 \sqrt{2m}) + O(\kappa + \rho)$.

Proof. We will first prove correctness. We show that by reaching step 9f, the honest parties compute correct shares $x^i, y^i, z^i, \langle x\alpha^j \rangle_i, \langle y\alpha^j \rangle_i, \langle z\alpha^j \rangle_i \in R$ which satisfy the desired correlation $x \cdot y = z$. Define $x = \sum_{i=1}^n x^i$ and $y = \sum_{i=1}^n y^i$ where $x^i = \sum_{r=1}^c a_r u_r^i$ (and y^i similarly). Then,

$$\begin{aligned}
\sum_{i=1}^n z^i &= \sum_{i=1}^n \left(x^i y^i + \sum_{r,s} \sum_{j \neq i} a_r a_s (\langle \mathbf{w}_{i,j,r,s} \rangle_i + \langle \mathbf{w}_{j,i,r,s} \rangle_i) \right) \\
&= \sum_{i=1}^n x^i y^i + \sum_{i=1}^n \sum_{j \neq i} \sum_{r,s} a_r a_s (\langle \mathbf{w}_{i,j,r,s} \rangle_i + \langle \mathbf{w}_{i,j,r,s} \rangle_j) \\
&= \sum_{i=1}^n x^i y^i + \sum_{i=1}^n \sum_{j \neq i} \sum_{r,s} a_r a_s \sum_{\ell=1}^t \sum_{\ell'=1}^t \beta_r^i[\ell] \cdot \gamma_s^j[\ell'] X^{\omega_r^i[\ell] + \eta_s^j[\ell']} \\
&= \sum_{i=1}^n x^i y^i + \sum_{i=1}^n \sum_{j \neq i} \sum_{r,s} a_r a_s \left(\sum_{\ell=1}^t \beta_r^i[\ell] X^{\omega_r^i[\ell]} \right) \cdot \left(\sum_{\ell'=1}^t \gamma_s^j[\ell'] X^{\eta_s^j[\ell']} \right) \\
&= \sum_{i=1}^n x^i y^i + \sum_{i=1}^n \sum_{j \neq i} \left(\sum_{r=1}^c a_r \sum_{\ell=1}^t \beta_r^i[\ell] X^{\omega_r^i[\ell]} \right) \cdot \left(\sum_{s=1}^c a_s \sum_{\ell'=1}^t \gamma_s^j[\ell'] X^{\eta_s^j[\ell']} \right) \\
&= \sum_{i=1}^n x^i y^i + \sum_{i=1}^n \sum_{j \neq i} x^i y^j = x \cdot y.
\end{aligned}$$

Now, for $\langle x\alpha^i \rangle$ (and similarly for $\langle y\alpha^i \rangle$) the following holds:

$$\begin{aligned}
\sum_{j=1}^n \langle x\alpha^i \rangle_j &= \langle x\alpha^i \rangle_i + \sum_{j \neq i} \langle x\alpha^i \rangle_j = x^i \alpha^i + \sum_{j \neq i} \sum_{r=1}^c a_r \langle \tilde{u}_{j,i,r} \rangle_i + \sum_{j \neq i} \sum_{r=1}^c a_r \langle \tilde{u}_{j,i,r} \rangle_j \\
&= x^i \alpha^i + \sum_{j \neq i} \sum_{r=1}^c a_r \sum_{\ell=1}^t \beta_r^j[\ell] \cdot \alpha^i X^{\omega_r^j[\ell]} \\
&= x^i \alpha^i + \sum_{j \neq i} x^j \alpha^i = x \alpha^i.
\end{aligned}$$

Finally, for $\langle z\alpha^i \rangle$ we have:

$$\begin{aligned}
\sum_{j=1}^n \langle z\alpha^i \rangle_j &= \langle z\alpha^i \rangle_i + \sum_{j \neq i} \langle z\alpha^i \rangle_j \\
&= x^i y^i \alpha^i + \sum_{r=1}^c \sum_{s=1}^c a_r a_s \left(\sum_{(j,k) \neq (i,i)}^n \langle \tilde{w}_{j,k,i,r,s} \rangle_i \right. \\
&\quad \left. + \sum_{j \neq i} (\langle \tilde{w}_{j,j,i,r,s} \rangle_j + \langle \tilde{w}_{i,j,i,r,s} \rangle_j + \sum_{k=1}^n \langle \tilde{w}_{j,k,i,r,s} \rangle_j) \right)
\end{aligned}$$

$$\begin{aligned}
&= \dots \\
&= x^i y^i \alpha^i + \sum_{j \neq i} (x^j y^j \alpha^i + x^i y^j \alpha^i + \sum_{k=1}^n x^j y^k \alpha^i) \\
&= \sum_{j=1}^n x^j y^j \alpha^i + \sum_{(j,k) \neq (i,i)} x^j y^k \alpha^i = xy \alpha^i.
\end{aligned}$$

Note that, apart from communication with $\mathcal{F}_{\text{DSPF}}$, there is no communication between the parties. Thus our simulator proceeds as follows. It waits to receive α^j for each $j \in \mathcal{C}$ and forwards them to $\mathcal{F}_{\text{nRandomTriple}}$. It chooses $\alpha^i \leftarrow \mathbb{F}_{2^p}$ for the honest parties $i \notin \mathcal{C}$. The simulator then emulates $\mathcal{F}_{\text{DSPF}}$ and $\mathcal{F}_{\text{Coin}}$ and executes the protocol on behalf of the simulated honest parties (and aborts if $\mathcal{F}_{\text{DSPF}}$ would abort). Since it knows all the inputs of the corrupted parties to $\mathcal{F}_{\text{DSPF}}$ and $\mathcal{F}_{\text{Coin}}$, it can compute x^j, y^j, z^j along with $\{\langle x\alpha^i \rangle_j, \langle y\alpha^i \rangle_j, \langle z\alpha^i \rangle_j\}_{i \in [n]}$ for all corrupted parties $j \in \mathcal{C}$. The simulator then sends $x^j, y^j, z^j, \{\langle x\alpha^i \rangle_j, \langle y\alpha^i \rangle_j, \langle z\alpha^i \rangle_j\}_{i \in [n]}$ as output shares to $\mathcal{F}_{\text{nRandomTriple}}$ on behalf of party P_j .

Indistinguishability between the real protocol and the simulation can be argued via a reduction to the hardness of $R^c\text{-LPN}_t$ using an approach similar to [AS22]. In summary, we use a series of hybrids. Initially, the shares of all honest parties are computed according to the protocol. With each hybrid step, we replace one honest party's share with a randomly sampled value instead of computing it according to the protocol. The final hybrid therefore is equivalent to $\mathcal{F}_{\text{nRandomTriple}}$ (which chooses all honest parties' shares at random). Let P_i be some honest party whose shares will be switched from computed to random in the hybrid step, then the positions ω^i and values β^i of the share x^i can be viewed as challenge for the $R^c\text{-LPN}_t$ distinguisher. Any guesses on ω^i are forwarded to the LPN challenger. Therefore, if the adversary efficiently distinguishes between these steps, it would successfully distinguish $R^c\text{-LPN}_t$. $\square$

Acknowledgments. This work was partly supported by the Flemish Government through FWO SBO project MOZAIK S003321N, by CyberSecurity Research Flanders with reference number VR20192203 and by the FWO under an Odysseus project GOH9718N.

References

- AOR⁺19. Abdelrahman Aly, Emmanuela Orsini, Dragos Rotaru, Nigel P. Smart, and Tim Wood. Zaphod: Efficiently combining LSSS and garbled circuits in SCALE. In Michael Brenner, Tancrède Lepoint, and Kurt Rohloff, editors, *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography, WAHC@CCS 2019, London, UK, November 11-15, 2019*, pages 33–44. ACM, 2019.
- AS22. Damiano Abram and Peter Scholl. Low-communication multiparty triple generation for SPDZ from ring-LPN. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022, Part I*, volume 13177 of *LNCS*, pages 221–251. Springer, Cham, March 2022.
- BBC⁺24. Maxime Bombar, Dung Bui, Geoffroy Couteau, Alain Couvreur, Clément Ducros, and Sacha Servan-Schreiber. FOLEAGE: $\mathbb{F}_4\text{OLE}$ -based multi-party computation for boolean circuits. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part VI*, volume 15489 of *LNCS*, pages 69–101. Springer, Singapore, December 2024.
- BCG⁺20. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudo-random correlation generators from ring-LPN. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 387–416. Springer, Cham, August 2020.

- BCO⁺21. Aner Ben-Efraim, Kelong Cong, Eran Omri, Emmanuela Orsini, Nigel P. Smart, and Eduardo Soria-Vazquez. Large scale, actively secure computation from LPN and free-XOR garbled circuits. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part III*, volume 12698 of *LNCS*, pages 33–63. Springer, Cham, October 2021.
- Bea92. Donald Beaver. Efficient multiparty protocols using circuit randomization. In Joan Feigenbaum, editor, *CRYPTO’91*, volume 576 of *LNCS*, pages 420–432. Springer, Berlin, Heidelberg, August 1992.
- BEBB⁺24. Aner Ben-Efraim, Lior Breitman, Jonathan Bronshtein, Olga Nissenbaum, and Eran Omri. MYao: Multiparty “yao” garbled circuits with row reduction, half gates, and efficient online computation. Cryptology ePrint Archive, Paper 2024/1430, 2024.
- BGH⁺23. Gabrielle Beck, Aarushi Goel, Aditya Hegde, Abhishek Jain, Zhengzhong Jin, and Gabriel Kaptchuk. Scalable multiparty garbling. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 2158–2172. ACM Press, November 2023.
- BGIK22. Elette Boyle, Niv Gilboa, Yuval Ishai, and Victor I. Kolobov. Programmable distributed point functions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 121–151. Springer, Cham, August 2022.
- BKKO20. Paul Bunn, Jonathan Katz, Eyal Kushilevitz, and Rafail Ostrovsky. Efficient 3-party distributed ORAM. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 215–232. Springer, Cham, September 2020.
- BLN⁺21. Sai Sheshank Burra, Enrique Larraia, Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, Emmanuela Orsini, Peter Scholl, and Nigel P. Smart. High-performance multi-party computation for binary circuits based on oblivious transfer. *Journal of Cryptology*, 34(3):34, July 2021.
- BLO16. Aner Ben-Efraim, Yehuda Lindell, and Eran Omri. Optimizing semi-honest secure multiparty computation for the internet. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 578–590. ACM Press, October 2016.
- BMR90. Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *22nd ACM STOC*, pages 503–513. ACM Press, May 1990.
- Can01. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- CDD⁺16. Ignacio Cascudo, Ivan Damgård, Bernardo David, Nico Döttling, and Jesper Buus Nielsen. Rate-1, linear time and additively homomorphic UC commitments. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 179–207. Springer, Berlin, Heidelberg, August 2016.
- CLL⁺14. Shan Chen, Rodolphe Lampe, Jooyoung Lee, Yannick Seurin, and John P. Steinberger. Minimizing the two-round Even-Mansour cipher. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 39–56. Springer, Berlin, Heidelberg, August 2014.
- CT21. Yu Long Chen and Stefano Tessaro. Better security-efficiency trade-offs in permutation-based two-party computation. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part II*, volume 13091 of *LNCS*, pages 275–304. Springer, Cham, December 2021.
- CWYY23. Hongrui Cui, Xiao Wang, Kang Yang, and Yu Yu. Actively secure half-gates with minimum overhead under duplex networks. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 35–67. Springer, Cham, April 2023.
- DILO22. Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. Authenticated garbling from simple correlations. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 57–87. Springer, Cham, August 2022.
- DPSZ12. Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662. Springer, Berlin, Heidelberg, August 2012.
- Gal19. Galois, Inc. swanky: A suite of rust libraries for secure computation. <https://github.com/GaloisInc/swanky>, 2019.
- GI14. Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 640–658. Springer, Berlin, Heidelberg, May 2014.
- GKW⁺20. Chun Guo, Jonathan Katz, Xiao Wang, Chenkai Weng, and Yu Yu. Better concrete security for half-gates garbling (in the multi-instance setting). In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 793–822. Springer, Cham, August 2020.

- GKWW20. Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841. IEEE Computer Society Press, May 2020.
- HK21. David Heath and Vladimir Kolesnikov. One hot garbling. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 574–593. ACM Press, November 2021.
- HSS17. Carmit Hazay, Peter Scholl, and Eduardo Soria-Vazquez. Low cost constant round MPC combining BMR and oblivious transfer. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 598–628. Springer, Cham, December 2017.
- HSS20. Carmit Hazay, Peter Scholl, and Eduardo Soria-Vazquez. Low cost constant round MPC combining BMR and oblivious transfer. *Journal of Cryptology*, 33(4):1732–1786, October 2020.
- KRRW18. Jonathan Katz, Samuel Ranellucci, Mike Rosulek, and Xiao Wang. Optimizing authenticated garbling for faster secure two-party computation. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 365–391. Springer, Cham, August 2018.
- KS08. Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz, editors, *ICALP 2008, Part II*, volume 5126 of *LNCS*, pages 486–498. Springer, Berlin, Heidelberg, July 2008.
- KY18. Marcel Keller and Avishay Yanai. Efficient maliciously secure multiparty computation for RAM. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 91–124. Springer, Cham, April / May 2018.
- LP12. Yehuda Lindell and Benny Pinkas. Secure two-party computation via cut-and-choose oblivious transfer. *Journal of Cryptology*, 25(4):680–722, October 2012.
- LPSY19. Yehuda Lindell, Benny Pinkas, Nigel P. Smart, and Avishay Yanai. Efficient constant-round multi-party computation combining BMR and SPDZ. *Journal of Cryptology*, 32(3):1026–1069, July 2019.
- LSS16. Yehuda Lindell, Nigel P. Smart, and Eduardo Soria-Vazquez. More efficient constant-round multi-party computation from BMR and SHE. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part I*, volume 9985 of *LNCS*, pages 554–581. Springer, Berlin, Heidelberg, October / November 2016.
- LWYY24. Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. The hardness of LPN over any integer ring and field for PCG applications. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 149–179. Springer, Cham, May 2024.
- LXYY25. Zhe Li, Chaoping Xing, Yizhou Yao, and Chen Yuan. Efficient pseudorandom correlation generators for any finite field. Cryptology ePrint Archive, Paper 2025/169, 2025.
- NO09. Jesper Buus Nielsen and Claudio Orlandi. LEGO for two-party secure computation. In Omer Reingold, editor, *TCC 2009*, volume 5444 of *LNCS*, pages 368–386. Springer, Berlin, Heidelberg, March 2009.
- NPS99. Moni Naor, Benny Pinkas, and Reuben Sumner. Privacy preserving auctions and mechanism design. In *Proceedings of the 1st ACM Conference on Electronic Commerce*, EC '99, page 129–139, New York, NY, USA, 1999. Association for Computing Machinery.
- NST17. Jesper Buus Nielsen, Thomas Schneider, and Roberto Trifiletti. Constant round maliciously secure 2PC with function-independent preprocessing using LEGO. In *NDSS 2017*. The Internet Society, February / March 2017.
- PSSW09. Benny Pinkas, Thomas Schneider, Nigel P. Smart, and Stephen C. Williams. Secure two-party computation is practical. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 250–267. Springer, Berlin, Heidelberg, December 2009.
- RR21. Mike Rosulek and Lawrence Roy. Three halves make a whole? Beating the half-gates lower bound for garbled circuits. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 94–124, Virtual Event, August 2021. Springer, Cham.
- RS22. Rahul Rachuri and Peter Scholl. Le mans: Dynamic and fluid MPC for dishonest majority. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 719–749. Springer, Cham, August 2022.
- sS11. abhi shelat and Chih-Hao Shen. Two-output secure computation with malicious adversaries. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 386–405. Springer, Berlin, Heidelberg, May 2011.
- WRK17a. Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Authenticated garbling and efficient maliciously secure two-party computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 21–37. ACM Press, October / November 2017.

- WRK17b. Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Global-scale secure multiparty computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 39–56. ACM Press, October / November 2017.
- WYKW21. Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091. IEEE Computer Society Press, May 2021.
- Yao86. Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.
- YWZ20. Kang Yang, Xiao Wang, and Jiang Zhang. More efficient MPC from improved triple generation and authenticated garbling. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1627–1646. ACM Press, November 2020.
- ZRE15. Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole - reducing data transfer in garbled circuits using half gates. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 220–250. Springer, Berlin, Heidelberg, April 2015.